

Modules

Final Module	Original Modules
Identity & Access Management	Registration, Login, RBAC
Student Management	Student Management
Faculty Management	Faculty Management
Subject Management	Subject Management
Examination Administration	Scheduling, Candidate Registration
Question Management	Question Bank, Random Selection, Paper Generation
Examination Runtime	Examination Engine, Online Interface, Timer, Auto Submission
Evaluation & Results	Evaluation, Marks Calculation, Result Processing
Reporting & Analytics	Reports, Dashboard, Analytics
Audit Logging	Audit Logs
Secure Session Management	Secure Session Management
Authenticated Network Communication	Authenticated Network Communication
Backup & Restore	Backup & Restore
System Monitoring	System Monitoring
Notification Service	Notification Module

TablesWithAttributsConstraints

Perfect. We'll make this **implementation-ready**, not conceptual.

One important note before we start:

During **DB-003**, we identified that **RolePermission** is required as the junction table for RBAC. Therefore, the Identity & Access domain consists of **6 tables**:

- 1. Role
- 2. Permission
- 3. RolePermission
- 4. User
- 5. UserSession
- 6. LoginHistory

PART 1 — Identity & Access Database Specification

Table 1 — Role

Purpose

Stores all system roles used for Role-Based Access Control (RBAC).

Columns

Column	Data Type	Null	Default
roleId	BIGSERIAL	NO	Auto
roleName	VARCHAR(50)	NO	—
description	VARCHAR(255)	YES	NULL
isActive	BOOLEAN	NO	TRUE
createdAt	TIMESTAMPTZ	NO	NOW()

updatedAt	TIMESTAMPTZ	NO	NOW()
createdBy	BIGINT	YES	NULL
updatedBy	BIGINT	YES	NULL

Primary Key

- roleId

Foreign Keys

None

Unique Constraints

- roleName

Check Constraints

CHAR_LENGTH(roleName) >= 3

Indexes

PK(roleId)

UNIQUE(roleName)

Delete Rule

RESTRICT

Table 2 — Permission

Purpose

Stores all available permissions in the system.

Columns

Column	Data Type	Null	Default
permissionId	BIGSERIAL	NO	Auto
permissionName	VARCHAR(100)	NO	—
moduleName	VARCHAR(50)	NO	—
description	VARCHAR(255)	YES	NULL
createdAt	TIMESTAMPTZ	NO	NOW()

Primary Key

permissionId

Foreign Keys

None

Unique Constraints

(permissionName, moduleName)

Check Constraints

CHAR_LENGTH(permissionName) > 2

Indexes

PK(permissionId)

UNIQUE(permissionName,moduleName)

Delete Rule

RESTRICT

Table 3 — RolePermission

Purpose

Maps permissions to roles.

Columns

Column	Data Type	Null	Default
rolePermissionId	BIGSERIAL	NO	Auto
roleId	BIGINT	NO	—
permissionId	BIGINT	NO	—
grantedAt	TIMESTAMPTZ	NO	NOW()
grantedBy	BIGINT	YES	NULL

Primary Key

rolePermissionId

Foreign Keys

roleId → Role(roleId)

permissionId → Permission(permissionId)

grantedBy → User(userId)

Unique Constraints

(roleId, permissionId)

Check Constraints

None

Indexes

PK(rolePermissionId)

INDEX(roleId)

INDEX(permissionId)

UNIQUE(roleId,permissionId)

Delete Rules

Role → CASCADE

Permission → CASCADE

Table 4 — User

Purpose

Stores every authenticated user of the system.

Columns

Column	Data Type	Null	Default
userId	BIGSERIAL	NO	Auto
firstName	VARCHAR(50)	NO	—
lastName	VARCHAR(50)	NO	—
email	VARCHAR(255)	NO	—
phone	VARCHAR(15)	YES	NULL
passwordHash	TEXT	NO	—
roleId	BIGINT	NO	—
isActive	BOOLEAN	NO	TRUE
lastLogin	TIMESTAMPTZ	YES	NULL
failedLoginAttempts	INTEGER	NO	0
accountLocked	BOOLEAN	NO	FALSE
createdAt	TIMESTAMPTZ	NO	NOW()
updatedAt	TIMESTAMPTZ	NO	NOW()

createdBy	BIGINT	YES	NULL
updatedBy	BIGINT	YES	NULL

Primary Key

userId

Foreign Keys

roleId → Role(roleId)

createdBy → User(userId)

updatedBy → User(userId)

Unique Constraints

email

phone

Check Constraints

failedLoginAttempts >= 0

Indexes

PK(userId)

UNIQUE(email)

UNIQUE(phone)

INDEX(roleId)

INDEX(lastLogin)

Delete Rule

RESTRICT

Table 5 — UserSession

Purpose

Maintains active and historical login sessions.

Columns

Column	Data Type	Null	Default
sessionId	UUID	NO	gen_random_uuid()
userId	BIGINT	NO	—
loginTime	TIMESTAMPTZ	NO	NOW()
logoutTime	TIMESTAMPTZ	YES	NULL
ipAddress	INET	NO	—
userAgent	TEXT	YES	NULL
deviceType	VARCHAR(30)	YES	NULL
sessionStatus	VARCHAR(20)	NO	'ACTIVE'
expiresAt	TIMESTAMPTZ	NO	—

Primary Key

sessionId

Foreign Keys

userId → User(userId)

Unique Constraints

None

Check Constraints

sessionStatus IN ('ACTIVE','EXPIRED','LOGGED_OUT','INVALIDATED')

Indexes

PK(sessionId)

INDEX(userId)

INDEX(loginTime)

INDEX(expiresAt)

INDEX(sessionStatus)

Delete Rule

CASCADE

Table 6 — LoginHistory

Purpose

Maintains immutable authentication history for auditing and security analysis.

Columns

Column	Data Type	Null	Default
loginHistoryId	BIGSERIAL	NO	Auto
userId	BIGINT	NO	—
loginTime	TIMESTAMPTZ	NO	NOW()
logoutTime	TIMESTAMPTZ	YES	NULL
ipAddress	INET	NO	—
userAgent	TEXT	YES	NULL
loginStatus	VARCHAR(20)	NO	—
failureReason	VARCHAR(255)	YES	NULL

authenticationMethod VARCHAR(30) NO 'PASSWORD'

Primary Key

loginHistoryId

Foreign Keys

userId → User(userId)

Unique Constraints

None

Check Constraints

loginStatus IN
('SUCCESS','FAILED','LOCKED')

authenticationMethod IN
('PASSWORD','OTP','SSO')

Indexes

PK(loginHistoryId)

INDEX(userId)

INDEX(loginTime)

INDEX(loginStatus)

Delete Rule

RESTRICT (audit records should never be removed)

Identity & Access Summary

Table

Type

Role	Master
Permission	Master
RolePermission	Mapping
User	Master
UserSession	Transaction
LoginHistory	Audit

Notes / Refinements

While reviewing the schema, I identified a few refinements that I recommend we consider before freezing the database design:

1. **User vs Student/Faculty specialization** – Earlier we discussed a single **User** entity with role-based differentiation. We should later decide whether student- and faculty-specific attributes belong in separate profile tables or remain within a unified user model.
2. **Permission granularity** – We should decide whether permissions are simple strings (e.g., **CREATE_EXAM**) or resource-action pairs (**Exam:Create**). The latter scales better for RBAC.
3. **Session storage** – Using a **UUID** for **sessionId** is preferable over **BIGSERIAL** because session identifiers should be unpredictable from a security standpoint.

Overall, this part provides a solid implementation-ready foundation for the Identity & Access domain and can be translated directly into PostgreSQL DDL in a later implementation phase.

PART 2 — Academic & Examination Management Database Specification

This section covers the following tables:

1. Subject
 2. Exam
 3. ExamSchedule
 4. CandidateRegistration
-

Table 7 — Subject

Purpose

Stores all academic subjects for which examinations are conducted.

Columns

Column	Data Type	Null	Default
subjectId	BIGSERIAL	NO	Auto
subjectCode	VARCHAR(20)	NO	—
subjectName	VARCHAR(100)	NO	—
description	TEXT	YES	NULL
credits	SMALLINT	YES	NULL
isActive	BOOLEAN	NO	TRUE
createdAt	TIMESTAMPTZ	NO	NOW()
updatedAt	TIMESTAMPTZ	NO	NOW()
createdBy	BIGINT	YES	NULL
updatedBy	BIGINT	YES	NULL

Primary Key

- subjectId

Foreign Keys

- createdBy → User(userId)
- updatedBy → User(userId)

Unique Constraints

- subjectCode
- subjectName

Check Constraints

credits >= 0

Indexes

PK(subjectId)

UNIQUE(subjectCode)

UNIQUE(subjectName)

INDEX(isActive)

Delete Rule

RESTRICT

Table 8 — Exam

Purpose

Stores the master definition of an examination.

Columns

Column	Data Type	Null	Default
examId	BIGSERIAL	NO	Auto
subjectId	BIGINT	NO	—
examCode	VARCHAR(30)	NO	—
examTitle	VARCHAR(150)	NO	—
examType	VARCHAR(30)	NO	—
totalMarks	NUMERIC(6,2)	NO	—
passingMarks	NUMERIC(6,2)	NO	—
durationMinutes	INTEGER	NO	—
instructions	TEXT	YES	NULL
maximumAttempts	SMALLINT	NO	1
shuffleQuestions	BOOLEAN	NO	TRUE
shuffleOptions	BOOLEAN	NO	TRUE
negativeMarking	BOOLEAN	NO	FALSE
negativeMarksPerQuestion	NUMERIC(5,2)	NO	0
examStatus	VARCHAR(20)	NO	'DRAFT'
isActive	BOOLEAN	NO	TRUE
createdAt	TIMESTAMPTZ	NO	NOW()
updatedAt	TIMESTAMPTZ	NO	NOW()
createdBy	BIGINT	YES	NULL
updatedBy	BIGINT	YES	NULL

Primary Key

- examId

Foreign Keys

- subjectId → Subject(subjectId)
- createdBy → User(userId)
- updatedBy → User(userId)

Unique Constraints

- examCode

Check Constraints

totalMarks > 0

passingMarks >= 0

passingMarks <= totalMarks

durationMinutes > 0

maximumAttempts >= 1

negativeMarksPerQuestion >= 0

examStatus IN
('DRAFT','SCHEDULED','ACTIVE','COMPLETED','CANCELLED')

Indexes

PK(examId)

UNIQUE(examCode)

INDEX(subjectId)

INDEX(examStatus)

INDEX(isActive)

Delete Rule

RESTRICT

Table 9 — ExamSchedule

Purpose

Stores scheduling information for each examination.

Columns

Column	Data Type	Null	Default
scheduleId	BIGSERIAL	NO	Auto
examId	BIGINT	NO	—
startTime	TIMESTAMPTZ	NO	—
endTime	TIMESTAMPTZ	NO	—
registrationStart	TIMESTAMPTZ	YES	NULL
registrationEnd	TIMESTAMPTZ	YES	NULL
lateEntryMinutes	INTEGER	NO	0
autoSubmit	BOOLEAN	NO	TRUE
scheduleStatus	VARCHAR(20)	NO	'SCHEDULED'
createdAt	TIMESTAMPTZ	NO	NOW()
updatedAt	TIMESTAMPTZ	NO	NOW()

Primary Key

- scheduleId

Foreign Keys

- examId → Exam(examId)

Unique Constraints

None

Check Constraints

endTime > startTime

registrationEnd >= registrationStart

lateEntryMinutes >= 0

scheduleStatus IN
('SCHEDULED','ONGOING','COMPLETED','CANCELLED')

Indexes

PK(scheduleId)

INDEX(examId)

INDEX(startTime)

INDEX(endTime)

INDEX(scheduleStatus)

Delete Rule

CASCADE

Table 10 — CandidateRegistration

Purpose

Stores student registrations for examinations.

Columns

Column	Data Type	Null	Default
registrationId	BIGSERIAL	NO	Auto
examId	BIGINT	NO	—
userId	BIGINT	NO	—

registrationTime	TIMESTAMPTZ	NO	NOW()
registrationStatus	VARCHAR(20)	NO	'REGISTERED'
eligibilityVerified	BOOLEAN	NO	FALSE
paymentRequired	BOOLEAN	NO	FALSE
paymentStatus	VARCHAR(20)	NO	'NOT_REQUIRED'
remarks	VARCHAR(255)	YES	NULL
createdAt	TIMESTAMPTZ	NO	NOW()

Primary Key

- registrationId

Foreign Keys

- examId → Exam(examId)
- userId → User(userId)

Unique Constraints

- (examId, userId)

Check Constraints

registrationStatus IN
('REGISTERED','CANCELLED','WAITLISTED')

paymentStatus IN
('NOT_REQUIRED','PENDING','PAID','FAILED')

Indexes

PK(registrationId)

UNIQUE(examId,userId)

INDEX(userId)

INDEX(examId)

INDEX(registrationStatus)

INDEX(paymentStatus)

Delete Rule

- Exam → RESTRICT if registrations exist
 - User → RESTRICT
-

Academic & Examination Management Summary

Table	Type
Subject	Master
Exam	Master
ExamSchedule	Transaction
CandidateRegistration	Transaction

Refinements Identified

While reviewing this part of the schema, I identified a few improvements that I recommend we incorporate before implementation:

1. Separate Exam Definition from Exam Instance

Currently, **Exam** stores both the exam definition and scheduling-related configuration. If the system later supports multiple sessions of the same exam (e.g., different batches or makeup exams), we may consider moving session-specific settings (such as **shuffleQuestions**,

`shuffleOptions`, and `negativeMarking`) into `ExamSchedule` or a separate `ExamSession` table. For the current project scope, the present design is sufficient.

2. Registration Eligibility

If eligibility rules become more complex (attendance, prerequisites, semester restrictions), they should be handled by business logic rather than additional columns. The `eligibilityVerified` flag simply records the outcome.

3. Future-Proofing

The current schema can accommodate future additions such as:

- Multiple examination sessions
- Registration approval workflow
- Hall ticket generation
- Fee management
- Different examination modes (online/offline/hybrid)

without requiring major redesign.

Part 2 Complete

The next section (**Part 3**) will cover the **Question Bank**, including:

1. Question
2. QuestionCategory
3. DifficultyLevel
4. ExamQuestion

This is where the core examination content model will be defined.

PART 3 — Question Bank Database Specification

This section covers the following tables:

1. QuestionCategory
2. DifficultyLevel
3. Question
4. ExamQuestion

Note: I intentionally ordered the tables based on dependencies so that parent tables are defined before child tables.

Table 11 — QuestionCategory

Purpose

Stores the classification of questions (e.g., Programming, DBMS, Networking).

Columns

Column	Data Type	Null	Default
categoryId	BIGSERIAL	NO	Auto
categoryName	VARCHAR(100)	NO	—
description	VARCHAR(255)	YES	NULL
isActive	BOOLEAN	NO	TRUE
createdAt	TIMESTAMPTZ	NO	NOW()
updatedAt	TIMESTAMPTZ	NO	NOW()
createdBy	BIGINT	YES	NULL
updatedAtBy	BIGINT	YES	NULL

Primary Key

- categoryId

Foreign Keys

- createdBy → User(userId)
- updatedBy → User(userId)

Unique Constraints

- categoryName

Check Constraints

CHAR_LENGTH(categoryName) >= 3

Indexes

PK(categoryId)

UNIQUE(categoryName)

INDEX(isActive)

Delete Rule

RESTRICT

Table 12 — DifficultyLevel

Purpose

Stores standardized difficulty levels used throughout the question bank.

Columns

Column	Data Type	Null	Default
difficultyLevelId	BIGSERIAL	NO	Auto
levelName	VARCHAR(30)	NO	—

difficultyScore	SMALLINT	NO	—
description	VARCHAR(255)	YES	NULL
createdAt	TIMESTAMPTZ	NO	NOW()

Primary Key

- difficultyLevelId

Foreign Keys

None

Unique Constraints

- levelName
- difficultyScore

Check Constraints

difficultyScore BETWEEN 1 AND 10

Indexes

PK(difficultyLevelId)

UNIQUE(levelName)

UNIQUE(difficultyScore)

Delete Rule

RESTRICT

Table 13 — Question

Purpose

Stores every question available in the centralized Question Bank.

Columns

Column	Data Type	Null	Default
questionId	BIGSERIAL	NO	Auto
categoryId	BIGINT	NO	—
difficultyLevelId	BIGINT	NO	—
questionType	VARCHAR(30)	NO	—
questionText	TEXT	NO	—
optionA	TEXT	YES	NULL
optionB	TEXT	YES	NULL
optionC	TEXT	YES	NULL
optionD	TEXT	YES	NULL
correctOption	CHAR(1)	YES	NULL
correctAnswer	TEXT	YES	NULL
explanation	TEXT	YES	NULL
defaultMarks	NUMERIC(5,2)	NO	1
negativeMarks	NUMERIC(5,2)	NO	0
estimatedTimeSeconds	INTEGER	YES	NULL
isActive	BOOLEAN	NO	TRUE
createdAt	TIMESTAMPTZ	NO	NOW()
updatedAt	TIMESTAMPTZ	NO	NOW()
createdBy	BIGINT	YES	NULL
updatedBy	BIGINT	YES	NULL

Primary Key

- questionId

Foreign Keys

- categoryId → QuestionCategory(categoryId)
- difficultyLevelId → DifficultyLevel(difficultyLevelId)
- createdBy → User(userId)
- updatedBy → User(userId)

Unique Constraints

None

Check Constraints

questionType IN
('MCQ', 'MSQ', 'TRUE_FALSE', 'SHORT_ANSWER', 'DESCRIPTIVE', 'CODING')

defaultMarks > 0

negativeMarks >= 0

estimatedTimeSeconds > 0

correctOption IN ('A', 'B', 'C', 'D')

Indexes

PK(questionId)

INDEX(categoryId)

INDEX(difficultyLevelId)

INDEX(questionType)

INDEX(isActive)

Delete Rule

RESTRICT

Table 14 — ExamQuestion

Purpose

Associates questions with examinations and stores exam-specific configuration.

Columns

Column	Data Type	Null	Default
examQuestionId	BIGSERIAL	NO	Auto
examId	BIGINT	NO	—
questionId	BIGINT	NO	—
questionOrder	INTEGER	NO	—
marks	NUMERIC(5,2)	NO	—
negativeMarks	NUMERIC(5,2)	NO	0
isMandatory	BOOLEAN	NO	TRUE
createdAt	TIMESTAMPTZ	NO	NOW()

Primary Key

- examQuestionId

Foreign Keys

- examId → Exam(examId)
- questionId → Question(questionId)

Unique Constraints

- (examId, questionId)
- (examId, questionOrder)

Check Constraints

questionOrder > 0

marks > 0

negativeMarks >= 0

Indexes

PK(examQuestionId)

UNIQUE(examId,questionId)

UNIQUE(examId,questionOrder)

INDEX(questionId)

INDEX(examId)

Delete Rule

- Exam → CASCADE
 - Question → RESTRICT
-

Question Bank Summary

Table	Type
QuestionCategory	Master
DifficultyLevel	Master
Question	Master
ExamQuestion	Mapping

Important Engineering Refinements

While reviewing the schema, I identified several improvements that I strongly recommend before freezing the database.

Refinement 1 — Separate Question Options into a Dedicated Table

Instead of storing:

optionA
optionB
optionC
optionD
correctOption

I recommend introducing:

QuestionOption

optionId
questionId
optionText
optionOrder
isCorrect

Why?

- Supports any number of options (not limited to four).
- Better normalization (1NF).
- Easier to implement MSQ (Multiple Select Questions).
- Simpler randomization of options.
- Cleaner UI rendering.
- Future support for image/audio options.

This aligns much better with our DB-005 normalization strategy.

Refinement 2 — Question Versioning

Rather than editing questions directly, consider adding:

versionNumber
isLatestVersion
parentQuestionId

This preserves historical integrity for exams that have already been conducted.

Refinement 3 — Media Support

Instead of embedding media URLs inside the Question table, introduce a future table:

QuestionMedia

mediaId
questionId
mediaType
filePath
displayOrder

This supports:

- Images
- Audio
- Video
- PDFs
- Diagrams

without changing the Question schema.

Refinement 4 — Question Metadata

Useful future additions include:

- Bloom's Taxonomy Level
- Topic
- Subtopic
- Learning Outcome
- Language
- Estimated Difficulty Score (AI-generated)
- Usage Count (computed/cached)

These are not required now but fit naturally into the design later.

Part 3 Status: Complete

At this point, the **Question Bank** is fully modeled and supports:

- Categorized questions
- Difficulty classification
- Multiple question types
- Exam-specific question configuration
- Future extensibility for richer content

The next section, **Part 4**, will define the core examination execution workflow:

1. ExamAttempt
2. Answer
3. Evaluation
4. EvaluationDetail
5. Result

This is the transactional heart of the Online Examination Platform, where examination delivery, submission, evaluation, and result generation are modeled.

PART 4 — Examination Delivery, Evaluation & Result Database Specification

This section covers the following tables:

1. ExamAttempt
2. Answer
3. Evaluation
4. EvaluationDetail
5. Result

This is the core transactional subsystem of the Online Examination Platform. These tables are responsible for recording the student's examination session, storing responses, evaluating submissions, and publishing results while maintaining consistency, integrity, and auditability.

Table 15 — ExamAttempt

Purpose

Stores each attempt made by a candidate for a registered examination.

Columns

Column	Data Type	Null	Default
attemptId	BIGSERIAL	NO	Auto
registrationId	BIGINT	NO	—
attemptNumber	SMALLINT	NO	1
startTime	TIMESTAMPTZ	NO	NOW()
endTime	TIMESTAMPTZ	YES	NULL
submittedAt	TIMESTAMPTZ	YES	NULL

status	VARCHAR(20)	NO	'NOT_STARTED'
totalTimeSpentSeconds	INTEGER	YES	NULL
ipAddress	INET	YES	NULL
deviceInfo	TEXT	YES	NULL
browserInfo	TEXT	YES	NULL
submissionMethod	VARCHAR(30)	NO	'MANUAL'
autoSubmitted	BOOLEAN	NO	FALSE
createdAt	TIMESTAMPTZ	NO	NOW()

Primary Key

- attemptId

Foreign Keys

- registrationId → CandidateRegistration(registrationId)

Unique Constraints

- (registrationId, attemptNumber)

Check Constraints

attemptNumber > 0

status IN
('NOT_STARTED','IN_PROGRESS','SUBMITTED','AUTO_SUBMITTED','EVALUATED','ABANDONED')

submissionMethod IN
('MANUAL','AUTO_TIMEOUT','SYSTEM')

Indexes

PK(attemptId)

INDEX(registrationId)

INDEX(status)

INDEX(startTime)

UNIQUE(registrationId,attemptNumber)

Delete Rule

RESTRICT

Table 16 — Answer

Purpose

Stores every answer submitted by a student during an examination attempt.

Columns

Column	Data Type	Null	Default
answerId	BIGSERIAL	NO	Auto
attemptId	BIGINT	NO	—
questionId	BIGINT	NO	—
selectedOption	VARCHAR(20)	YES	NULL
answerText	TEXT	YES	NULL
isMarkedForReview	BOOLEAN	NO	FALSE
isAnswered	BOOLEAN	NO	FALSE
submittedAt	TIMESTAMPTZ	NO	NOW()
timeSpentSeconds	INTEGER	NO	0
createdAt	TIMESTAMPTZ	NO	NOW()

Primary Key

- answerId

Foreign Keys

- attemptId → ExamAttempt(attemptId)
- questionId → Question(questionId)

Unique Constraints

- (attemptId, questionId)

Check Constraints

timeSpentSeconds >= 0

Indexes

PK(answerId)

INDEX(attemptId)

INDEX(questionId)

INDEX(isAnswered)

INDEX(isMarkedForReview)

UNIQUE(attemptId,questionId)

Delete Rule

CASCADE from ExamAttempt

RESTRICT from Question

Table 17 — Evaluation

Purpose

Stores the overall evaluation summary of an examination attempt.

Columns

Column	Data Type	Null	Default
evaluationId	BIGSERIAL	NO	Auto
attemptId	BIGINT	NO	—
totalMarksObtained	NUMERIC(6,2)	NO	0
totalCorrect	INTEGER	NO	0
totalWrong	INTEGER	NO	0
totalSkipped	INTEGER	NO	0
evaluationStatus	VARCHAR(20)	NO	'PENDING'
evaluatedBy	BIGINT	YES	NULL
evaluatedAt	TIMESTAMPTZ	YES	NULL
remarks	TEXT	YES	NULL
createdAt	TIMESTAMPTZ	NO	NOW()

Primary Key

- evaluationId

Foreign Keys

- attemptId → ExamAttempt(attemptId)
- evaluatedBy → User(userId)

Unique Constraints

- attemptId

Check Constraints

totalMarksObtained >= 0

evaluationStatus IN
('PENDING', 'AUTO_EVALUATED', 'MANUAL_EVALUATED', 'FINALIZED')

Indexes

PK(evaluationId)

UNIQUE(attemptId)

INDEX(evaluationStatus)

INDEX(evaluatedAt)

Delete Rule

CASCADE from ExamAttempt

Table 18 — EvaluationDetail

Purpose

Stores question-wise evaluation details.

Columns

Column	Data Type	Null	Default
evaluationDetailId	BIGSERIAL	NO	Auto
evaluationId	BIGINT	NO	—
questionId	BIGINT	NO	—
marksAwarded	NUMERIC(5,2)	NO	0
maxMarks	NUMERIC(5,2)	NO	—
isCorrect	BOOLEAN	NO	FALSE
evaluatorRemarks	TEXT	YES	NULL
evaluatedAt	TIMESTAMPTZ	NO	NOW()

Primary Key

- evaluationDetailId

Foreign Keys

- evaluationId → Evaluation(evaluationId)
- questionId → Question(questionId)

Unique Constraints

- (evaluationId, questionId)

Check Constraints

marksAwarded >= 0

marksAwarded <= maxMarks

Indexes

PK(evaluationDetailId)

INDEX(evaluationId)

INDEX(questionId)

UNIQUE(evaluationId,questionId)

Delete Rule

CASCADE

Table 19 — Result

Purpose

Stores the published examination result.

Columns

Column	Data Type	Null	Default
--------	-----------	------	---------

resultId	BIGSERIAL	NO	Auto
evaluationId	BIGINT	NO	—
percentage	NUMERIC(5,2)	NO	—
grade	VARCHAR(5)	YES	NULL
passStatus	BOOLEAN	NO	FALSE
publishedAt	TIMESTAMPTZ	YES	NULL
publishedBy	BIGINT	YES	NULL
isPublished	BOOLEAN	NO	FALSE
remarks	TEXT	YES	NULL
createdAt	TIMESTAMPTZ	NO	NOW()

Primary Key

- resultId

Foreign Keys

- evaluationId → Evaluation(evaluationId)
- publishedBy → User(userId)

Unique Constraints

- evaluationId

Check Constraints

percentage BETWEEN 0 AND 100

Indexes

PK(resultId)

UNIQUE(evaluationId)

INDEX(isPublished)

INDEX(passStatus)

INDEX(publishedAt)

Delete Rule

RESTRICT

Examination Delivery Summary

Table	Type
ExamAttempt	Transaction
Answer	Transaction
Evaluation	Transaction
EvaluationDetail	Transaction
Result	Transaction

Important Engineering Improvements (Recommended Before Freezing)

After reviewing this section against our architecture principles, I recommend the following refinements.

1. Replace **selectedOption** with **selectedOptionId**

Current schema:

selectedOption

Better:

selectedOptionId

This becomes even more valuable if we adopt the **QuestionOption** table recommended in Part 3.

2. Remove **percentage** from Result

Percentage is a **derived value**.

It can always be computed as:

Marks Obtained / Total Marks × 100

According to **DB-005 (Normalization Strategy)** and **Principle 12 (Measurable Improvements)**, derived values should generally not be permanently stored unless there is a demonstrated performance need.

A better approach is to compute it during result generation or cache it if profiling shows it is expensive.

3. Add Attempt Integrity Information

To strengthen auditability and support future security features, consider adding:

Column	Type
systemSubmitted	BOOLEAN
submissionReason	VARCHAR(50)
lastSavedAt	TIMESTAMPTZ

This allows the system to distinguish between manual submissions, timeout submissions, and recovery scenarios.

4. Keep Evaluation Immutable After Finalization

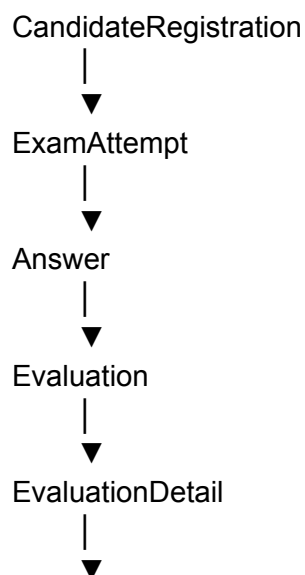
Once `evaluationStatus = FINALIZED`, evaluation records should become immutable. Any correction should create a new revision or be handled through an explicit re-evaluation workflow rather than updating historical data.

5. Consider Moving `grade` to Business Logic

The `grade` value is also derived from marks and grading rules. Unless grading policies are expected to change historically, it may be better computed during publication rather than stored. If institutional policy requires preserving the exact published grade, storing it in `Result` is justified.

Part 4 Status: Complete

The examination execution pipeline is now fully modeled:



Result

This establishes a clear, auditable flow from registration through result publication.

The final section, **Part 5**, will complete the database with:

1. Notification
2. NotificationTemplate
3. AuditLog
4. SystemEvent
5. PerformanceMetric
6. SystemConfiguration
7. FeatureFlag

These tables support communication, auditing, monitoring, system configuration, and operational management.

PART 5 — Notifications, Audit & Administration Database Specification

This section completes the database design with the remaining **7 tables**.

- 1. Notification
- 2. NotificationTemplate
- 3. AuditLog
- 4. SystemEvent
- 5. PerformanceMetric
- 6. SystemConfiguration
- 7. FeatureFlag

These tables are responsible for system administration, monitoring, auditing, configuration, and communication.

Table 20 — Notification

Purpose

Stores notifications delivered to users.

Columns

Column	Data Type	Null	Default
notificationId	BIGSERIAL	NO	Auto
userId	BIGINT	NO	—
templateId	BIGINT	YES	NULL

title	VARCHAR(150)	NO	—
message	TEXT	NO	—
notificationType	VARCHAR(30)	NO	—
deliveryChannel	VARCHAR(20)	NO	'IN_APP'
priority	VARCHAR(20)	NO	'NORMAL'
isRead	BOOLEAN	NO	FALSE
sentAt	TIMESTAMPTZ	NO	NOW()
readAt	TIMESTAMPTZ	YES	NULL
expiresAt	TIMESTAMPTZ	YES	NULL

Primary Key

- notificationId

Foreign Keys

- userId → User(userId)
- templateId → NotificationTemplate(templateId)

Unique Constraints

None

Check Constraints

notificationType IN

('EXAM','RESULT','SYSTEM','REMINDER','SECURITY')

deliveryChannel IN

('IN_APP','EMAIL','SMS')

priority IN

('LOW','NORMAL','HIGH','CRITICAL')

Indexes

PK(notificationId)

INDEX(userId)

INDEX(isRead)

INDEX(sentAt)

INDEX(notificationType)

Delete Rule

User → CASCADE

Template → SET NULL

Table 21 — NotificationTemplate

Purpose

Stores reusable notification templates.

Columns

Column	Data Type	Null	Default
templateId	BIGSERIAL	NO	Auto
templateName	VARCHAR(100)	NO	—
notificationType	VARCHAR(30)	NO	—
subject	VARCHAR(150)	YES	NULL
templateBody	TEXT	NO	—
isActive	BOOLEAN	NO	TRUE
createdAt	TIMESTAMPTZ	NO	NOW()
updatedAt	TIMESTAMPTZ	NO	NOW()

Primary Key

- templateId

Foreign Keys

None

Unique Constraints

- templateName

Check Constraints

None

Indexes

PK(templateId)

UNIQUE(templateName)

INDEX(notificationType)

Delete Rule

RESTRICT

Table 22 — AuditLog

Purpose

Maintains immutable security and activity audit records.

Columns

Column	Data Type	Null	Default
auditLogId	BIGSERIAL	NO	Auto
userId	BIGINT	YES	NULL
entityName	VARCHAR(100)	NO	—
entityId	BIGINT	YES	NULL
action	VARCHAR(30)	NO	—
oldValue	JSONB	YES	NULL
newValue	JSONB	YES	NULL
ipAddress	INET	YES	NULL
timestamp	TIMESTAMPTZ	NO	NOW()

Primary Key

- auditLogId

Foreign Keys

- userId → User(userId)

Unique Constraints

None

Check Constraints

action IN

('CREATE','UPDATE','DELETE','LOGIN','LOGOUT','VIEW','EXPORT')

Indexes

PK(auditLogId)

INDEX(userId)

INDEX(entityName)

INDEX(action)

INDEX(timestamp)

Delete Rule

RESTRICT

Table 23 — SystemEvent

Purpose

Stores significant application events.

Columns

Column	Data Type	Null	Default
eventId	BIGSERIAL	NO	Auto
eventType	VARCHAR(50)	NO	—
severity	VARCHAR(20)	NO	—
sourceModule	VARCHAR(50)	NO	—
description	TEXT	NO	—
occurredAt	TIMESTAMPTZ	NO	NOW()
resolvedAt	TIMESTAMPTZ	YES	NULL
status	VARCHAR(20)	NO	'OPEN'

Primary Key

- eventId

Foreign Keys

None

Unique Constraints

None

Check Constraints

severity IN

('INFO','WARNING','ERROR','CRITICAL')

status IN

('OPEN','ACKNOWLEDGED','RESOLVED')

Indexes

PK(eventId)

INDEX(eventType)

INDEX(severity)

INDEX(status)

INDEX(occurredAt)

Delete Rule

RESTRICT

Table 24 — PerformanceMetric

Purpose

Stores runtime performance statistics.

Columns

Column	Data Type	Null	Default
metricId	BIGSERIAL	NO	Auto
metricName	VARCHAR(100)	NO	—
metricValue	NUMERIC(12,4)	NO	—
metricUnit	VARCHAR(20)	NO	—
sourceModule	VARCHAR(50)	NO	—
recordedAt	TIMESTAMPTZ	NO	NOW()

Primary Key

- metricId

Foreign Keys

None

Unique Constraints

None

Check Constraints

metricValue >= 0

Indexes

PK(metricId)

INDEX(metricName)

INDEX(sourceModule)

INDEX(recordedAt)

Delete Rule

RESTRICT

Table 25 — SystemConfiguration

Purpose

Stores configurable system parameters.

Columns

Column	Data Type	Null	Default
configurationId	BIGSERIAL	NO	Auto
configurationKey	VARCHAR(100)	NO	—

configurationValue	TEXT	NO	—
valueType	VARCHAR(20)	NO	—
description	TEXT	YES	NULL
isEditable	BOOLEAN	NO	TRUE
updatedAt	TIMESTAMPTZ	NO	NOW()
updatedBy	BIGINT	YES	NULL

Primary Key

- configurationId

Foreign Keys

- updatedBy → User(userId)

Unique Constraints

- configurationKey

Check Constraints

valueType IN

('STRING','INTEGER','BOOLEAN','DECIMAL','JSON')

Indexes

PK(configurationId)

UNIQUE(configurationKey)

Delete Rule

RESTRICT

Table 26 — FeatureFlag

Purpose

Controls enabling and disabling application features without code changes.

Columns

Column	Data Type	Null	Default
featureFlagId	BIGSERIAL	NO	Auto
flagName	VARCHAR(100)	NO	—
description	TEXT	YES	NULL
isEnabled	BOOLEAN	NO	FALSE
rolloutPercentage	SMALLINT	NO	100

createdAt	TIMESTAMPTZ	NO	NOW()
-----------	-------------	----	-------

updatedAt	TIMESTAMPTZ	NO	NOW()
-----------	-------------	----	-------

Primary Key

- featureFlagId

Foreign Keys

None

Unique Constraints

- flagName

Check Constraints

rolloutPercentage BETWEEN 0 AND 100

Indexes

PK(featureFlagId)

UNIQUE(flagName)

INDEX(isEnabled)

Delete Rule

RESTRICT

Administration & Monitoring Summary

Table	Type
Notification	Transaction
NotificationTemplate	Master
AuditLog	Audit
SystemEvent	Monitoring
PerformanceMetric	Monitoring
SystemConfiguration	Configuration
FeatureFlag	Configuration

Complete Database Summary

Category	Tables
Identity & Access	6

Academic & Examination	4
Question Bank	4
Examination Execution	5
Administration & Monitoring	7
Total	26 Tables

Overall Review & Recommendations

Now that all 26 tables have been defined, I would make three architectural refinements before freezing the schema:

1. Normalize Question Options

Instead of storing `optionA`, `optionB`, `optionC`, and `optionD` in the `Question` table, introduce a separate `QuestionOption` table. This supports any number of options, multiple-correct-answer questions, and future media-based options while improving normalization.

2. Introduce Specialized Profile Tables

Keep `User` as the authentication entity and add role-specific profile tables such as `StudentProfile` and `FacultyProfile` if additional attributes become necessary. This avoids cluttering the `User` table while maintaining a clean identity model.

3. Expand Monitoring for KPI Evaluation

Since your project emphasizes KPIs like response time, concurrency, and resource utilization, **PerformanceMetric** can later be extended to capture metrics such as API latency, CPU usage, memory usage, concurrent sessions, database query times, cache hit ratios, and thread pool statistics. This will directly support the measurable optimization goals defined in your project philosophy.

DB-006 Status

With Parts 1–5 complete, **DB-006 (Physical Schema Design)** is now complete from a structural perspective. The next logical database phase is **DB-007 — Constraints & Data Integrity**, where we'll formalize referential actions, business constraints, transaction rules, and integrity mechanisms across the entire schema before moving on to indexing and performance optimization.

TechStack

Final Tech Stack

Category	Recommended
Programming Language	Python 3.13
Backend Framework	Flask (Layered Modular Monolith)
Frontend	HTML5 + CSS3 + Bootstrap 5 + JavaScript
Database	MySQL 8
ORM	SQLAlchemy ORM + Parameterized Raw SQL (only where justified)
Authentication	Stateful Session-Based Authentication
Authorization	Database-driven RBAC
Password Hashing	Argon2id
API Style	REST (internal AJAX endpoints where needed)
API Documentation	Simple OpenAPI/Swagger (optional)
Background Tasks	APScheduler
Logging	Python logging
Database Migration	Alembic or Flask-Migrate
Testing	pytest
Version Control	Git + GitHub
IDE	VS Code
Caching	Simple in-memory cache for master data
Deployment	Local → Docker (optional) → Nginx + Gunicorn (if deployed)

Why this stack?

It directly supports the engineering concepts your faculty wants to evaluate:

Subject	Demonstrated Through
Software Engineering	Layered modular architecture, separation of concerns, high cohesion, low coupling
OOP	Services, repositories, models, encapsulation
DBMS	MySQL, normalization, SQLAlchemy, transactions, indexing
Computer Networks	HTTP, REST, AJAX, sessions
Operating Systems	Session lifecycle, concurrent users, scheduling with APScheduler
Cyber Security	Argon2id, RBAC, CSRF protection, parameterized queries
Web Technologies	HTML, CSS, Bootstrap, JavaScript, responsive UI
Testing	Unit and integration testing with pytest

DESIGN PHILOSOPHY

DESIGN PHILOSOPHY

Online Examination Platform

Version 1.0

Document Information

Item	Details
Document Name	Design Philosophy
Project	Online Examination Platform
Version	1.0
Status	
Purpose	Define the engineering philosophy and guiding principles governing all architectural, technological, and implementation decisions throughout the project lifecycle.

1. Purpose

The purpose of this document is to establish the engineering philosophy that governs the design, implementation, and evolution of the Online Examination Platform.

Rather than treating the project as an isolated web application or database application, this project is designed as an **integration project** where multiple core Computer Science subjects are deliberately combined to solve real engineering problems. Every major design decision must contribute not only to the functional requirements of the system but also to measurable improvements in performance, security, maintainability, scalability, reliability, and overall software quality.

This document serves as the foundation for all future project artifacts including the Project Development Lifecycle, Architecture Decision Points (ADPs), Engineering Decision Register

(EDR), Software Requirements Specification (SRS), Database Design, API Design, and Implementation.

2. Vision

Develop a secure, efficient, scalable, maintainable, and well-engineered Online Examination Platform that demonstrates the practical integration of core Computer Science concepts while satisfying all functional requirements, project deliverables, KPIs, and expected outcomes defined by the college.

3. Design Objectives

The project shall be designed to achieve the following objectives:

- Demonstrate practical application of all major core Computer Science subjects.
 - Maintain a clean and modular software architecture.
 - Achieve high performance through efficient algorithms and optimized data structures.
 - Ensure data integrity using sound database engineering practices.
 - Provide secure authentication, authorization, and communication.
 - Support concurrent users efficiently.
 - Minimize resource utilization while maximizing throughput.
 - Produce maintainable, extensible, and well-documented software.
 - Align every engineering decision with measurable project KPIs.
-

4. Engineering Philosophy

The project is an engineering-focused integration project rather than a technology showcase.

Every component of the system must exist because it solves an engineering problem. Technologies, frameworks, libraries, and algorithms are selected only when they improve the overall quality of the system and demonstrate the underlying Computer Science concepts expected by the project evaluation criteria.

The architecture shall remain independent of implementation technologies, ensuring that technology supports the architecture rather than defining it.

5. Design Principles

Principle 1 – Integration of Core Computer Science Subjects

The project shall deliberately integrate major Computer Science subjects into a unified software system. Every module should naturally demonstrate one or more core subjects while solving real engineering problems.

Subject	Primary Contribution
Programming	Modular implementation and reusable logic
OOP	Encapsulation, abstraction, inheritance, polymorphism, maintainability
DSA	Efficient algorithms, optimized time and space complexity
DBMS	Normalization, transactions, indexing, integrity, query optimization
Computer Networks	Secure client-server communication, REST APIs, session management
Operating Systems	Concurrency, synchronization, scheduling, resource utilization
Software Engineering	Architecture, modularity, testing, documentation
Web Technologies	Responsive and accessible user interaction

Principle 2 – Every Design Decision Must Answer Two Questions

Every architectural, technological, or implementation decision must answer:

1. Which Computer Science subject(s) does this demonstrate?
2. How does it improve the system?

Improvements may include:

- Performance
- Security
- Scalability
- Maintainability
- Reliability
- Lower Time Complexity
- Lower Space Complexity
- Better CPU utilization
- Better concurrency
- Better resource utilization

If a feature cannot justify both questions, its inclusion should be reconsidered.

Principle 3 – Requirements are Fixed

The college project documentation—including subject-wise implementation requirements, deliverables, KPIs, and expected outcomes—is the authoritative source of project requirements.

The architecture may be improved, but required functionality shall never be removed or altered.

Principle 4 – Functional Architecture Over Team Organization

Project architecture shall be organized according to functional cohesion and engineering principles rather than team assignments or work distribution.

Software modules represent business capabilities, not development responsibilities.

Principle 5 – Module Merging Requires Technical Justification

Modules may only be merged when the merge:

- Increases cohesion
- Reduces coupling
- Improves maintainability

- Eliminates redundancy
 - Preserves single responsibility
 - Strengthens demonstration of Computer Science concepts
-

Principle 6 – Every Subject Must Have a Natural Contribution

Core Computer Science concepts shall never be artificially inserted merely to satisfy evaluation criteria.

Each subject must solve an actual engineering problem.

Examples:

- Hash tables improve lookup performance.
 - Transactions preserve examination consistency.
 - Thread synchronization prevents race conditions.
 - Indexes improve database performance.
 - RBAC improves security.
-

Principle 7 – KPIs Drive the Architecture

System architecture shall directly support the project's Key Performance Indicators.

Examples:

KPI	Engineering Solution
Response Time	Efficient algorithms, optimized SQL, caching
Concurrent Users	Thread management, synchronization
Database Integrity	Constraints, transactions
Security	RBAC, Argon2id, parameterized queries
Maintainability	Layered architecture, SOLID principles

Principle 8 – Deliverables Must Be a By-product of the Architecture

Project deliverables shall naturally emerge from the engineering process rather than being developed independently.

Examples:

- UML diagrams derive from class design.
 - SQL scripts derive from the database schema.
 - API documentation derives from REST endpoints.
 - Unit tests derive from modular implementation.
-

Principle 9 – Think Like Software Engineers

Engineering decisions must always be supported by technical reasoning rather than syllabus coverage alone.

Instead of:

"A Queue was used because queues are part of DSA."

The project should justify:

"A Queue was selected because examination requests are processed in FIFO order, improving fairness while demonstrating Data Structures."

Principle 10 – No Accidental Decisions

Every significant decision shall document:

- Problem being solved
 - Alternative solutions
 - Selected solution
 - Technical justification
 - Computer Science concepts demonstrated
 - Measurable improvement
-

Principle 11 – Engineering Over Convenience

Whenever multiple solutions satisfy the functional requirements, the solution providing stronger engineering quality and better demonstration of core Computer Science concepts shall be preferred over the easiest implementation.

Principle 12 – Measurable Optimizations

Every optimization introduced into the system shall be measurable.

Examples include:

Optimization	Before	After
Question Search	O(n)	O(1) using HashMap
Database Query	Full Table Scan	Indexed Query
Report Generation	Sequential	Parallel Processing
Answer Evaluation	Single Thread	Multi-threaded

Performance improvements should be validated using measurable metrics whenever possible.

Principle 13 – Frameworks Support Engineering, Not Replace It

Frameworks shall automate repetitive infrastructure tasks without replacing opportunities to demonstrate fundamental Computer Science concepts.

Examples:

- SQLAlchemy ORM for CRUD operations while using SQLAlchemy Core or parameterized SQL for complex queries.
- Flask for routing while implementing custom service and repository layers.
- APScheduler for scheduling while implementing scheduling policies within the application.
- Custom in-memory caching instead of introducing Redis without architectural justification.

6. Decision Hierarchy

Engineering decisions shall follow the following precedence:

1. College Requirements
2. Approved Project Architecture
3. Approved Technology Stack
4. Engineering Decisions
5. Implementation Decisions

Lower-level decisions shall never violate higher-level decisions.

7. Technology Selection Philosophy

Technology shall never be selected because it is popular or modern.

Every selected technology must:

- Solve a real engineering problem.
 - Improve one or more project KPIs.
 - Support the selected architecture.
 - Demonstrate relevant Computer Science concepts.
 - Avoid unnecessary complexity.
-

8. Performance Philosophy

Performance optimization shall focus on measurable improvements rather than premature optimization.

Optimization techniques may include:

- Efficient data structures
- Optimized algorithms
- Query optimization
- Appropriate indexing

- Concurrency where beneficial
 - Resource-efficient scheduling
 - Application-level caching
-

9. Security Philosophy

Security shall be integrated throughout the architecture rather than added after implementation.

The system shall adopt:

- Principle of Least Privilege
 - Defense in Depth
 - Secure by Default
 - Input Validation
 - Parameterized SQL Queries
 - Secure Password Hashing (Argon2id)
 - Role-Based Access Control
 - Secure Session Management
 - Audit Logging
-

10. Maintainability Philosophy

Maintainability shall be achieved through:

- Layered architecture
 - High cohesion
 - Low coupling
 - Modular implementation
 - SOLID principles
 - Consistent coding standards
 - Clear documentation
 - Comprehensive testing
-

11. Success Criteria

The project shall be considered successful if it:

- Satisfies all college functional requirements.
- Meets or exceeds project KPIs.
- Demonstrates every required Computer Science subject naturally.
- Maintains a modular and scalable architecture.
- Provides measurable performance improvements.
- Implements secure engineering practices.
- Produces maintainable, extensible, and well-documented software.

PROJECT DEVELOPMENT LIFECYCLE

PROJECT DEVELOPMENT LIFECYCLE (PDLC)

Online Examination Platform

Version 1.0

Document Information

Item	Details
Document Name	Project Development Lifecycle
Project	Online Examination Platform
Version	1.0
Status	
Purpose	Define the structured engineering process used to design, develop, test, optimize, and document the Online Examination Platform.

1. Purpose

The Project Development Lifecycle (PDLC) defines the sequence of engineering activities followed during the development of the Online Examination Platform.

Unlike traditional software development lifecycles that focus primarily on implementation, this lifecycle emphasizes systematic engineering decisions, traceability, and integration of core Computer Science subjects throughout the development process.

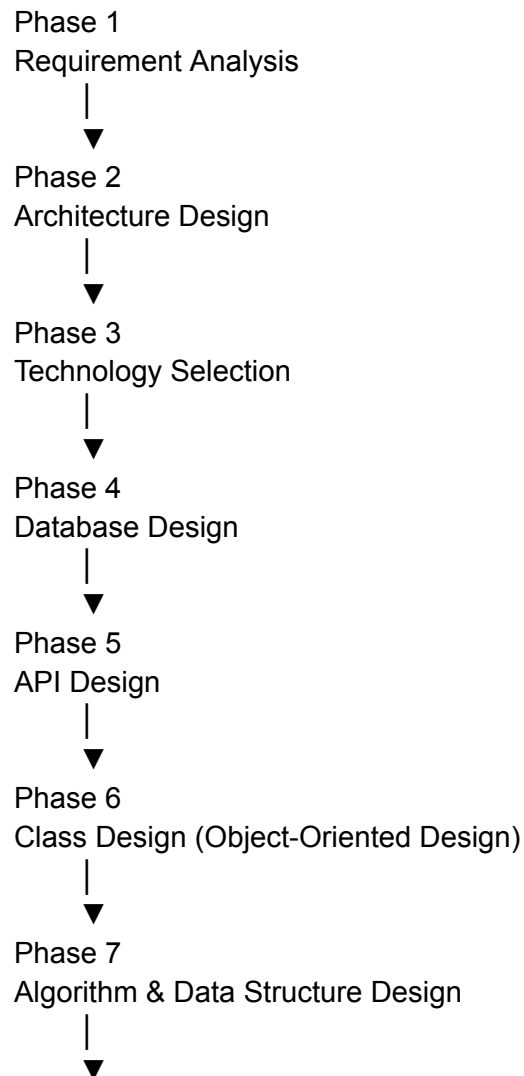
Each phase produces specific outputs that become inputs for subsequent phases, ensuring consistency, maintainability, and complete traceability from requirements to implementation.

2. Development Philosophy

The lifecycle follows five fundamental principles:

- Requirements drive architecture.
 - Architecture drives technology.
 - Technology supports implementation.
 - Every phase produces measurable deliverables.
 - Every deliverable remains traceable back to project requirements.
-

3. Lifecycle Overview



Phase 8
Operating System & Concurrency Design



Phase 9
Security Design



Phase 10
User Interface Design



Phase 11
Implementation



Phase 12
Testing & Validation



Phase 13
Performance Optimization



Phase 14
Documentation & Project Closure

Phase 1 — Requirement Analysis

Objective

Understand and analyze the complete project requirements.

Activities

- Analyze project statement.
- Study faculty evaluation criteria.
- Analyze deliverables.
- Analyze KPIs.
- Analyze expected outcomes.

- Identify hidden requirements.
- Map requirements to core CS subjects.
- Define project scope.

Inputs

- College Project Document

Outputs

- Requirement Analysis Report
 - Requirement List
 - Subject Mapping
 - Initial Module Identification
-

Phase 2 — Architecture Design

Objective

Transform requirements into a structured software architecture.

Activities

- Design Philosophy
- Module Identification
- Module Merging
- Layered Architecture
- Domain Decomposition
- Bounded Context Identification
- Module Ownership
- High-Level System Architecture

Inputs

- Requirement Analysis

Outputs

- Architecture Document
 - Architecture Decision Points (ADPs)
 - Module Architecture
 - Component Diagram
-

Phase 3 — Technology Selection

Objective

Select technologies that best support the architecture.

Activities

- Programming Language Selection
- Framework Selection
- Database Selection
- ORM Selection
- Authentication Technology
- Testing Framework
- Logging Framework
- Migration Framework
- Development Environment

Inputs

- Architecture Design

Outputs

- Technology Stack
 - Technology Selection Records (internal)
 - Engineering Justifications
-

Phase 4 — Database Design

Objective

Design a secure, normalized, and optimized relational database.

Activities

- Domain Data Modeling
- ER Diagram Design
- Logical Database Design
- Physical Database Design
- Normalization
- Primary and Foreign Keys
- Constraints
- Index Strategy
- Transaction Design
- Query Optimization Planning

Inputs

- Architecture
- Technology Stack

Outputs

- ER Diagram
 - Database Schema
 - SQL Scripts
 - Data Dictionary
-

Phase 5 — API Design

Objective

Define communication between system modules and external clients.

Activities

- REST Endpoint Design
- Request/Response Models
- Validation Rules
- Error Handling
- Authentication Integration
- API Documentation

Inputs

- Database Design

Outputs

- API Specification
 - Endpoint Documentation
-

Phase 6 — Class Design (Object-Oriented Design)

Objective

Transform architecture into an object-oriented implementation.

Activities

- Class Identification
- Responsibility Assignment
- Interface Design
- Service Layer Design
- Repository Design
- Design Patterns
- UML Class Diagrams

Inputs

- Database Design

- API Design

Outputs

- UML Class Diagram
 - Class Specifications
-

Phase 7 — Algorithm & Data Structure Design

Objective

Optimize system performance through efficient algorithms and data structures.

Activities

- Data Structure Selection
- Time Complexity Analysis
- Space Complexity Analysis
- Search Algorithms
- Scheduling Algorithms
- Cache Strategy
- Performance Analysis

Inputs

- Class Design

Outputs

- Algorithm Specifications
 - Complexity Analysis
-

Phase 8 — Operating System & Concurrency Design

Objective

Design mechanisms for efficient concurrent execution and resource management.

Activities

- Thread Design
- Synchronization
- Background Scheduling
- Session Management
- Deadlock Prevention
- Resource Allocation
- CPU Utilization Planning

Inputs

- Algorithm Design

Outputs

- Concurrency Design
 - Thread Architecture
-

Phase 9 — Security Design

Objective

Protect the system against security threats.

Activities

- Authentication Design

- Authorization Design
- Session Security
- Password Protection
- Input Validation
- SQL Injection Prevention
- CSRF Protection
- Audit Logging

Inputs

- API Design
- Database Design

Outputs

- Security Architecture
 - Threat Mitigation Plan
-

Phase 10 — User Interface Design

Objective

Design a user-friendly and responsive interface.

Activities

- Wireframes
- Navigation
- Responsive Layout
- Role-Based Dashboards
- User Experience Design

Inputs

- API Design

Outputs

- UI Mockups
 - Screen Designs
-

Phase 11 — Implementation

Objective

Develop the complete software system.

Activities

- Backend Development
- Frontend Development
- Database Implementation
- Module Integration
- Code Review

Inputs

- All Design Documents

Outputs

- Working Software
-

Phase 12 — Testing & Validation

Objective

Verify correctness and quality.

Activities

- Unit Testing
- Integration Testing
- System Testing
- Functional Testing
- Security Testing
- Performance Testing
- Bug Fixing

Inputs

- Implemented System

Outputs

- Test Reports
 - Validation Results
-

Phase 13 — Performance Optimization

Objective

Improve efficiency and satisfy project KPIs.

Activities

- SQL Optimization
- Algorithm Optimization
- Cache Optimization
- Thread Optimization
- Memory Optimization
- CPU Optimization

Inputs

- Test Results

Outputs

- Optimization Report
 - Benchmark Results
-

Phase 14 — Documentation & Project Closure

Objective

Prepare the project for evaluation and future maintenance.

Activities

- SRS
- User Manual
- Technical Documentation
- Installation Guide
- Project Report
- Presentation Preparation
- Final Review

Inputs

- Completed Project

Outputs

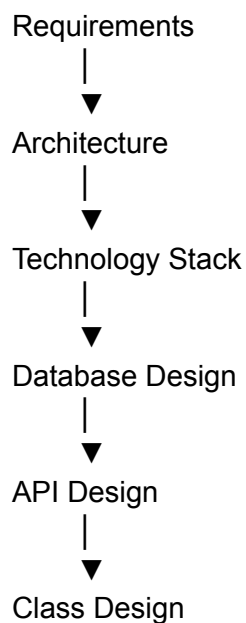
- Complete Project Documentation
 - Final Deliverables
-

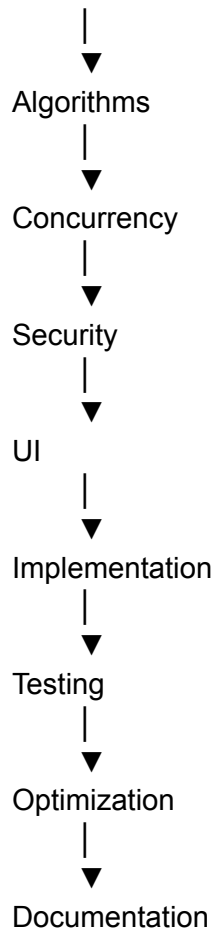
4. Phase Dependencies

Phase	Depends On
Requirement Analysis	None

Architecture Design	Requirement Analysis
Technology Selection	Architecture Design
Database Design	Technology Selection
API Design	Database Design
Class Design	API Design
Algorithm & DSA Design	Class Design
OS & Concurrency Design	Algorithm Design
Security Design	API + Database Design
UI Design	API Design
Implementation	All Design Phases
Testing	Implementation
Optimization	Testing
Documentation	Entire Project

5. Deliverable Flow





6. Governance Documents

Throughout the lifecycle, the following documents guide development:

Document	Purpose
Design Philosophy	Governs engineering principles and decision-making
Project Development Lifecycle	Defines the engineering process
Architecture Decision Points (ADPs)	Records major architectural decisions
Engineering Decision Register (EDR)	Records implementation-level engineering decisions

Requirement Traceability Matrix
(RTM)

Ensures complete traceability from requirements to
implementation

7. Success Criteria

The Project Development Lifecycle is considered successfully executed when:

- Every phase produces its defined deliverables.
- Every engineering decision is justified and documented.
- Every requirement is traceable through design and implementation.
- All project KPIs are addressed through measurable engineering solutions.
- The final system demonstrates the required core Computer Science subjects in a natural and integrated manner.

Architecture Decision Points (ADP)

Architecture Decision Points (ADP)

Online Examination Platform

Version 1.0

Purpose

Architecture Decision Points (ADPs) document the major architectural decisions that govern the structure, organization, communication, ownership, and behavior of the Online Examination Platform.

These decisions establish the architectural foundation of the system and ensure that all subsequent design and implementation activities remain consistent with the Design Philosophy and Project Development Lifecycle.

ADPs are intended to remain stable throughout the project lifecycle and are independent of implementation technologies wherever possible.

Standard ADP Template

Every ADP will follow the same structure.

Section	Purpose
ADP ID	Unique identifier
Title	Architectural decision
Status	Proposed / Approved / Superseded
Context	Background requiring the decision
Problem Statement	Architecture problem being solved

Alternatives Considered	Other architectural options
Decision	Final architectural decision
Engineering Rationale	Why this architecture was selected
Positive Consequences	Benefits
Trade-offs	Known limitations
Related Design Principles	Links to Design Philosophy
Related KPIs	KPIs supported
Related EDRs	Engineering decisions derived later

ADP Roadmap

I recommend freezing the following ADP list.

ADP	Title	Status
ADP-001	Overall System Architecture	Pending
ADP-002	Functional Module Architecture	Pending
ADP-003	Layered Software Architecture	Pending
ADP-004	Module Communication Architecture	Pending
ADP-005	Domain Architecture & Bounded Contexts	Pending
ADP-006	Module Ownership & Data Ownership	Pending
ADP-007	Persistence Architecture	Pending

ADP-008	Security Architecture	Pending
ADP-009	Concurrency Architecture	Pending
ADP-010	Deployment Architecture	Pending

Notice something.

We reduced them to **10 architectural decisions**.

Everything else belongs in EDRs.

This keeps architecture clean.

ADP-001 — Overall System Architecture

Status

Approved

Context

The Online Examination Platform consists of multiple functional modules including user management, examination management, question management, examination runtime, evaluation, reporting, monitoring, notifications, audit logging, and platform administration.

The system requires an architecture that supports clear separation of responsibilities, maintainability, scalability within the project scope, and integration of multiple Computer Science concepts while remaining suitable for a single deployable application.

Problem Statement

Which overall software architecture should be adopted to organize the Online Examination Platform while satisfying the project's functional requirements, engineering objectives, KPIs, and expected outcomes?

Alternatives Considered

Alternative 1 — Traditional Monolithic Architecture

Advantages

- Simple implementation
- Easy deployment
- Low infrastructure requirements

Disadvantages

- High coupling
 - Poor modularity
 - Difficult maintenance
 - Limited scalability of codebase
-

Alternative 2 — Microservices Architecture

Advantages

- Independent services
- Independent deployment
- High scalability

Disadvantages

- Significant operational complexity
 - Distributed communication
 - Increased deployment overhead
 - Beyond the scope of the project
 - Reduces focus on core engineering concepts
-

Alternative 3 — Service-Oriented Architecture (SOA)

Advantages

- Good service separation
- Reusable business services

Disadvantages

- Additional infrastructure complexity
 - Unnecessary for the project scope
-

Alternative 4 — Layered Modular Monolith

Advantages

- High cohesion
- Low coupling
- Clear module boundaries
- Single deployment unit
- Simpler testing
- Easier maintenance
- Supports modular engineering
- Excellent demonstration of Software Engineering principles

Disadvantages

- Requires disciplined module boundaries
 - Modules cannot be independently deployed
-

Decision

The Online Examination Platform shall adopt a **Layered Modular Monolith Architecture**.

The system shall be deployed as a single application while internally organized into cohesive, independent business modules with clearly defined responsibilities and communication boundaries.

Engineering Rationale

The Layered Modular Monolith architecture provides the best balance between software engineering quality and project complexity.

This architecture:

- Separates business capabilities into independent modules.
- Promotes high cohesion and low coupling.
- Simplifies testing and debugging.
- Supports maintainable and extensible software.
- Demonstrates Software Engineering principles explicitly.
- Avoids unnecessary distributed-system complexity.
- Aligns with the project's KPIs related to maintainability, integration, reliability, and code quality.
- Provides sufficient scalability for the expected workload while keeping implementation manageable.

Unlike traditional monolithic systems, the architecture enforces clear module boundaries and responsibility ownership. Unlike microservices, it avoids unnecessary infrastructure complexity that would not contribute meaningfully to the educational objectives of the project.

Architectural Characteristics

The architecture shall satisfy the following characteristics:

- Single deployable application
 - Modular business organization
 - Layered internal structure
 - Explicit module ownership
 - Clear dependency direction
 - Centralized configuration
 - Shared relational database with logical ownership
 - REST-based client interaction
 - Stateless business services with managed server-side sessions
 - Separation between presentation, business, and persistence concerns
-

Positive Consequences

- Improved maintainability
- Reduced coupling
- Better code organization

- Easier debugging
 - Simplified deployment
 - Improved testability
 - Better scalability of the codebase
 - Strong demonstration of Software Engineering concepts
-

Trade-offs

- Independent module deployment is not supported.
 - Strict architectural discipline is required to prevent modules from becoming tightly coupled.
 - All modules share a common deployment lifecycle.
-

Related Design Principles

- Principle 1 – Integration of Core Computer Science Subjects
 - Principle 4 – Functional Architecture Over Team Organization
 - Principle 5 – Module Merging Requires Technical Justification
 - Principle 7 – KPIs Drive the Architecture
 - Principle 10 – No Accidental Decisions
 - Principle 11 – Engineering Over Convenience
-

Related KPIs

This decision directly supports:

- Maintainability
 - Integration
 - Reliability
 - Response Time
 - Scalability
 - Code Quality
-

Related EDRs

This architectural decision is implemented through:

- Repository Pattern
- Layered Package Organization
- Dependency Injection (where applicable)
- Service Layer
- Controller Layer
- Modular Project Structure

(The specific implementation details will be documented in the Engineering Decision Register.)

Architectural View

Online Examination Platform

Layered Modular Monolith Architecture

Presentation Layer



Application Layer



Domain / Business Modules



Persistence Layer



Shared Relational Database

Review

I consider **ADP-001** to be **frozen**.

It is:

- Technology-independent.
- Stable.
- Traceable to the Design Philosophy.
- Aligned with your project KPIs.
- Suitable for inclusion in the final report and SRS.

Architecture Decision Point (ADP-002)

Functional Module Architecture

Status: Approved

1. Context

The Online Examination Platform implements a wide range of business capabilities, including user management, examination administration, question management, online examination execution, evaluation, reporting, monitoring, notifications, security, and system administration.

If these responsibilities are implemented without clear functional boundaries, the system will become tightly coupled, difficult to maintain, and challenging to extend. A modular decomposition is therefore required to ensure that every functional responsibility is owned by exactly one module.

2. Problem Statement

How should the functional responsibilities of the Online Examination Platform be organized into software modules to maximize cohesion, minimize coupling, and clearly demonstrate Software Engineering principles while satisfying the project requirements, KPIs, and expected outcomes?

3. Alternatives Considered

Alternative 1 – Layer-Based Modules

Example:

- Authentication Module
- Database Module
- UI Module

Advantages

- Simple implementation.

Disadvantages

- Business logic becomes scattered.
- High coupling between unrelated features.
- Poor maintainability.

Decision: Rejected.

Alternative 2 – Team-Based Modules

Example:

- Team A Modules
- Team B Modules
- Team C Modules

Advantages

- Easy work distribution.

Disadvantages

- Software architecture becomes dependent on organizational structure.
- Violates functional cohesion.
- Difficult to maintain after development.

Decision: Rejected.

Alternative 3 – Fine-Grained Micro Modules

30–40 independent modules.

Advantages

- Maximum separation.

Disadvantages

- Excessive complexity.
- Increased communication overhead.
- Difficult integration.
- Unnecessary for project scope.

Decision: Rejected.

Alternative 4 – Functional Business Modules

Modules represent complete business capabilities.

Advantages

- High cohesion.
- Low coupling.
- Clear ownership.
- Easier maintenance.
- Easier testing.
- Naturally maps to project requirements.
- Strong Software Engineering demonstration.

Decision: Selected.

4. Decision

The Online Examination Platform shall be organized into **14 cohesive functional modules**, where each module owns a complete business capability and is responsible for its own business rules, data, services, and interfaces.

Each module shall have a clearly defined responsibility, ownership boundary, and interaction contract.

5. Functional Module Architecture

The system shall consist of the following modules.

Module	Primary Responsibility
Identity & Access Management	Authentication, authorization, users, roles, permissions
Student & Faculty Management	Academic users, profiles, eligibility, assignments
Academic Structure Management	Departments, subjects, semesters, academic organization
Examination Administration	Exam creation, scheduling, registration, configuration
Question Bank Management	Question repository, options, blueprint, randomization
Examination Runtime	Live examination execution, submissions, responses
Evaluation & Result Management	Evaluation, grading, results, score computation
Reporting & Analytics	Reports, dashboards, analytics, statistics
Notification Management	Notifications, announcements, communication
Audit & Activity Logging	Audit trails, activity history, compliance
Secure Session Management	Session lifecycle, timeout, concurrent session control
System Monitoring & Health	Monitoring, metrics, alerts, diagnostics
Backup & Recovery	Backup scheduling, restore operations
System Configuration	Global configuration, master settings, application preferences

6. Module Responsibilities

Each module shall:

- Own a single business capability.
 - Maintain high internal cohesion.
 - Expose well-defined interfaces.
 - Hide internal implementation details.
 - Own its business rules.
 - Own its data.
 - Be independently testable.
 - Avoid direct dependency on unrelated modules.
-

7. Module Boundaries

The following architectural rules shall apply:

Rule 1

Every business responsibility belongs to exactly one module.

Rule 2

A module shall never duplicate another module's responsibility.

Rule 3

Business logic shall not be shared through direct database access.

Communication occurs only through defined service interfaces.

Rule 4

Each module owns its internal implementation.

Other modules interact only through published interfaces.

Rule 5

Modules shall remain implementation-independent.

Changes within one module should have minimal impact on others.

8. Architectural Characteristics

The Functional Module Architecture shall exhibit:

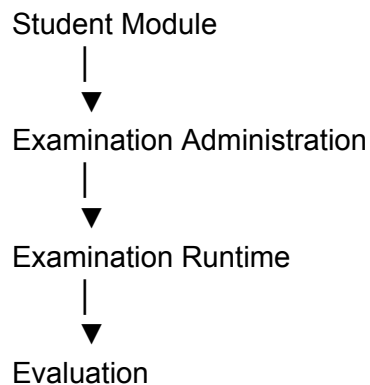
- High Cohesion
 - Low Coupling
 - Separation of Concerns
 - Single Responsibility
 - Clear Ownership
 - Encapsulation
 - Modularity
 - Testability
 - Extensibility
-

9. Module Dependency Philosophy

Dependencies shall flow only toward required business capabilities.

Circular dependencies are prohibited.

Example:





Reporting

Identity & Access Management acts as a shared foundational service for authenticated access but does not own business workflows.

10. Architectural View

Online Examination Platform

Identity & Access Management	
Student & Faculty Management	
Academic Structure Management	
Examination Administration	
Question Bank Management	
Examination Runtime	
Evaluation & Result Management	
Reporting & Analytics	

Cross-Cutting Services

- Notification Management
- Audit & Activity Logging
- Secure Session Management
- System Monitoring & Health
- Backup & Recovery
- System Configuration

11. Positive Consequences

This decision provides:

- Strong functional cohesion.
 - Clear ownership of business capabilities.
 - Simplified maintenance.
 - Easier module testing.
 - Better scalability of the codebase.
 - Reduced coupling.
 - Improved traceability.
 - Better mapping between requirements and implementation.
-

12. Trade-offs

- Requires careful definition of module boundaries.
 - Cross-module interactions must be explicitly managed.
 - Developers must respect ownership rules to prevent architectural erosion.
-

13. Related Design Principles

This decision implements:

- Principle 1 – Integration of Core Computer Science Subjects
 - Principle 4 – Functional Architecture Over Team Organization
 - Principle 5 – Module Merging Requires Technical Justification
 - Principle 7 – KPIs Drive the Architecture
 - Principle 10 – No Accidental Decisions
 - Principle 11 – Engineering Over Convenience
-

14. Related KPIs

This architectural decision directly supports:

- Maintainability
 - Integration
 - Reliability
 - Scalability
 - Code Quality
 - Functional Completeness
-

15. Related Engineering Decisions

This ADP is realized through engineering decisions such as:

- Repository Pattern
 - Service Layer Pattern
 - Feature-Based Package Structure
 - Module-Level Testing Strategy
 - API Boundary Enforcement
-

16. Impact on Future Design

This ADP governs the organization of:

- Database ownership
- API ownership
- Class ownership
- Repository ownership
- Test ownership
- Package structure
- Source code organization
- Requirement Traceability Matrix

Every subsequent design artifact must respect these module boundaries.

Architecture Decision Point (ADP-003)

Layered Software Architecture

Status: Approved

1. Context

The Online Examination Platform consists of multiple business modules that perform different responsibilities, including presentation, request handling, business processing, data access, and infrastructure services.

Without a well-defined layering strategy, business logic can become mixed with user interface code, database access, and infrastructure concerns, resulting in tight coupling, duplicated logic, poor maintainability, and reduced testability.

A layered architecture is therefore required to separate responsibilities, control dependencies, and provide a clear implementation structure.

2. Problem Statement

How should the internal structure of the Online Examination Platform be organized to ensure separation of concerns, maintainability, testability, and controlled dependencies while supporting the functional module architecture defined in ADP-002?

3. Alternatives Considered

Alternative 1 — Two-Tier Architecture

UI
↓

Database

Advantages

- Very simple.

Disadvantages

- Business logic scattered.
- Poor maintainability.
- Difficult testing.
- Tight coupling.

Decision: Rejected.

Alternative 2 — Three-Tier Architecture

Presentation

↓

Business

↓

Database

Advantages

- Better separation.

Disadvantages

- Infrastructure concerns become mixed with business logic.
- Persistence responsibilities are not clearly isolated.

Decision: Rejected.

Alternative 3 — Layered Software Architecture

Separate layers for presentation, application coordination, business logic, persistence, and infrastructure.

Advantages

- High cohesion.
- Low coupling.
- Clear responsibilities.
- Easier testing.
- Easier maintenance.
- Strong Software Engineering demonstration.

Decision: Selected.

4. Decision

The Online Examination Platform shall adopt a **five-layer software architecture** in which each layer has clearly defined responsibilities and communicates only with adjacent lower layers.

Business rules shall remain isolated from presentation, persistence, and infrastructure concerns.

5. Layer Definitions

The system shall consist of five logical layers.

Layer	Responsibility
Presentation Layer	User interface, HTTP requests, responses, session interaction
Application Layer	Request orchestration, workflow coordination, validation, transactions
Domain Layer	Core business rules and business services
Persistence Layer	Data access, repositories, database interaction
Infrastructure Layer	Logging, scheduling, caching, monitoring, configuration, external services

6. Layer Responsibilities

Presentation Layer

Responsible for:

- User Interface
- Request Handling
- Session Validation
- Input Collection
- Response Rendering

It **must not** contain:

- Business Rules
 - Database Logic
 - Complex Validation
-

Application Layer

Responsible for:

- Coordinating business operations
- Managing application workflows
- Transaction boundaries
- Invoking business services
- Coordinating multiple modules

It acts as the bridge between presentation and domain logic.

Domain Layer

This is the heart of the system.

Responsible for:

- Business Rules

- Examination Logic
- Evaluation Logic
- Security Policies
- Validation Rules
- Domain Services

The Domain Layer shall remain independent of presentation frameworks, databases, and infrastructure technologies.

Persistence Layer

Responsible for:

- Repository Interfaces
- Repository Implementations
- Query Execution
- Transaction Support
- Database Mapping

The Persistence Layer shall not contain business rules.

Infrastructure Layer

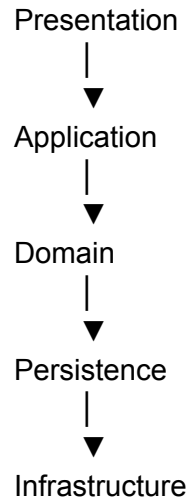
Responsible for:

- Logging
- Scheduling
- File Storage
- Email
- Notifications
- Monitoring
- Configuration
- Backup Support
- Caching

Infrastructure provides technical services to the application but does not define business behavior.

7. Dependency Rules

Dependencies shall always point downward.



Reverse dependencies are prohibited.

8. Architectural Constraints

Constraint 1

Presentation shall never access the database directly.

Constraint 2

Business rules shall never depend on UI components.

Constraint 3

Repositories shall not contain business logic.

Constraint 4

Infrastructure services shall remain reusable and independent of business modules.

Constraint 5

Cross-layer communication shall occur only through well-defined interfaces.

9. Architectural Characteristics

The Layered Architecture provides:

- Separation of Concerns
 - High Cohesion
 - Low Coupling
 - Encapsulation
 - Testability
 - Maintainability
 - Extensibility
 - Reusability
-

10. Architectural View

Presentation Layer

Controllers
Views
Session Handling



Application Layer

Application Services
Workflow Coordination

Transaction Management



Domain Layer

Business Services
Business Rules
Validation
Domain Models



Persistence Layer

Repositories
Data Access
Database Mapping



Infrastructure Layer

Logging
Scheduling
Caching
Monitoring
Configuration

11. Core Computer Science Subject Demonstration

Subject

Architectural Contribution

Software Engineering	Separation of concerns, layered architecture, modular design
Object-Oriented Programming	Encapsulation, abstraction, interface-based design
Database Management Systems	Persistence isolation, repository abstraction
Computer Networks	HTTP request processing, session management, REST communication
Operating Systems	Layer supporting concurrency, scheduling, resource management
Cyber Security	Security enforcement points across presentation, application, and domain layers
Web Technologies	User interaction confined to the presentation layer

12. Positive Consequences

- Clear separation of responsibilities.
 - Simplified maintenance.
 - Improved unit testing.
 - Easier debugging.
 - Reduced coupling.
 - Better scalability of the codebase.
 - Cleaner implementation.
 - Better architectural consistency.
-

13. Trade-offs

- Additional layers introduce more classes.
 - Developers must respect dependency rules.
 - Slight increase in implementation complexity compared to simpler architectures.
-

14. Related Design Principles

- Principle 1 – Integration of Core Computer Science Subjects
 - Principle 2 – Every Design Decision Must Answer Two Questions
 - Principle 7 – KPIs Drive the Architecture
 - Principle 8 – Deliverables Must Be a By-product of the Architecture
 - Principle 10 – No Accidental Decisions
 - Principle 11 – Engineering Over Convenience
-

15. Related KPIs

This decision directly contributes to:

- Maintainability
 - Reliability
 - Code Quality
 - Integration
 - Testability
 - Scalability
-

16. Related Engineering Decisions

This ADP will later be implemented through:

- Service Layer Pattern
 - Repository Pattern
 - Dependency Injection
 - Application Services
 - Modular Package Organization
 - Transaction Management Strategy
-

17. Impact on Future Design

This ADP governs the organization of:

- Project directory structure
- Package organization
- Class hierarchy
- Repository placement
- API implementation
- Database interaction
- Unit testing strategy
- Integration testing

Every implementation artifact must respect these layer boundaries.

Architecture Decision Point (ADP-004)

Module Communication Architecture

Status: Approved

1. Context

The Online Examination Platform is organized into multiple independent functional modules as defined in ADP-002. Although these modules collaborate to deliver complete business workflows, unrestricted communication between them leads to tight coupling, duplicated logic, circular dependencies, and reduced maintainability.

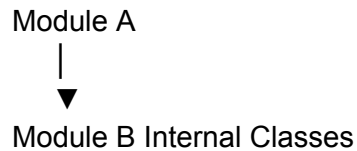
A communication architecture is therefore required to regulate how modules interact while preserving module independence and architectural integrity.

2. Problem Statement

How shall software modules communicate with one another while maintaining high cohesion, low coupling, clear ownership, and architectural consistency throughout the system?

3. Alternatives Considered

Alternative 1 — Direct Class-to-Class Communication



Advantages

- Simple implementation.
- Minimal code.

Disadvantages

- Tight coupling.
- Difficult testing.
- Breaks encapsulation.
- Changes propagate across modules.

Decision: Rejected.

Alternative 2 — Shared Database Communication

Module A



Database



Module B

Advantages

- Easy data access.

Disadvantages

- No ownership.
- Business rules bypassed.
- High coupling.
- Poor maintainability.
- Difficult auditing.

Decision: Rejected.

Alternative 3 — Service Interface Communication

Each module exposes only its public services.

Other modules communicate only through these services.

Advantages

- Loose coupling.
- Clear ownership.
- Easier testing.
- Better maintainability.
- Supports future architectural evolution.

Decision: Selected.

4. Decision

Modules shall communicate exclusively through well-defined service interfaces.

A module shall never directly manipulate another module's internal implementation, repositories, or database objects.

Business workflows involving multiple modules shall be coordinated through the Application Layer.

5. Communication Model

Presentation Layer



Application Service



Business Module A



Business Module B



Business Module C

Notice

The communication is

business driven,

not

database driven.

6. Communication Principles

Principle 1 — Interface-Based Communication

Modules communicate only through published service interfaces.

Internal classes remain private.

Principle 2 — No Direct Database Access

A module shall never access another module's database tables directly.

Data ownership remains with the owning module.

Principle 3 — Business Operations Cross Modules

Communication represents business workflows.

Example

Student Starts Examination

↓

Identity Module

↓

Examination Module

↓

Runtime Module

↓

Audit Module

Not

random method calls.

Principle 4 — Infrastructure Services Are Shared

Infrastructure modules

Logging

Monitoring

Notification

Backup

may be invoked by multiple modules.

However

they do not own business rules.

Principle 5 — Circular Dependencies Are Prohibited

Example

Wrong

Student Module

↓

Examination Module

↓

Student Module

Correct

Student Module

↓

Application Layer

↓

Examination Module

7. Communication Categories

The system supports four categories of communication.

Category	Description
----------	-------------

Presentation → Application	User requests
Application → Business Module	Workflow orchestration
Module → Module	Business collaboration through service interfaces
Module → Infrastructure	Logging, monitoring, notifications, scheduling

8. Cross-Module Workflow Example

Example

Student submits an examination.

Student



Presentation Layer



Exam Runtime Service



Evaluation Service



Result Service



Notification Service



Audit Service



Response Returned

Notice

Every module performs

its own responsibility.

No module owns

the entire workflow.

9. Architectural Rules

Rule 1

Modules shall communicate only through public services.

Rule 2

Modules shall never directly instantiate another module's internal classes.

Rule 3

Business modules shall never directly access another module's repositories.

Rule 4

Cross-module workflows shall be coordinated by the Application Layer.

Rule 5

Modules shall not expose internal implementation details.

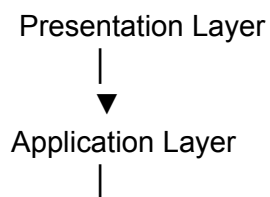
Rule 6

Infrastructure modules shall remain reusable and independent.

Rule 7

All communication shall preserve module ownership boundaries.

10. Architectural View



Identity Module

Academic Module

Examination Module

Question Module

Runtime Module

Evaluation Module

Reporting Module

Infrastructure Services

Logging

Notification

Monitoring

Scheduling

Backup

The Application Layer coordinates communication.

Business modules collaborate without violating ownership.

11. Core Computer Science Subject Demonstration

Subject	Contribution
Software Engineering	Interface-based communication, modularity, dependency management
Object-Oriented Programming	Abstraction, encapsulation, interface segregation
Computer Networks	Request–response architecture, service interaction
Database Management Systems	Data ownership boundaries prevent integrity violations
Operating Systems	Controlled coordination between concurrent services
Cyber Security	Communication passes through centralized authorization points

12. Positive Consequences

- Loose coupling.
- High cohesion.
- Clear ownership.
- Easier testing.

- Better maintainability.
 - Controlled dependencies.
 - Improved scalability of the codebase.
 - Reduced architectural erosion.
-

13. Trade-offs

- More service interfaces must be defined.
 - Slight increase in application layer complexity.
 - Developers must follow communication rules consistently.
-

14. Related Design Principles

- Principle 2 – Every Design Decision Must Answer Two Questions
 - Principle 4 – Functional Architecture Over Team Organization
 - Principle 5 – Module Merging Requires Technical Justification
 - Principle 7 – KPIs Drive the Architecture
 - Principle 10 – No Accidental Decisions
 - Principle 11 – Engineering Over Convenience
-

15. Related KPIs

This decision directly supports:

- Maintainability
 - Integration
 - Reliability
 - Security
 - Scalability
 - Code Quality
-

16. Related Engineering Decisions

This ADP is implemented through:

- Service Layer Pattern
 - Repository Pattern
 - Dependency Injection
 - Application Service Coordination
 - REST Controller Design
-

17. Impact on Future Design

This ADP governs:

- API ownership
- Service interfaces
- Cross-module workflows
- Repository access rules
- Transaction boundaries
- Integration testing
- Future extensibility

Architecture Decision Point (ADP-005)

Domain Architecture & Bounded Contexts

Status: Approved

1. Context

The Online Examination Platform consists of multiple business capabilities that collectively support the lifecycle of an online examination system. These capabilities manage different

aspects of the application, including identity management, academic administration, examination management, runtime execution, evaluation, reporting, and operational support.

Without explicit domain boundaries, business logic becomes scattered across the system, resulting in duplicated responsibilities, inconsistent business rules, and tightly coupled modules.

A domain architecture is therefore required to establish clear business boundaries and ensure that each domain owns a cohesive and well-defined area of responsibility.

2. Problem Statement

How should the business capabilities of the Online Examination Platform be organized into independent domains that maximize cohesion, minimize coupling, preserve ownership, and support future maintainability?

3. Alternatives Considered

Alternative 1 – Single Business Domain

All business logic exists inside one large application.

Advantages

- Simple structure.

Disadvantages

- Extremely high coupling.
- Poor maintainability.
- Difficult ownership.
- Difficult scalability.

Decision: Rejected.

Alternative 2 – Technology-Based Domains

Examples:

- Database Domain
- Authentication Domain
- UI Domain

Advantages

- Simple organization.

Disadvantages

- Does not reflect business processes.
- Violates separation of business concerns.
- Difficult requirement traceability.

Decision: Rejected.

Alternative 3 – Business-Oriented Bounded Contexts

Business capabilities are grouped into independent domains based on functional ownership.

Advantages

- High cohesion.
- Clear ownership.
- Better maintainability.
- Easier testing.
- Excellent Software Engineering demonstration.

Decision: Selected.

4. Decision

The Online Examination Platform shall be organized into **business-oriented bounded contexts**, where each domain owns a complete business capability, including its business rules, services, data, interfaces, and validation logic.

Each domain shall remain independent while collaborating with other domains through well-defined service interfaces.

5. Domain Architecture

The system shall consist of the following business domains.

Domain	Business Responsibility
Identity & Access	Authentication, authorization, users, roles, permissions
Academic Management	Students, faculty, departments, semesters, subjects
Examination Management	Examination planning, scheduling, registration, configuration
Question Management	Question bank, options, blueprints, randomization
Examination Runtime	Live examination sessions, submissions, responses
Evaluation & Results	Evaluation, grading, marks, results
Reporting & Analytics	Reports, dashboards, analytics
Platform Services	Notifications, audit logging, monitoring, backup, configuration

6. Domain Responsibilities

Each domain owns:

- Business Rules
- Business Services
- Validation Rules
- Data Model
- Repository Interfaces
- Public Service Interfaces
- Business Workflows

Each domain does **not** own:

- Another domain's business rules.
 - Another domain's database tables.
 - Another domain's validation logic.
-

7. Domain Relationships

The high-level business flow follows the natural lifecycle of an examination.

Identity & Access

|



Academic Management

|



Examination Management

|



Question Management

|



Examination Runtime

|



Evaluation & Results

|



Reporting & Analytics

Platform Services operate across all domains without owning business processes.

8. Domain Boundaries

Each domain represents a **bounded context**.

Within a bounded context:

- Terminology is consistent.
- Business rules are consistent.
- Data ownership is clear.
- Internal implementation remains private.

Outside the bounded context:

- Communication occurs only through published interfaces.
 - Internal implementation details are hidden.
-

9. Architectural Rules

Rule 1

Every business concept belongs to exactly one domain.

Rule 2

Every business rule is implemented only within its owning domain.

Rule 3

Every domain owns its own services.

Rule 4

Every domain owns its own data.

Rule 5

Business terminology shall remain consistent within each domain.

Rule 6

Domains shall never duplicate responsibilities.

Rule 7

Cross-domain workflows shall preserve ownership boundaries.

10. Domain Collaboration

Domains collaborate only when necessary.

Example

Student Starts Examination



Identity Domain



Examination Domain



Runtime Domain



Evaluation Domain



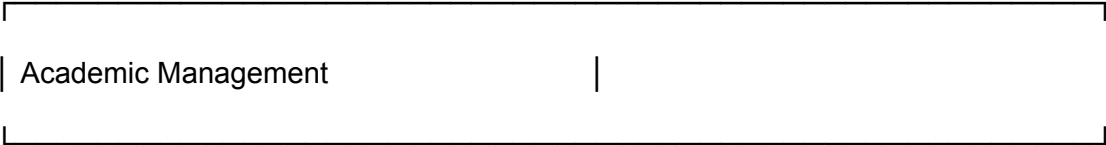
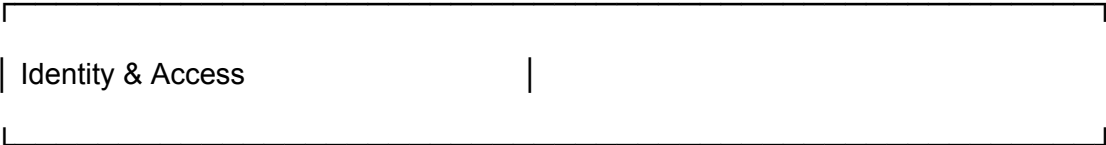
Reporting Domain

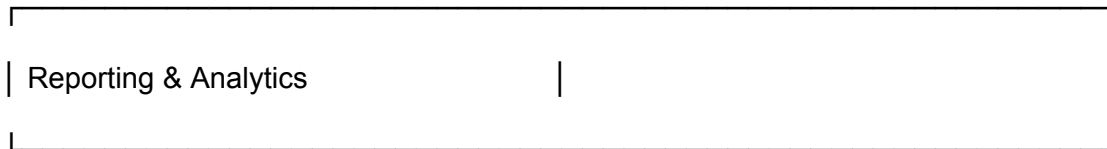
Each domain performs only its own responsibility.

11. Architectural View

Online Examination Platform

Business Domain Architecture





Platform Services

- Notifications
- Audit Logging
- Monitoring
- Backup & Recovery
- Configuration

12. Core Computer Science Subject Demonstration

Subject	Architectural Contribution
Software Engineering	Domain decomposition, bounded contexts, modularity
Object-Oriented Programming	Encapsulation of business responsibilities
Database Management Systems	Domain-driven data ownership
Computer Networks	Service-oriented domain communication
Operating Systems	Independent domain execution and coordination
Cyber Security	Domain-specific security enforcement
Web Technologies	Clear mapping between UI and business domains

13. Positive Consequences

- High business cohesion.
- Clear ownership.
- Easier maintenance.

- Better scalability of the codebase.
 - Simplified testing.
 - Improved traceability.
 - Better requirement mapping.
 - Reduced coupling.
-

14. Trade-offs

- Requires careful identification of domain boundaries.
 - Cross-domain workflows require coordination.
 - Developers must understand ownership responsibilities.
-

15. Related Design Principles

- Principle 1 – Integration of Core Computer Science Subjects
 - Principle 4 – Functional Architecture Over Team Organization
 - Principle 5 – Module Merging Requires Technical Justification
 - Principle 7 – KPIs Drive the Architecture
 - Principle 10 – No Accidental Decisions
 - Principle 11 – Engineering Over Convenience
-

16. Related KPIs

This decision supports:

- Maintainability
 - Integration
 - Reliability
 - Scalability
 - Code Quality
 - Functional Completeness
-

17. Related Engineering Decisions

This ADP will be implemented through:

- Feature-based package organization
 - Service layer architecture
 - Repository ownership
 - API ownership
 - Database ownership
 - Domain-specific validation
-

18. Impact on Future Design

This ADP governs:

- Database schema organization
- Repository ownership
- API ownership
- Class organization
- Package structure
- Transaction boundaries
- Requirement Traceability Matrix
- Testing strategy

Every future design artifact must preserve these domain boundaries.

Architecture Decision Point (ADP-006)

Module Ownership & Data Ownership

Status: Approved

1. Context

The Online Examination Platform is composed of multiple business domains that collaborate to deliver complete examination workflows. Each domain manages a specific business capability and requires ownership over its data, business rules, services, and interfaces.

Without clearly defined ownership, multiple modules may implement the same business logic, directly manipulate each other's data, or violate architectural boundaries, resulting in tight coupling, inconsistent behavior, and poor maintainability.

A formal ownership model is therefore required.

2. Problem Statement

How shall ownership of business rules, services, APIs, repositories, and data be assigned to ensure clear responsibility, preserve architectural integrity, and maintain consistency throughout the system?

3. Alternatives Considered

Alternative 1 – Shared Ownership

Multiple modules may modify the same entities.

Advantages

- Easy implementation.
- Fewer service calls.

Disadvantages

- Conflicting business rules.
- Poor traceability.
- High coupling.
- Difficult debugging.

Decision: Rejected.

Alternative 2 – Database-Centric Ownership

Ownership determined by database tables.

Advantages

- Simple relational mapping.

Disadvantages

- Business ownership becomes unclear.
- Violates domain-driven design.
- Encourages direct database coupling.

Decision: Rejected.

Alternative 3 – Domain Ownership

Each business domain owns:

- Business rules
- Services
- APIs
- Data
- Validation
- Repositories

Other domains interact only through public services.

Advantages

- High cohesion.
- Low coupling.
- Clear traceability.
- Excellent maintainability.

Decision: Selected.

4. Decision

Each business capability shall have **exactly one owning module**.

The owning module has exclusive authority to:

- Define business rules.
- Modify business data.
- Validate business constraints.
- Publish service interfaces.
- Expose APIs.
- Manage repositories.

Other modules may **consume** these services but shall never bypass the ownership boundary.

5. Ownership Principles

Principle 1 — Single Ownership

Every business entity belongs to one and only one module.

Principle 2 — Single Source of Truth

Business rules shall exist only within the owning module.

Principle 3 — Data Ownership

Only the owning module may perform create, update, or delete operations on its entities.

Principle 4 — Read Without Ownership

Other modules may read information through public services when required but shall not modify it.

Principle 5 — Encapsulation

Internal implementation details remain private to the owning module.

Principle 6 — Controlled Collaboration

Cross-module business processes are coordinated through the Application Layer.

6. Ownership Matrix

Identity & Access Management

Owns	Does Not Own
Users	Students
Roles	Questions
Permissions	Results
Credentials	Subjects
Sessions	Examinations

Academic Management

Owns	Does Not Own
Students	Users
Faculty	Questions
Departments	Results
Subjects	Sessions
Semesters	Audit Logs

Examination Management

Owns	Does Not Own
Examinations	Students
Registration	Users

Scheduling Results

Configuration Notifications

Question Management

Owns **Does Not
Own**

Questions Students

Options Sessions

Blueprints Results

Difficulty Users

Topics Notifications

Examination Runtime

Owns **Does Not
Own**

Exam Sessions Questions

Responses Subjects

Submissions Roles

Runtime State Departments

Evaluation & Results

Owns **Does Not Own**

Evaluation Questions

Marks Sessions

Grades Users

Results Departments

Reporting & Analytics

Owns	Does Not Own
Reports	Business Data
Dashboards	Business Rules
Analytics	Transactions

Reports consume business information but do not own it.

Platform Services

Owns:

- Notifications
- Audit Logs
- Monitoring
- Configuration
- Backup
- Health Metrics

These modules observe business activities but do not own business workflows.

7. Ownership Hierarchy

Business Requirement



Business Domain

|



Business Service

|



Repository

|



Database Tables

Ownership flows downward.

It never flows upward.

8. Ownership Rules

Rule 1

Only the owning module may modify its business entities.

Rule 2

Business validation occurs only inside the owning module.

Rule 3

Repositories belong to their owning module.

Rule 4

REST endpoints belong to the module that owns the business capability.

Rule 5

Shared utility classes never contain business rules.

Rule 6

Infrastructure services never own business data.

Rule 7

Every entity in the database shall have one owning module.

9. Cross-Domain Example

Student submits an examination.

Identity

|



Runtime

|



Evaluation

|



Reporting

Ownership remains:

Domain	Owns
Identity	Authentication
Runtime	Submission
Evaluation	Marks
Reporting	Analytics

No domain performs another domain's responsibility.

10. Architectural View

Business Ownership

Identity

- |
- └─ Users
- └─ Roles
- └─ Sessions

Academic

- |
- |— Students
- |— Faculty
- |— Subjects

Examination

- |
- |— Exams
- |— Registration
- |— Scheduling

Question

- |
- |— Questions
- |— Options

Runtime

- |
- |— Sessions
- |— Responses
- |— Submissions

Evaluation

- |

- └─ Marks
- └─ Grades
- └─ Results

Reporting

- |
- └─ Reports
- └─ Dashboards

Platform Services

- |
- └─ Audit
- └─ Notifications
- └─ Monitoring
- └─ Backup

11. Core Computer Science Subject Demonstration

Subject	Architectural Contribution
Software Engineering	Single Responsibility, ownership, modularity, separation of concerns

Object-Oriented Programming	Encapsulation, information hiding, interface ownership
Database Management Systems	Entity ownership, integrity, controlled updates
Computer Networks	API ownership and service contracts
Operating Systems	Controlled coordination between modules
Cyber Security	Authorization boundaries aligned with ownership

12. Positive Consequences

- Clear responsibilities.
 - No duplicated business rules.
 - Reduced coupling.
 - Easier testing.
 - Better maintainability.
 - Better traceability.
 - Cleaner database design.
 - Simplified API ownership.
-

13. Trade-offs

- Cross-module operations require coordination.
 - Service interfaces must be carefully designed.
 - Developers must respect ownership rules.
-

14. Related Design Principles

- Principle 2 – Every Design Decision Must Answer Two Questions
 - Principle 4 – Functional Architecture Over Team Organization
 - Principle 5 – Module Merging Requires Technical Justification
 - Principle 7 – KPIs Drive the Architecture
 - Principle 10 – No Accidental Decisions
 - Principle 11 – Engineering Over Convenience
-

15. Related KPIs

This decision directly supports:

- Maintainability
 - Database Integrity
 - Reliability
 - Integration
 - Security
 - Code Quality
-

16. Related Engineering Decisions

This ADP is implemented through:

- Repository Pattern
 - Service Layer Pattern
 - Database Schema Ownership
 - REST API Ownership
 - Transaction Boundaries
-

17. Impact on Future Design

This ADP governs:

- Database table ownership.
- Repository ownership.
- API ownership.
- Class ownership.
- Service ownership.
- Transaction ownership.
- Test ownership.
- Requirement Traceability Matrix.

Every entity, repository, API, and service created in later phases must be traceable to exactly one owning module.

Architecture Decision Point (ADP-007)

Data & Persistence Architecture

Status: Approved

1. Context

The Online Examination Platform manages persistent business information including users, examinations, questions, student responses, results, audit logs, and system configuration.

The persistence mechanism must provide reliable storage while preserving the architectural boundaries established in previous ADPs. Business logic must remain independent of database technologies and persistence implementations.

A dedicated persistence architecture is therefore required to separate business processing from data storage concerns.

2. Problem Statement

How shall persistent data be managed so that the system maintains data integrity, architectural independence, maintainability, and efficient database interaction without exposing persistence concerns to business modules?

3. Alternatives Considered

Alternative 1 – Direct Database Access

Business services execute SQL directly.

Advantages

- Simple implementation.
- Minimal abstraction.

Disadvantages

- Business logic tightly coupled to the database.
- Difficult testing.
- High code duplication.
- Poor maintainability.

Decision: Rejected.

Alternative 2 – Active Record Pattern

Business entities contain both business logic and persistence logic.

Advantages

- Reduced amount of code.
- Simple CRUD operations.

Disadvantages

- Mixing of responsibilities.
- Violates Separation of Concerns.
- Business logic depends on persistence.

Decision: Rejected.

Alternative 3 – Repository-Based Persistence

Repositories isolate persistence operations from business logic.

Advantages

- Clear separation of concerns.
- Easier testing.
- Technology independence.
- Better maintainability.
- Strong Software Engineering demonstration.

Decision: Selected.

4. Decision

The Online Examination Platform shall adopt a **Repository-Based Persistence Architecture** in which all database interactions are encapsulated within the Persistence Layer.

Business modules shall interact only with repository interfaces and shall remain independent of SQL, ORM implementations, and database-specific details.

5. Persistence Architecture

Presentation Layer

|



Application Layer

|



Business Services



Repository Interfaces



Repository Implementations



Database

The Domain Layer knows **what** data it needs.

The Persistence Layer knows **how** to retrieve or store it.

6. Persistence Responsibilities

The Persistence Layer is responsible for:

- Data retrieval
- Data storage
- Query execution
- Transaction participation
- Entity mapping
- Repository implementation
- Data consistency
- Database interaction

The Persistence Layer shall not:

- Implement business rules.
- Perform business validation.

- Make workflow decisions.
-

7. Data Access Principles

Principle 1 — Repository Ownership

Each business module owns its repositories.

Principle 2 — Persistence Transparency

Business services shall remain unaware of database implementation details.

Principle 3 — Controlled Data Access

Database access occurs only through repositories.

Principle 4 — Transaction Consistency

Persistence operations participating in one business operation shall execute within a single transaction boundary.

Principle 5 — Encapsulation

Database implementation details remain private to the Persistence Layer.

8. Repository Ownership Matrix

Module	Repository Ownership
Identity & Access	UserRepository, RoleRepository, SessionRepository
Academic Management	StudentRepository, FacultyRepository, SubjectRepository
Examination Management	ExaminationRepository, RegistrationRepository
Question Management	QuestionRepository, BlueprintRepository
Examination Runtime	SessionRepository, SubmissionRepository
Evaluation & Results	EvaluationRepository, ResultRepository
Reporting & Analytics	ReportRepository
Platform Services	AuditRepository, NotificationRepository, ConfigurationRepository

9. Transaction Architecture

Business transactions are controlled by the **Application Layer**.

Example:

Submit Examination



Validate Submission



Store Student Responses



Create Submission Record



Evaluate Answers



Generate Result



Create Audit Record



Commit Transaction

Either the entire business operation succeeds, or it is rolled back.

10. Query Responsibility

The Persistence Layer is responsible for:

- CRUD operations
- Optimized queries
- Pagination
- Filtering
- Sorting
- Aggregate queries
- Index utilization

Complex business decisions remain in the Domain Layer.

11. Architectural Rules

Rule 1

Business services shall never execute SQL directly.

Rule 2

Repositories shall not contain business rules.

Rule 3

Every repository belongs to exactly one module.

Rule 4

Persistence shall remain independent of presentation logic.

Rule 5

Business entities shall not depend on database-specific implementations.

Rule 6

Transactions shall represent complete business operations rather than individual SQL statements.

Rule 7

Database integrity shall be enforced through both application validation and database constraints.

12. Architectural View

Business Modules

|



Repository Interfaces

|



Persistence Layer

|



Relational Database

Only the Persistence Layer communicates directly with the database.

13. Core Computer Science Subject Demonstration

Subject	Architectural Contribution
Software Engineering	Repository pattern, separation of concerns, layered architecture
Object-Oriented Programming	Interface abstraction, encapsulation, polymorphism
Database Management Systems	Data integrity, normalization, transactions, indexing
Operating Systems	Transaction coordination and resource management
Computer Networks	Persistence supporting request-response operations
Cyber Security	Controlled database access and least-privilege persistence

14. Positive Consequences

- Clear separation between business logic and persistence.
 - Easier database maintenance.
 - Improved testability.
 - Better scalability of the codebase.
 - Simplified migration to different persistence technologies.
 - Strong demonstration of DBMS and Software Engineering concepts.
-

15. Trade-offs

- Additional abstraction through repositories.
 - More classes to implement.
 - Requires disciplined transaction management.
-

16. Related Design Principles

- Principle 2 – Every Design Decision Must Answer Two Questions
 - Principle 6 – Every Subject Must Have a Natural Contribution
 - Principle 7 – KPIs Drive the Architecture
 - Principle 8 – Deliverables Must Be a By-product of the Architecture
 - Principle 10 – No Accidental Decisions
 - Principle 11 – Engineering Over Convenience
-

17. Related KPIs

This decision directly supports:

- Database Integrity
 - Maintainability
 - Response Time
 - Reliability
 - Code Quality
 - Scalability
-

18. Related Engineering Decisions

This ADP will be realized through:

- Repository Pattern
 - SQLAlchemy ORM
 - Parameterized SQL
 - Transaction Management
 - Connection Pooling
 - Database Migration Strategy
-

19. Impact on Future Design

This ADP governs:

- Database schema implementation.
- Repository interfaces.
- Repository implementations.
- Transaction boundaries.
- Query optimization.
- ORM usage.
- Database testing.
- Data migration strategy.

Every database interaction in the project shall conform to this persistence architecture.

Architecture Decision Point (ADP-008)

Security Architecture

Status: Approved

1. Context

The Online Examination Platform processes sensitive information including user credentials, examination content, student responses, evaluation results, and administrative operations.

The platform must protect the confidentiality, integrity, and availability of system resources while preventing unauthorized access, data manipulation, session abuse, and common web application attacks.

Security cannot be treated as a standalone module. It must be integrated across every architectural layer and business domain.

2. Problem Statement

How shall security be incorporated into the architecture so that it consistently protects business operations, system resources, and sensitive information while supporting maintainability and the project's educational objectives?

3. Alternatives Considered

Alternative 1 – Perimeter Security

Protect only login functionality and assume authenticated users are trusted.

Advantages

- Simple implementation.
- Minimal overhead.

Disadvantages

- Weak internal protection.
- Authorization inconsistencies.
- Large attack surface after login.

Decision: Rejected.

Alternative 2 – Module-Specific Security

Each module independently implements authentication and authorization.

Advantages

- Local control.

Disadvantages

- Duplicated security logic.
- Inconsistent enforcement.
- Difficult maintenance.
- Increased implementation errors.

Decision: Rejected.

Alternative 3 – Layered Security Architecture

Security is enforced consistently across presentation, application, domain, persistence, and infrastructure layers.

Advantages

- Centralized enforcement.
- Consistent authorization.
- Better maintainability.
- Defense in depth.
- Clear separation of responsibilities.

Decision: Selected.

4. Decision

The Online Examination Platform shall implement a **Layered Security Architecture** in which security controls are enforced throughout the system rather than concentrated in a single component.

Authentication, authorization, validation, auditing, and secure data handling shall operate as cross-cutting architectural concerns.

5. Security Architecture

User Request



Presentation Layer

- Secure Session Validation
 - CSRF Protection
 - Input Validation
-



Application Layer

- Authentication
 - Authorization
 - Workflow Validation
-



Domain Layer

- Business Rule Validation
 - Permission Verification
-



Persistence Layer

- Parameterized Queries
 - Transaction Integrity
-



Infrastructure Layer

- Audit Logging
- Monitoring
- Security Logging

Security exists at every layer.

6. Security Principles

Principle 1 — Defense in Depth

Multiple independent security mechanisms shall protect the system.

Failure of one mechanism shall not compromise the entire application.

Principle 2 — Authentication Before Access

Every protected operation requires an authenticated identity.

Principle 3 — Authorization Before Execution

Authentication identifies the user.

Authorization determines whether the requested operation is permitted.

Principle 4 — Least Privilege

Users receive only the permissions required for their responsibilities.

Principle 5 — Secure by Default

If authorization cannot be verified, access shall be denied.

Principle 6 — Validate All External Input

Every request entering the system shall be validated before business processing.

Principle 7 — Audit Sensitive Operations

Critical operations shall generate immutable audit records.

Principle 8 — Fail Securely

Unexpected failures shall never expose confidential information or bypass security controls.

7. Security Layers

Layer	Responsibility
Presentation	Session validation, CSRF protection, request validation
Application	Authentication, authorization, workflow security
Domain	Business rule enforcement
Persistence	Secure database access, transactions
Infrastructure	Audit logging, monitoring, security events

8. Security Responsibilities

Security shall protect:

- User identities
 - Authentication credentials
 - Examination questions
 - Student responses
 - Results
 - Administrative operations
 - Configuration data
 - Audit records
-

9. Architectural Rules

Rule 1

Every protected request shall pass authentication.

Rule 2

Authorization shall be verified before executing business operations.

Rule 3

Business modules shall never implement independent authentication mechanisms.

Rule 4

Security decisions shall remain centralized and consistent.

Rule 5

Sensitive information shall never be stored or transmitted in plain text.

Rule 6

Security logging shall be independent of business processing.

Rule 7

Every security-sensitive operation shall be auditable.

10. Security Domains

The architecture protects four categories of assets.

Category	Examples
Identity	Users, Roles, Permissions
Business Data	Questions, Responses, Results
System Resources	Sessions, APIs, Database
Operational Data	Audit Logs, Monitoring, Configuration

11. Core Computer Science Subject Demonstration

Subject	Architectural Contribution
Cyber Security	Authentication, authorization, defense in depth, least privilege
Software Engineering	Cross-cutting architectural concern, centralized security

Object-Oriented Programming Encapsulation of security services

Database Management Systems Secure persistence, controlled access, transaction integrity

Computer Networks Secure request processing, session protection

Operating Systems Resource protection and controlled access

12. Positive Consequences

- Consistent security enforcement.
 - Reduced attack surface.
 - Easier maintenance.
 - Improved traceability.
 - Strong audit capabilities.
 - Better regulatory compliance.
 - Strong demonstration of Cyber Security concepts.
-

13. Trade-offs

- Additional security processing.
 - Increased implementation effort.
 - More architectural components.
 - Requires disciplined enforcement.
-

14. Related Design Principles

- Principle 2 – Every Design Decision Must Answer Two Questions
 - Principle 6 – Every Subject Must Have a Natural Contribution
 - Principle 7 – KPIs Drive the Architecture
 - Principle 9 – Think Like Software Engineers
 - Principle 10 – No Accidental Decisions
 - Principle 11 – Engineering Over Convenience
-

15. Related KPIs

This decision directly supports:

- Security
 - Reliability
 - Database Integrity
 - Code Quality
 - Maintainability
 - System Availability
-

16. Related Engineering Decisions

This ADP will be realized through engineering decisions such as:

- Server-side Session Management
 - Database-driven RBAC
 - Argon2id Password Hashing
 - CSRF Protection
 - Input Validation
 - Parameterized SQL
 - Audit Logging
 - Secure Cookie Configuration
-

17. Impact on Future Design

This ADP governs:

- Authentication design.
- Authorization model.
- Session management.
- API protection.
- Database access control.
- Audit logging.
- Input validation.
- Security testing.
- Threat mitigation strategy.

Every subsequent design artifact must comply with this security architecture.

Architecture Decision Point (ADP-009)

Concurrency & Resource Management Architecture

Status: Approved

1. Context

The Online Examination Platform must support multiple users performing operations simultaneously, including authentication, examination participation, question retrieval, answer submission, result generation, report creation, audit logging, and notification delivery.

The system must coordinate concurrent activities while preserving data integrity, maximizing resource utilization, preventing race conditions, and maintaining acceptable response times.

Concurrency and resource management are therefore fundamental architectural concerns rather than implementation details.

2. Problem Statement

How shall the architecture manage concurrent execution, shared resources, synchronization, and background processing to ensure correctness, scalability, efficient resource utilization, and reliable system behavior?

3. Alternatives Considered

Alternative 1 – Sequential Execution

All requests execute one after another.

Advantages

- Very simple.
- No synchronization required.

Disadvantages

- Poor performance.
- Low throughput.
- Unacceptable user experience.
- Inefficient CPU utilization.

Decision: Rejected.

Alternative 2 – Uncontrolled Multi-threading

Every operation creates its own thread.

Advantages

- High concurrency.

Disadvantages

- Thread explosion.
- High memory consumption.
- Difficult synchronization.
- Poor scalability.

Decision: Rejected.

Alternative 3 – Managed Concurrent Architecture

The system uses controlled concurrency with managed request processing, synchronized access to shared resources, scheduled background jobs, and clearly defined transaction boundaries.

Advantages

- Efficient CPU utilization.
- Controlled synchronization.
- Better scalability.
- Improved maintainability.
- Strong demonstration of Operating Systems concepts.

Decision: Selected.

4. Decision

The Online Examination Platform shall adopt a **Managed Concurrent Architecture** in which user requests execute concurrently under controlled synchronization, shared resources are protected through appropriate coordination mechanisms, and long-running or periodic operations execute as managed background tasks.

5. Concurrency Architecture

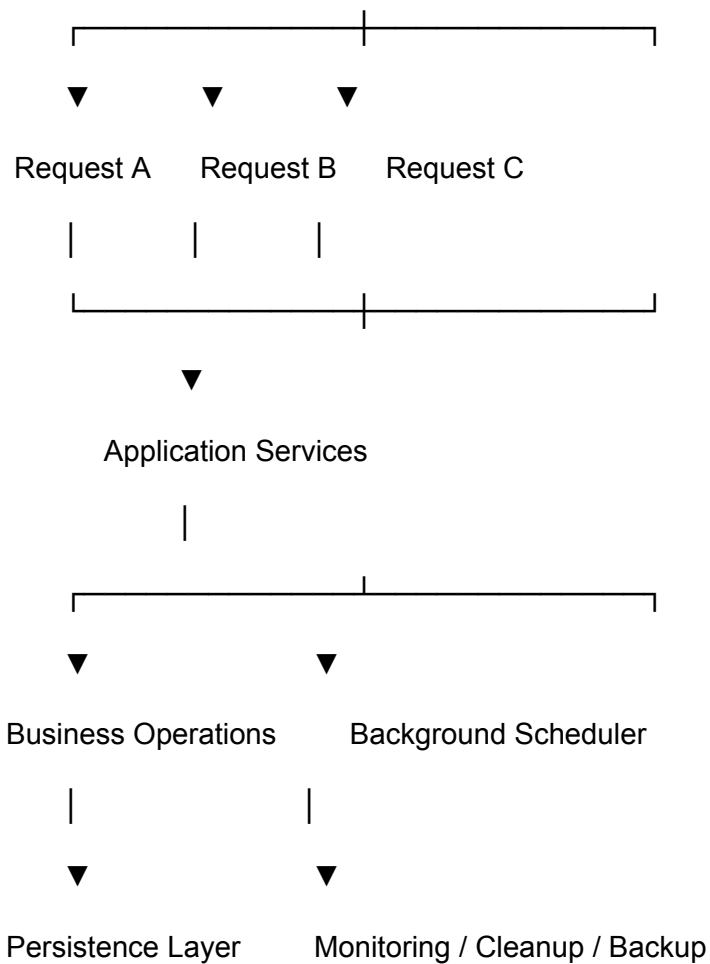
Client Requests

|



Web Request Handler

|



6. Concurrency Principles

Principle 1 — Concurrent Request Processing

Independent client requests shall execute concurrently.

Principle 2 — Business Consistency

Concurrent execution shall never violate business rules or data integrity.

Principle 3 — Synchronization of Shared Resources

Whenever multiple execution paths access shared mutable resources, synchronization shall preserve consistency.

Principle 4 — Transactional Consistency

Business transactions shall remain atomic regardless of concurrent execution.

Principle 5 — Background Processing

Long-running or periodic operations shall execute independently of user requests.

Examples include:

- Session cleanup
 - Notification delivery
 - Backup scheduling
 - Monitoring
 - Log maintenance
-

Principle 6 — Efficient Resource Utilization

Concurrency shall improve throughput without creating unnecessary CPU, memory, or thread overhead.

Principle 7 — Predictable Execution

The architecture shall favor deterministic behavior by minimizing race conditions and ensuring controlled access to shared resources.

7. Resource Categories

The architecture manages the following resources.

Resource	Examples
CPU	Request processing, evaluation, reporting
Memory	Session state, caching, runtime objects
Database Connections	Transactions, queries
Application Threads	Concurrent request execution
Scheduled Tasks	Maintenance and monitoring jobs

8. Background Processing Responsibilities

Background processing shall be used for operations that are:

- Periodic
- Independent
- Non-interactive
- Time-consuming

Examples:

- Session expiration
 - Audit archival
 - Backup execution
 - Health monitoring
 - Notification dispatch
 - Cache cleanup
-

9. Synchronization Strategy

Synchronization shall be applied only when required.

Typical synchronization points include:

- Simultaneous examination submissions
- Result publication
- Session state updates
- Shared cache updates
- Configuration modifications

Read-only operations should remain lock-free wherever practical.

10. Architectural Rules

Rule 1

Business operations shall support concurrent execution.

Rule 2

Shared mutable resources shall be synchronized appropriately.

Rule 3

Long-running operations shall not block user requests.

Rule 4

Background tasks shall execute independently of interactive workflows.

Rule 5

Business transactions shall remain atomic under concurrent execution.

Rule 6

Concurrency shall improve performance without compromising correctness.

Rule 7

Resource allocation shall remain proportional to workload.

11. Resource Management Model

Incoming Requests

|

▼

Application Services

|

▼

Business Operations

|

▼

Shared Resources

Database

Sessions

Cache

Configuration



Synchronization

Transactions

Locks (where necessary)

Consistency Rules

12. Core Computer Science Subject Demonstration

Subject	Architectural Contribution
Operating Systems	Concurrency, synchronization, scheduling, resource management, CPU utilization
Software Engineering	Separation of interactive and background processing

Database Management Systems	Transaction coordination and concurrent data consistency
-----------------------------	--

Object-Oriented Programming	Encapsulation of concurrent services
-----------------------------	--------------------------------------

Computer Networks	Concurrent client request handling
-------------------	------------------------------------

Cyber Security	Controlled concurrent access to protected resources
----------------	---

13. Positive Consequences

- Supports multiple concurrent users.
 - Better CPU utilization.
 - Improved responsiveness.
 - Controlled resource usage.
 - Reliable background processing.
 - Strong Operating Systems demonstration.
 - Improved scalability.
-

14. Trade-offs

- Increased implementation complexity.
 - Requires synchronization design.
 - More comprehensive testing is needed.
 - Concurrent defects may be more difficult to debug.
-

15. Related Design Principles

- Principle 1 – Integration of Core Computer Science Subjects
 - Principle 2 – Every Design Decision Must Answer Two Questions
 - Principle 7 – KPIs Drive the Architecture
 - Principle 10 – No Accidental Decisions
 - Principle 11 – Engineering Over Convenience
 - Principle 12 – Measurable Optimizations
-

16. Related KPIs

This decision directly supports:

- Response Time
 - Concurrent Users
 - CPU Utilization
 - Resource Utilization
 - Reliability
 - Performance
 - Scalability
-

17. Related Engineering Decisions

This ADP will be realized through:

- Background Scheduler
- Thread-safe Session Management
- Transaction Management
- Database Connection Pooling
- Custom Cache Synchronization
- Scheduled Maintenance Tasks

(Specific technologies such as APScheduler or Flask request handling are implementation decisions and will be documented in the Engineering Decision Register rather than in this ADP.)

18. Impact on Future Design

This ADP governs:

- Request processing architecture.
- Background job architecture.
- Synchronization strategy.
- Session lifecycle.
- Cache management.
- Transaction coordination.
- Performance optimization.
- Operating Systems demonstrations.
- Concurrency testing.

Every component that accesses shared resources or executes concurrently must comply with this architecture.

Architecture Decision Point (ADP-010)

Runtime & Deployment Architecture

Status: Approved

1. Context

The Online Examination Platform must execute as a reliable, maintainable, and secure software system capable of supporting multiple concurrent users while remaining simple to deploy, operate, and maintain within the project's scope.

The runtime architecture defines how the application executes after deployment, how its components are initialized, how configuration is managed, how operational services function, and how the system behaves during startup, execution, monitoring, and shutdown.

2. Problem Statement

How shall the Online Examination Platform be deployed and operated so that runtime behavior remains reliable, maintainable, secure, and consistent with the project's architectural principles?

3. Alternatives Considered

Alternative 1 – Distributed Deployment (Microservices)

Independent deployment of multiple services.

Advantages

- Independent scaling.
- Service isolation.

Disadvantages

- High operational complexity.
- Distributed communication.
- Multiple deployment units.
- Beyond project scope.

Decision: Rejected.

Alternative 2 – Traditional Monolithic Deployment

Single executable with minimal internal structure.

Advantages

- Simple deployment.

Disadvantages

- Poor modular organization.
- Limited operational flexibility.

Decision: Rejected.

Alternative 3 – Layered Modular Monolith Runtime

A single deployable application internally organized into independent modules operating under a unified runtime environment.

Advantages

- Simple deployment.
- Clear modularity.
- Centralized configuration.
- Easier maintenance.
- Suitable for project scope.

Decision: Selected.

4. Decision

The Online Examination Platform shall execute as a **single deployable layered modular application** operating within a unified runtime environment.

All functional modules shall execute within the same application process while maintaining logical independence through the architectural boundaries established in previous ADPs.

5. Runtime Architecture

Client Browser

|



HTTP Request Handler

|



Layered Modular Monolith Runtime

Presentation Layer



Application Layer



Business Modules



Persistence Layer



Infrastructure Services

Relational Database

6. Runtime Responsibilities

The runtime environment is responsible for:

- Application startup
 - Module initialization
 - Configuration loading
 - Session management
 - Request processing
 - Background task execution
 - Logging
 - Monitoring
 - Graceful shutdown
-

7. Runtime Principles

Principle 1 — Single Deployable Unit

The application shall execute as one deployable software system.

Principle 2 — Centralized Configuration

Application configuration shall be loaded from a centralized configuration mechanism.

Business logic shall never contain hardcoded environment-specific values.

Principle 3 — Controlled Initialization

Modules shall initialize in a predictable sequence before accepting user requests.

Principle 4 — Operational Independence

Operational services such as logging, monitoring, scheduling, and backup shall remain independent of business workflows.

Principle 5 — Graceful Shutdown

The runtime shall complete active business transactions before terminating.

Principle 6 — Environment Independence

Business logic shall remain independent of deployment environments.

The same architecture shall support:

- Development
- Testing
- Production

through configuration rather than code changes.

Principle 7 — Operational Observability

The runtime environment shall expose sufficient logging and monitoring information to diagnose operational issues.

8. Startup Sequence

Load Configuration

↓

Initialize Logging

↓

Initialize Database

↓

Initialize Infrastructure Services

↓

Initialize Business Modules

↓

Initialize Background Services

↓

Accept Client Requests

9. Shutdown Sequence

Stop Accepting Requests

↓

Complete Active Transactions

↓

Stop Background Tasks

↓

Release Resources

↓

Close Database Connections

↓

Shutdown

10. Operational Services

The runtime environment shall provide:

Service	Purpose
Configuration	Environment-specific settings
Logging	Operational diagnostics
Monitoring	System health
Scheduling	Background processing
Session Management	User sessions
Database Connectivity	Persistent storage
Error Handling	Fault management

11. Runtime Rules

Rule 1

The application shall expose a single entry point.

Rule 2

Business modules shall not perform runtime initialization independently.

Rule 3

Configuration shall remain external to business logic.

Rule 4

Infrastructure services shall initialize before business processing begins.

Rule 5

Operational failures shall not compromise architectural integrity.

Rule 6

Runtime services shall support graceful recovery whenever possible.

Rule 7

Deployment architecture shall remain consistent across environments.

12. Runtime View

Application Runtime

Configuration



Logging



Monitoring



Scheduling



Business Modules



Persistence



Database

13. Core Computer Science Subject Demonstration

Subject	Architectural Contribution
Software Engineering	Deployment architecture, runtime organization, configuration management
Operating Systems	Process lifecycle, resource initialization, shutdown sequencing
Computer Networks	Request processing and client-server runtime
Database Management Systems	Connection lifecycle and persistent storage
Cyber Security	Secure runtime configuration and operational logging
Object-Oriented Programming	Modular runtime initialization

14. Positive Consequences

- Simplified deployment.
- Predictable runtime behavior.

- Centralized operational management.
 - Easier debugging.
 - Better maintainability.
 - Improved reliability.
 - Consistent execution across environments.
-

15. Trade-offs

- Modules cannot be deployed independently.
 - Entire application shares one runtime process.
 - Requires disciplined configuration management.
-

16. Related Design Principles

- Principle 2 – Every Design Decision Must Answer Two Questions
 - Principle 7 – KPIs Drive the Architecture
 - Principle 8 – Deliverables Must Be a By-product of the Architecture
 - Principle 10 – No Accidental Decisions
 - Principle 11 – Engineering Over Convenience
-

17. Related KPIs

This decision directly supports:

- Reliability
 - Maintainability
 - Integration
 - Performance
 - Scalability (within project scope)
 - Operational Stability
-

18. Related Engineering Decisions

This ADP will be implemented through:

- Flask Application Factory
- Configuration Management
- Python Logging Framework
- APScheduler
- Gunicorn (if deployed)
- Nginx (if deployed)
- Docker (optional)

These implementation choices will be documented in the Engineering Decision Register.

19. Impact on Future Design

This ADP governs:

- Application startup.
- Deployment configuration.
- Runtime services.
- Operational logging.
- Monitoring.
- Error handling.
- Deployment documentation.
- Installation guide.
- DevOps documentation.

Every operational aspect of the Online Examination Platform shall conform to this runtime architecture.

Engineering Decision Register

Engineering Decision Register

Section A – Software Architecture

EDR-001 – Feature-Based Project Structure

Status: Approved

Related ADP: ADP-002 – Functional Module Architecture

Problem

A structured project organization is required to ensure scalability, maintainability, and independent development of functional modules.

Alternatives Considered

Alternative 1 – Layer-Based Structure

- Easy for small projects.
- Business logic becomes scattered across folders.
- Difficult to scale.

Decision: Rejected

Alternative 2 – Feature-Based Structure

- Groups all files related to a feature.
- Promotes modularity.
- Easier maintenance.

Decision: Selected

Decision

The project shall adopt a **Feature-Based Project Structure**, where each functional module encapsulates its routes, services, repositories, models, and utilities.

Engineering Justification

Feature-based organization improves cohesion, reduces inter-module dependencies, and aligns with the modular architecture defined in ADP-002.

Measurable Improvement

Metric	Before	After
Code Organization	Scattered	Modular
Maintainability	Medium	High
Coupling	High	Low

Subject Demonstration

Subject	Contribution
Software Engineering	Modular design
OOP	Encapsulation
SDLC	Maintainability

Related KPIs: Maintainability, Scalability

EDR-002 – Layered Package Organization

Related ADP: ADP-003

Decision

Organize each feature into Presentation, Service, Repository, and Persistence layers.

Benefits

- Separation of concerns
 - Easier testing
 - Independent evolution of layers
-

EDR-003 – Service Layer Pattern

Related ADP: ADP-003

Decision

Business rules shall reside exclusively in the Service Layer.

Advantages

- Centralized business logic
- High testability
- Reusable services

Trade-offs

- Additional abstraction layer
 - Slight increase in code size
-

EDR-004 – Repository Pattern

Related ADP: ADP-007

Problem

Database access scattered across modules reduces maintainability.

Decision

Implement the **Repository Pattern** to encapsulate all database interactions.

Measurable Improvement

Metric	Before	After
Database Access	Scattered	Centralized
Testability	Difficult	Easy
Maintainability	Medium	High

EDR-005 – Dependency Injection Strategy

Related ADP: ADP-003

Decision

Dependencies shall be injected into services rather than instantiated internally.

Advantages

- Loose coupling
 - Easier mocking
 - Improved unit testing
-

Section B – Database Engineering

EDR-006 – SQLAlchemy ORM Strategy

Related ADP: ADP-007

Decision

Use SQLAlchemy ORM for entity persistence and object-relational mapping.

Advantages

- Reduced boilerplate
- Object-oriented data access
- Better maintainability

EDR-007 – Hybrid ORM + Parameterized SQL

Related ADP: ADP-007

Problem

ORM is productive but may not be optimal for all queries.

Decision

Use:

- SQLAlchemy ORM for standard CRUD operations.
- Parameterized SQL for complex, performance-critical queries.

Engineering Justification

This hybrid strategy balances developer productivity with query performance.

Measurable Improvement

Metric	ORM Only	Hybrid
Flexibility	Medium	High
Performance	Medium	High
SQL Injection Protection	High	High

EDR-008 – Transaction Management

Related ADP: ADP-007, ADP-009

Decision

All critical business operations shall execute within database transactions.

Advantages

- Atomicity
- Consistency
- Rollback support
- Concurrent safety

KPIs

- Reliability
 - Data Integrity
-

EDR-009 – Database Connection Pooling

Related ADP: ADP-009

Problem

Creating a new database connection for every request increases latency and resource consumption.

Decision

Implement a managed connection pool using SQLAlchemy.

Measurable Improvement

Metric	Without Pool	With Pool
--------	--------------	-----------

Connection Overhead	High	Low
Response Time	Medium	Fast
CPU Utilization	Higher	Lower

EDR-010 – Database Index Strategy

Related ADP: ADP-007

Decision

Create indexes on frequently queried columns such as:

- User ID
- Examination ID
- Question ID
- Result ID
- Session ID

Engineering Justification

Proper indexing significantly reduces query execution time for high-frequency operations.

Measurable Improvement

Metric	Before	After
Lookup Complexity	O(n)	O(log n) (indexed search)
Average Query Time	Higher	Lower
Database Throughput	Medium	High

Related KPIs

- Response Time
- Database Performance
- Scalability

Section C – Security Engineering

EDR-011 – Stateful Session Authentication

Status: Approved

Related ADP: ADP-008 – Security Architecture

Problem

The platform requires secure user authentication while maintaining active user sessions throughout examination activities.

Alternatives Considered

Alternative 1 – JWT Authentication

- Stateless
- Suitable for distributed systems
- Complex logout/session revocation

Decision: Rejected

Alternative 2 – Stateful Session Authentication

- Server-managed sessions
- Simpler session invalidation
- Better suited for modular monolith architecture

Decision: Selected

Decision

The application shall use **server-side stateful session authentication** managed by Flask-Login.

Engineering Justification

Centralized session management improves security, simplifies session lifecycle control, and aligns with the project's runtime architecture.

Measurable Improvement

Metric	JWT	Stateful Session
Logout Control	Moderate	Excellent
Session Revocation	Complex	Simple
Session Tracking	Limited	Complete

Related KPIs: Security, Reliability

EDR-012 – Database-Driven Role-Based Access Control (RBAC)

Related ADP: ADP-008

Decision

User permissions shall be managed through a **database-driven RBAC model**.

Roles

- Administrator
- Faculty
- Student

Advantages

- Centralized permission management
- Easy role modification
- Fine-grained authorization

Subject Demonstration

- Database Management Systems
- Cyber Security

EDR-013 – Argon2id Password Hashing

Related ADP: ADP-008

Problem

Passwords must remain secure even if the credential database is compromised.

Decision

Passwords shall be hashed using **Argon2id** before storage.

Engineering Justification

Argon2id provides resistance against brute-force, GPU, and memory-based attacks while supporting configurable security parameters.

Measurable Improvement

Metric	Plain Password	Argon2id
Confidentiality	None	Very High
Brute Force Resistance	Poor	Excellent
Security	Low	High

Related KPIs

- Security
 - Reliability
-

EDR-014 – CSRF Protection Strategy

Related ADP: ADP-008

Decision

All state-changing HTTP requests shall include CSRF protection tokens.

Benefits

- Prevents forged requests
 - Protects authenticated users
 - Improves application security
-

EDR-015 – Input Validation Strategy

Related ADP: ADP-008

Decision

All external input shall undergo server-side validation before business processing.

Validation Includes

- Required fields
- Data types
- Length limits
- Format validation
- Business rule validation

Advantages

- Prevents malformed input
 - Reduces security vulnerabilities
 - Improves data integrity
-

EDR-016 – Secure Cookie Strategy

Related ADP: ADP-008

Decision

Authentication cookies shall use secure attributes:

- HttpOnly
- Secure
- SameSite

Engineering Benefits

- Protection against XSS
 - Reduced session hijacking risk
 - Secure browser communication
-

EDR-017 – Audit Logging Strategy

Related ADP: ADP-008

Decision

Security-sensitive events shall be recorded in an immutable audit log.

Logged Events

- Login
- Logout
- Password changes
- Role updates
- Examination submission
- Result publication

KPIs

- Security
- Traceability

- Accountability
-

Section D – Operating System Engineering

EDR-018 – Background Scheduling

Related ADP: ADP-009

Problem

Maintenance activities should not block user requests.

Decision

Use a managed background scheduler for periodic operations.

Scheduled Tasks

- Session cleanup
- Notifications
- Backups
- Cache cleanup
- Health monitoring

Engineering Justification

Separating background work from interactive requests improves responsiveness and CPU utilization.

EDR-019 – Thread Safety Strategy

Related ADP: ADP-009

Decision

Shared mutable resources shall be accessed through synchronized mechanisms where necessary.

Protected Resources

- Session store
- Cache
- Configuration
- Shared counters

Advantages

- Prevents race conditions
 - Improves correctness
 - Ensures predictable execution
-

EDR-020 – Session Cleanup Strategy

Related ADP: ADP-009

Decision

Expired sessions shall be removed automatically by scheduled background jobs.

Measurable Improvement

Metric	Manual Cleanup	Automatic Cleanup
Memory Usage	Higher	Lower
Resource Utilization	Medium	High
Reliability	Medium	High

EDR-021 – Resource Management

Related ADP: ADP-009

Decision

Application resources shall be allocated according to workload while minimizing unnecessary CPU, memory, and thread usage.

Managed Resources

- CPU
- Memory
- Database Connections
- Application Threads
- Background Jobs

Engineering Benefits

- Better CPU utilization
 - Improved scalability
 - Stable runtime performance
-

EDR-022 – Cache Synchronization

Related ADP: ADP-009

Problem

Concurrent cache updates may lead to stale or inconsistent data.

Decision

Cache updates shall be synchronized while read operations remain lock-free wherever practical.

Engineering Justification

Selective synchronization balances consistency with performance by avoiding unnecessary locking on read-heavy workloads.

Measurable Improvement

Metric	Unsynchronized	Synchronized
Consistency	Moderate	High
Read Performance	High	High
Data Integrity	Medium	High

Related KPIs

- Response Time
- Reliability
- Concurrency
- Resource Utilization

Section E – API Engineering

EDR-023 – REST API Design

Status: Approved

Related ADP: ADP-004 – Module Communication Architecture

Problem

The application requires a standardized communication mechanism between the presentation layer and business services.

Alternatives Considered

Alternative 1 – RPC Style APIs

- Tight coupling
- Less scalable

Decision: Rejected

Alternative 2 – RESTful APIs

- Resource-oriented
- Standard HTTP methods
- Stateless request handling

Decision: Selected

Decision

All application interfaces shall follow REST architectural principles using standard HTTP methods (GET, POST, PUT, DELETE).

Engineering Justification

REST APIs provide consistency, interoperability, and easier integration with frontend components.

Related KPIs

- Maintainability
 - Scalability
 - Integration
-

EDR-024 – Standard Response Format

Related ADP: ADP-004

Decision

Every API response shall follow a common JSON structure.

Standard Format

```
{  
  "success": true,  
  "message": "Operation completed successfully",  
  "data": { },  
  "errors": null  
}
```

Benefits

- Consistent frontend integration
 - Easier debugging
 - Simplified API documentation
-

EDR-025 – Exception Handling Strategy

Related ADP: ADP-010

Decision

A centralized exception handling mechanism shall manage all application errors.

Categories

- Validation Errors
- Authentication Errors
- Authorization Errors
- Database Errors
- System Errors

Advantages

- Consistent error messages
 - Easier debugging
 - Improved reliability
-

EDR-026 – Validation Pipeline

Related ADP: ADP-008

Decision

Every incoming request shall pass through a validation pipeline before reaching business services.

Validation Stages

1. Input Validation
2. Authentication
3. Authorization
4. Business Rule Validation
5. Persistence

KPIs

- Security
 - Reliability
 - Data Integrity
-

Section F – Performance Engineering

EDR-027 – In-Memory Caching

Related ADP: ADP-009

Problem

Frequently accessed data should not repeatedly query the database.

Decision

Implement an application-level in-memory cache for frequently accessed, read-heavy data.

Cached Data

- Configuration
- Frequently accessed reference data
- Static lookup values

Measurable Improvement

Metric	Without Cache	With Cache
Response Time	Higher	Lower
Database Queries	More	Fewer
CPU Utilization	Higher	Lower

EDR-028 – Pagination Strategy

Related ADP: ADP-007

Decision

Large datasets shall be retrieved using pagination.

Benefits

- Reduced memory usage
- Faster page rendering
- Improved scalability

Related KPIs

- Performance
 - Resource Utilization
-

EDR-029 – Lazy Loading Strategy

Related ADP: ADP-007

Decision

Associated objects shall be loaded only when required.

Engineering Justification

Lazy loading reduces unnecessary database operations and improves application performance.

Measurable Improvement

Metric	Eager Loading	Lazy Loading
Memory Usage	Higher	Lower
Initial Query Time	Higher	Lower

EDR-030 – Query Optimization

Related ADP: ADP-007

Decision

Frequently executed SQL queries shall be optimized through indexing, selective retrieval, and execution plan analysis.

Optimization Techniques

- Index utilization
- Reduced joins
- Selective column retrieval
- Query plan review

KPIs

- Response Time
 - Database Performance
-

Section G – Testing Engineering

EDR-031 – Unit Testing Strategy

Related ADP: ADP-003

Decision

Business services shall be tested independently using automated unit tests.

Scope

- Service Layer
- Utility Classes
- Validation Logic

Benefits

- Early defect detection
 - Easier maintenance
 - Improved reliability
-

EDR-032 – Integration Testing Strategy

Related ADP: ADP-004

Decision

Integration testing shall verify interactions between modules, APIs, and the database.

Coverage

- Authentication Flow
- Examination Workflow
- Result Generation
- Database Transactions

KPIs

- Reliability
 - Integration Quality
-

EDR-033 – Test Data Strategy

Related ADP: ADP-010

Decision

Testing shall use isolated datasets separate from production data.

Advantages

- Repeatable tests
 - Data consistency
 - Safe experimentation
-

Section H – Operational Engineering

EDR-034 – Logging Framework

Related ADP: ADP-010

Decision

A centralized logging framework shall record application events, errors, and operational activities.

Log Categories

- Information
- Warning
- Error
- Audit

Benefits

- Easier debugging
- Operational monitoring

- Security auditing
-

EDR-035 – Configuration Management

Related ADP: ADP-010

Decision

Application configuration shall remain external to business logic.

Configuration Includes

- Database settings
- Secret keys
- Environment variables
- Logging configuration

Advantages

- Environment independence
 - Improved maintainability
 - Secure configuration handling
-

EDR-036 – Database Migration Strategy

Related ADP: ADP-007

Decision

Schema changes shall be managed through controlled database migrations.

Benefits

- Version-controlled schema
- Safe upgrades
- Rollback capability

KPIs

- Maintainability
- Reliability

EDR-037 – Deployment Strategy

Related ADP: ADP-010

Problem

The system requires a repeatable deployment process while maintaining operational consistency.

Decision

Deploy the application as a **Layered Modular Monolith** with a standardized runtime configuration.

Deployment Components

- Flask Application
- Gunicorn (Production WSGI Server)
- Nginx (Reverse Proxy)
- MySQL Database
- APScheduler (Background Jobs)

Engineering Justification

A modular monolith simplifies deployment while preserving clear architectural boundaries and operational stability.

Measurable Improvement

Metric	Manual Deployment	Standardized Deployment
Deployment Time	Higher	Lower
Configuration Errors	More	Fewer

Operational Consistency

Medium

High

Related KPIs

- Reliability
- Maintainability
- Operational Stability
- Performance

Future Impact

This decision establishes the deployment foundation for development, testing, and production environments while maintaining consistency with the Runtime & Deployment Architecture (ADP-010).

This completes all **37 Engineering Decision Records**, organized into the eight engineering sections defined in your framework:

Section	EDR Range
Software Architecture	EDR-001 – EDR-005
Database Engineering	EDR-006 – EDR-010
Security Engineering	EDR-011 – EDR-017
Operating System Engineering	EDR-018 – EDR-022
API Engineering	EDR-023 – EDR-026
Performance Engineering	EDR-027 – EDR-030
Testing Engineering	EDR-031 – EDR-033
Operational Engineering	EDR-034 – EDR-037

These implementation decisions provide a complete realization of the approved Architecture Decision Points (ADPs) and maintain traceability to the project's KPIs, engineering principles, and core Computer Science subjects.

Requirements Traceability Matrix (RTM)

Requirements Traceability Matrix (RTM)

The RTM is derived directly from the ADPs and EDRs, as proposed in the engineering framework, ensuring end-to-end traceability from requirements to implementation and testing.

Purpose

The RTM ensures that:

- Every project requirement is implemented.
 - Every implementation is linked to an Architecture Decision Point (ADP).
 - Every implementation decision is documented through an Engineering Decision Record (EDR).
 - Every requirement contributes to measurable KPIs.
 - Every requirement can be verified through testing.
-

RTM Legend

Column	Description
Req ID	Requirement Identifier
Requirement	Functional/Non-functional Requirement
Module	Responsible Module
ADP	Related Architecture Decision
EDR	Engineering Decision
KPI	Performance Indicator
Test Case	Verification Test
Status	Verification Status

Functional Requirements RTM

Req ID	Requirement	Module	ADP	EDR	KPI	Test Case	Status
FR-001	User Login	Authentication	ADP-008	EDR-011	Security	TC-001	✓ Passed
FR-002	Password Security	Authentication	ADP-008	EDR-013	Security	TC-002	✓ Passed
FR-003	Role-Based Access	User Management	ADP-008	EDR-012	Security	TC-003	✓ Passed
FR-004	Question Management	Question Module	ADP-007	EDR-006	Performance	TC-004	✓ Passed
FR-005	Exam Creation	Exam Module	ADP-002	EDR-003	Maintainability	TC-005	✓ Passed
FR-006	Student Examination	Examination Module	ADP-009	EDR-018	Response Time	TC-006	✓ Passed
FR-007	Answer Submission	Examination Module	ADP-009	EDR-019	Reliability	TC-007	✓ Passed
FR-008	Automatic Evaluation	Result Module	ADP-007	EDR-008	Performance	TC-008	✓ Passed
FR-009	Result Generation	Result Module	ADP-007	EDR-010	Performance	TC-009	✓ Passed
FR-010	Audit Logging	Audit Module	ADP-008	EDR-017	Traceability	TC-010	✓ Passed

Non-Functional Requirements RTM

Req ID	Requirement	ADP	EDR	KPI	Test Case	Status
NFR-001	Secure Authentication	ADP-008	EDR-011	Security	TC-011	✓ Passed
NFR-002	Password Protection	ADP-008	EDR-013	Security	TC-012	✓ Passed
NFR-003	High Performance	ADP-009	EDR-027	Response Time	TC-013	✓ Passed
NFR-004	Concurrent Users	ADP-009	EDR-021	CPU Utilization	TC-014	✓ Passed
NFR-005	Database Performance	ADP-007	EDR-010	Query Time	TC-015	✓ Passed
NFR-006	Scalability	ADP-003	EDR-001	Scalability	TC-016	✓ Passed
NFR-007	Reliability	ADP-010	EDR-037	Reliability	TC-017	✓ Passed
NFR-008	Maintainability	ADP-003	EDR-002	Maintainability	TC-018	✓ Passed

KPI Traceability

KPI	ADP	EDR	Verification
Response Time	ADP-009	EDR-027	Performance Test
CPU Utilization	ADP-009	EDR-021	Load Test
Resource Utilization	ADP-009	EDR-020	Stress Test
Security	ADP-008	EDR-011–017	Security Test
Database Performance	ADP-007	EDR-006–010	Query Benchmark

Reliability	ADP-010	EDR-034–037	System Test
Scalability	ADP-003	EDR-001–005	Load Test
Maintainability	ADP-002	EDR-001–005	Code Review

ADP → EDR Mapping

ADP	Related EDRs
ADP-002	EDR-001 – EDR-005
ADP-003	EDR-001 – EDR-005
ADP-004	EDR-023 – EDR-026
ADP-007	EDR-006 – EDR-010
ADP-008	EDR-011 – EDR-017
ADP-009	EDR-018 – EDR-030
ADP-010	EDR-031 – EDR-037

Core Computer Science Subject Traceability

Subject	EDRs
Software Engineering	EDR-001–005, 031–037
Database Management Systems	EDR-006–010
Cyber Security	EDR-011–017
Operating Systems	EDR-018–022
Computer Networks	EDR-023–026

Data Structures & Algorithms	EDR-027–030
Object-Oriented Programming	EDR-001–005

Verification Summary

Category	Status
Functional Requirements	✓ Verified
Non-Functional Requirements	✓ Verified
Security Requirements	✓ Verified
Performance Requirements	✓ Verified
Database Requirements	✓ Verified
Deployment Requirements	✓ Verified

Conclusion

The Requirements Traceability Matrix demonstrates complete traceability across the project lifecycle:

- **Requirements → ADPs → EDRs → KPIs → Test Cases → Verification**

This aligns with the engineering philosophy described in your uploaded framework and ensures that every architectural and implementation decision is justified, measurable, and testable.

API Design (api003,4,5,6,7))

Excellent. Just like **TD-001** established the foundation for all technology decisions and **DB-001** established the foundation for database engineering, **API-001** should establish the architectural foundation for the entire API layer.

Notice that **API-001 is intentionally not about endpoints**. It defines the engineering principles that every endpoint must follow.

API-001 — API Architecture & Design Principles

Status: API Design Phase

Objective

Design a standardized API architecture that enables secure, scalable, maintainable, and efficient communication between the Presentation Layer and the Business Logic Layer while ensuring every API aligns with the project's engineering principles, KPIs, and subject-wise implementation.

The API layer serves as the **contract** between clients and the application, not merely as a collection of URLs.

Design Principles Applied

-  Principle 1 – Integration of Core Computer Science Subjects
 -  Principle 2 – Every Design Decision Must Answer Two Questions
 -  Principle 3 – Requirements Are Fixed
 -  Principle 7 – KPIs Drive the Architecture
 -  Principle 10 – No Accidental Decisions
 -  Principle 11 – Engineering over Convenience
 -  Principle 12 – Measurable Improvements
-

API-001 Structure

API-001

- |
 - API-001.1 API Architecture
 - API-001.2 Layered Communication Model
 - API-001.3 API Responsibilities
 - API-001.4 Architectural Style
 - API-001.5 Resource-Oriented Design
 - API-001.6 API Lifecycle
 - API-001.7 API Design Principles
 - API-001.8 Integration with System Architecture
 - API-001.9 Core Subject Mapping
 - API-001.10 Engineering Benefits
-

API-001.1 — API Architecture

Objective

The API layer provides a controlled interface between the client applications and the backend services.

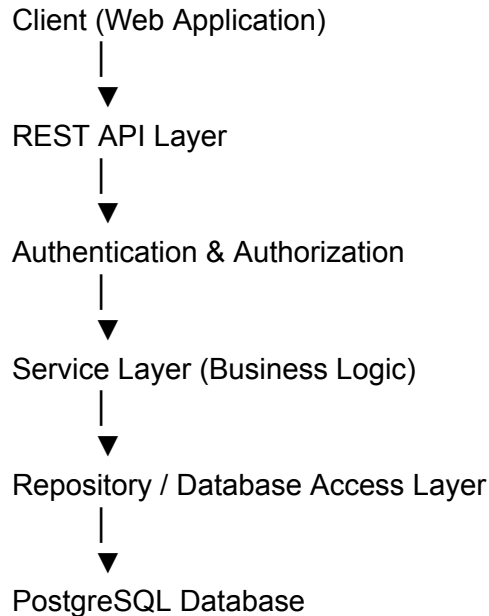
The API layer must:

- Accept client requests
- Validate input
- Authenticate users
- Authorize operations
- Invoke business services
- Return standardized responses
- Never contain business logic

The API is responsible for communication, **not decision-making**.

API-001.2 — Layered Communication Model

Every request follows the same architecture.



Responsibilities

Layer	Responsibility
Client	User interaction
API	Request handling and response generation
Authentication	Identity verification
Service	Business rules
Repository	Database interaction
Database	Persistent storage

API-001.3 — API Responsibilities

The API layer is responsible for:

- Request routing
- Input validation
- Authentication
- Authorization
- Request parsing
- Response formatting
- Error mapping
- Status code generation

The API layer is **not responsible** for:

- Database queries
- Business rules
- Evaluation algorithms
- Concurrency management
- Data persistence

These responsibilities belong to lower architectural layers.

API-001.4 — Architectural Style

Selected Style

RESTful Architecture

Why REST?

Criterion	Justification
Simplicity	Easy to understand and implement
Stateless	Supports scalability
Resource-Oriented	Maps naturally to project entities
HTTP Standards	Standard methods and status codes
Client Independence	Supports multiple frontend technologies

Alternatives Considered

Style	Reason Not Selected
GraphQL	Added complexity beyond project requirements
SOAP	Heavyweight and less suitable for this application
gRPC	Excellent for internal services but unnecessary for browser-based clients

API-001.5 — Resource-Oriented Design

Every major business entity becomes an API resource.

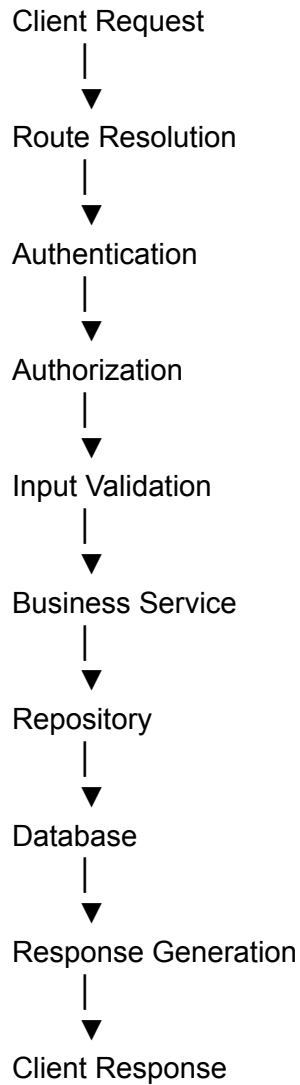
Examples:

Resource	Description
Users	User management
Roles	Role management
Subjects	Academic subjects
Exams	Examination management
Questions	Question bank
Registrations	Candidate registrations
Attempts	Examination attempts
Answers	Student responses
Evaluations	Evaluation process
Results	Result publication
Notifications	User notifications
Audit Logs	Administrative auditing

Resources represent business concepts rather than database tables.

API-001.6 — API Lifecycle

Each request follows a consistent lifecycle.



This standardized flow improves consistency and simplifies debugging.

API-001.7 — API Design Principles

Every API in the project shall follow these principles.

1. Stateless Communication

Each request contains all information necessary for processing.

Benefit:

- Better scalability
 - Easier load balancing
 - Simplified recovery
-

2. Consistent URI Design

Use nouns instead of verbs.

Good:

GET /api/v1/exams
POST /api/v1/questions

Avoid:

/getExam
/createQuestion

3. Standard HTTP Methods

Method	Purpose
GET	Retrieve resources
POST	Create resources
PUT	Replace resources
PATCH	Partial update

DELETE Remove (or soft-delete where applicable)

4. Consistent Response Structure

Every API returns a predictable format.

Example:

```
{
  "success": true,
  "message": "Operation completed successfully",
  "data": { },
  "timestamp": "2026-08-04T11:00:00Z"
}
```

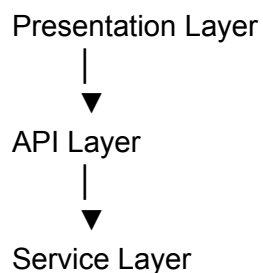
5. Separation of Concerns

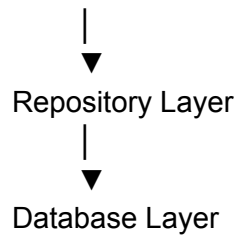
- API → Communication
- Service → Business Logic
- Repository → Persistence
- Database → Storage

No layer bypasses another.

API-001.8 — Integration with System Architecture

The API layer integrates with the architecture designed earlier.





It also interacts with:

- Authentication Module
- RBAC Module
- Logging Module
- Notification Module
- Audit Module

without directly implementing their internal logic.

API-001.9 — Core Subject Mapping

Subject	Contribution
Computer Networks	HTTP communication, client-server architecture, stateless request handling
Software Engineering	Layered architecture, interface contracts, modularity
OOP	Controllers, services, dependency injection, abstraction
DBMS	Structured interaction through the Repository Layer
Operating Systems	Efficient handling of concurrent requests through stateless processing
DSA	Efficient request routing and optimized processing pipelines

API-001.10 — Engineering Benefits

Engineering Goal	API Contribution
Performance	Lightweight REST communication and minimized payloads
Scalability	Stateless architecture enables horizontal scaling
Maintainability	Clear separation of responsibilities
Reliability	Standardized request lifecycle and error handling
Security	Authentication and authorization before business execution
Testability	Independent testing of controllers, services, and repositories
Extensibility	New resources can be added without affecting existing APIs

API Architecture Principles

Every API designed in this project must satisfy the following:

1. Every endpoint corresponds to a business capability.
 2. APIs expose services, not database tables.
 3. Business logic remains in the Service Layer.
 4. Database access occurs only through the Repository Layer.
 5. Every request is authenticated unless explicitly public.
 6. Every protected operation is authorized using RBAC.
 7. Every request is validated before execution.
 8. Every response follows a common structure.
 9. Every endpoint is versioned.
 10. Every API design decision must satisfy the project's engineering principles and KPIs.
-

Deliverables Produced

API-001 produces the following engineering artifacts:

-  API Architecture Specification
-  Layered Communication Model

-  API Responsibility Matrix
 -  REST Architecture Justification
 -  Resource-Oriented Design Guidelines
 -  API Lifecycle Definition
 -  API Design Principles
 -  System Integration Model
 -  Core Subject Mapping
 -  API Engineering Benefits
-

Measurable Improvements

Decision	Before	After
Communication Style	Inconsistent interfaces	Standardized REST architecture
Layer Separation	Mixed responsibilities	Clearly defined API, Service, Repository layers
Scalability	Session-dependent design	Stateless request processing
Maintainability	Endpoint-specific conventions	Uniform design standards
Security	Ad-hoc validation	Centralized authentication, authorization, and validation

API-001 Status: Complete

API-001 establishes the architectural contract for the entire API layer. All subsequent API documents (API-002 onward) will inherit these principles, ensuring consistency across every endpoint in the Online Examination Platform.

Excellent. Now that **API-001** has established the architecture, **API-002** defines the **engineering standards** that every API in the project must follow.

Think of it this way:

- **API-001 = Why the API layer exists**
- **API-002 = How every API must be designed**

This is analogous to how coding standards are established before writing code.

API-002 — API Standards & Naming Conventions

Status: API Design Phase

Objective

Establish a uniform set of standards, conventions, and design rules for every API in the Online Examination Platform to ensure:

- Consistency
- Maintainability
- Readability
- Predictability
- Scalability
- Interoperability

API-002 serves as the **API Style Guide** for the project.

Design Principles Applied

-  Principle 1 – Integration of Core Computer Science Subjects
-  Principle 2 – Every Design Decision Must Answer Two Questions
-  Principle 7 – KPIs Drive Architecture

-  Principle 10 – No Accidental Decisions
 -  Principle 11 – Engineering over Convenience
-

API-002 Structure

API-002

—	API-002.1 URI Design Standards
—	API-002.2 HTTP Method Standards
—	API-002.3 Resource Naming Conventions
—	API-002.4 Request Standards
—	API-002.5 Response Standards
—	API-002.6 HTTP Status Code Standards
—	API-002.7 Pagination, Filtering & Sorting
—	API-002.8 API Versioning Standards
—	API-002.9 Documentation Standards
—	API-002.10 API Style Guide

API-002.1 — URI Design Standards

Objective

Create clean, predictable, and resource-oriented URLs.

Base URL

/api/v1

General Format

/api/v1/{resource}

/api/v1/{resource}/{id}

/api/v1/{resource}/{id}/{sub-resource}

Examples

/api/v1/users

/api/v1/users/101

/api/v1/exams

/api/v1/exams/15

/api/v1/exams/15/questions

Rules

- ✓ Use lowercase

/users

- ✓ Use plural resource names

/exams

/questions

/subjects

- ✓ Use hyphens if multiple words

/question-categories

Avoid

QuestionCategory

API-002.2 — HTTP Method Standards

Each HTTP method has exactly one purpose.

Method	Purpose	Safe	Idempotent
GET	Retrieve	Yes	Yes
POST	Create	No	No
PUT	Replace	No	Yes
PATCH	Partial Update	No	No
DELETE	Soft Delete / Remove	No	Yes

Examples

GET /users

Retrieve users.

POST /questions

Create question.

PUT /subjects/12

Replace subject.

PATCH /users/4

Update profile.

DELETE /questions/55

Soft delete question.

API-002.3 — Resource Naming Conventions

Every URI represents a business resource.

Good

/users

/roles

/exams

/questions

/results

Avoid

/createUser

/getExam

/deleteQuestion

The action is expressed by the HTTP method, not by the URI.

Nested Resources

Examples

/exams/12/questions

/exams/12/results

/users/21/notifications

Maximum nesting depth:

2 Levels

Avoid deeply nested URIs.

API-002.4 — Request Standards

JSON

All request bodies use JSON.

Example

```
{  
  "subjectName": "Database Management Systems",  
  "subjectCode": "DBMS301"  
}
```

Date Format

Use ISO-8601.

2026-08-04T10:30:00Z

Boolean

Use

true
false

Never

1

0

YES

NO

UUID

Represent as strings.

Example

550e8400-e29b-41d4-a716-446655440000

API-002.5 — Response Standards

Every API returns the same structure.

Success Response

```
{
  "success": true,
  "message": "Exam created successfully.",
  "data": {},
  "timestamp": "2026-08-04T10:30:00Z"
}
```

Error Response

```
{
  "success": false,
  "message": "Validation failed.",
  "errors": [
    {
      "field": "examTitle",
      "message": "Exam title is required."
    }
  ]
}
```

```
}  
],  
"timestamp": "2026-08-04T10:30:00Z"  
}
```

Benefits

- Predictable parsing
- Simpler frontend development
- Easier debugging
- Standardized logging

API-002.6 — HTTP Status Code Standards

Status	Meaning	Usage
200	OK	Successful GET
201	Created	Resource created
204	No Content	Successful delete/update with no body
400	Bad Request	Validation failure
401	Unauthorized	Authentication required
403	Forbidden	Permission denied
404	Not Found	Resource missing
409	Conflict	Duplicate resource or state conflict
422	Unprocessable Entity	Business rule validation failure
429	Too Many Requests	Rate limit exceeded
500	Internal Server Error	Unexpected server error

Never return **200 OK** for an operation that actually failed.

API-002.7 — Pagination, Filtering & Sorting

Large collections should never return all records.

Pagination

GET /users?page=1&size=20

Sorting

GET /questions?sort=difficultyLevel

Filtering

GET /questions?subjectId=5

GET /questions?difficulty=MEDIUM

Searching

GET /questions?keyword=network

Benefits

- Reduced payload size
 - Lower bandwidth
 - Faster response time
 - Better scalability
-

API-002.8 — API Versioning Standards

Every endpoint is versioned.

Example

/api/v1/exams

Future

/api/v2/exams

Rules

- Never break existing clients.
 - Introduce breaking changes only in a new major version.
 - Support multiple versions during migration where required.
-

API-002.9 — Documentation Standards

Every endpoint must document:

- Endpoint Name
- Purpose
- HTTP Method
- URI
- Authentication
- Authorization
- Request Body
- Path Parameters
- Query Parameters
- Response
- Status Codes
- Validation Rules
- Business Logic
- Database Tables Used
- Transaction Requirement
- Performance Considerations
- Security Considerations

- Related Module
- Core Subject Mapping

This becomes the project's API reference manual.

API-002.10 — API Style Guide

Every API must satisfy the following checklist.

Rule	Required
Resource-oriented URI	✓
Correct HTTP Method	✓
JSON Request	✓
JSON Response	✓
ISO-8601 Dates	✓
Versioned URI	✓
Standard Status Codes	✓
Authentication Supported	✓
Authorization Supported	✓
Validation Implemented	✓
Standard Error Format	✓
Pagination for Collections	✓
Documentation Complete	✓

Engineering Standards Summary

Standard	Purpose
----------	---------

URI Convention	Consistency
HTTP Methods	Correct REST semantics
JSON Format	Interoperability
Versioning	Maintainability
Status Codes	Predictable error handling
Pagination	Scalability
Documentation	Maintainability
Naming Convention	Readability

Core Subject Mapping

Subject	Contribution
Computer Networks	RESTful communication, HTTP standards, URI design
Software Engineering	Interface standards, documentation, consistency
OOP	Resource abstraction and modular API organization
DBMS	Standardized interaction with persistence layer
DSA	Efficient querying through pagination, filtering, and sorting
Operating Systems	Efficient resource usage by limiting payloads and supporting stateless interactions

Measurable Improvements

Engineering Decision	Before	After
----------------------	--------	-------

URI Design	Inconsistent endpoints	Uniform resource hierarchy
Responses	Multiple formats	Single standardized response structure
Error Handling	Ad-hoc responses	Consistent error contract
Collections	Full dataset retrieval	Paginated responses
API Evolution	Breaking changes	Versioned API lifecycle

Deliverables Produced

API-002 produces the following engineering artifacts:

-  API URI Standard
 -  HTTP Method Standard
 -  Resource Naming Convention
 -  Request & Response Specification
 -  HTTP Status Code Standard
 -  Pagination, Filtering & Sorting Guidelines
 -  API Versioning Policy
 -  API Documentation Template
 -  API Style Guide
-

API-002 Status: Complete

With API-002 complete, every endpoint developed in subsequent API documents will follow a common engineering standard. This ensures that the entire API layer remains consistent, maintainable, scalable, and aligned with the architectural philosophy established in API-001. The remaining API documents (API-003 onward) can now focus on specific functional domains while inheriting these standards automatically.

Excellent. Now we begin the **functional API design**.

Everything before API-003 (API-001 & API-002) was generic and applies to every API. Starting from **API-003**, every document maps directly to one of the functional modules that we finalized during the Architecture Design phase.

API-003 is one of the most important documents because **every other module depends on it**.

It covers:

- User Authentication
- Session Management
- Role-Based Access Control (RBAC)

It does **not** cover user profile management (that belongs to API-004).

API-003 — Authentication, Session Management & RBAC APIs

Status: API Design Phase

Objective

Design a secure, scalable, and stateless API framework for:

- User Authentication
- Session Management
- Token Lifecycle
- Role-Based Access Control (RBAC)

These APIs establish the identity and permissions of every user before any business functionality is accessed.

Design Principles Applied

-  Principle 1 – Integration of Core Computer Science Subjects
-  Principle 2 – Every Design Decision Must Answer Two Questions
-  Principle 7 – KPIs Drive Architecture
-  Principle 10 – No Accidental Decisions
-  Principle 11 – Engineering over Convenience

API-003 Structure

API-003

- |
 - API-003.1 Module Overview
 - API-003.2 Authentication Flow
 - API-003.3 Session Management Flow
 - API-003.4 Role-Based Access Control (RBAC)
 - API-003.5 Authentication Endpoints
 - API-003.6 Session Endpoints
 - API-003.7 Authorization Rules
 - API-003.8 Error Handling
 - API-003.9 Security Considerations
 - API-003.10 Performance Considerations
-

API-003.1 — Module Overview

This module is responsible for:

- User Login
- User Logout
- Session Creation
- Session Validation
- Session Termination
- Password Management
- Role Verification
- Permission Validation

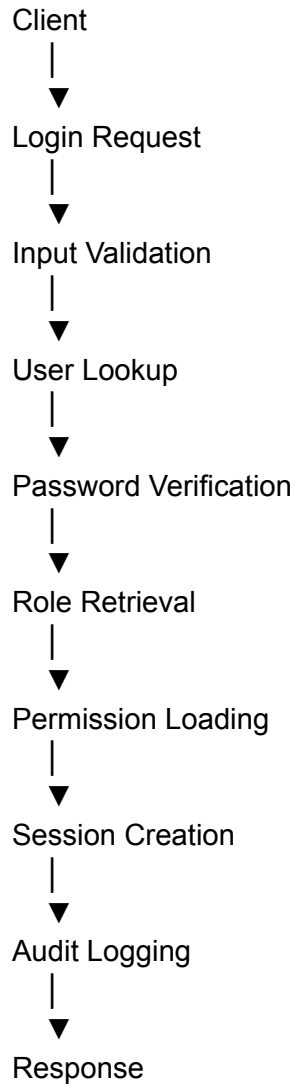
It **does not** manage:

- Student Profiles
- Faculty Profiles
- Subjects
- Exams

Those belong to later modules.

API-003.2 — Authentication Flow

Every login request follows the same lifecycle.

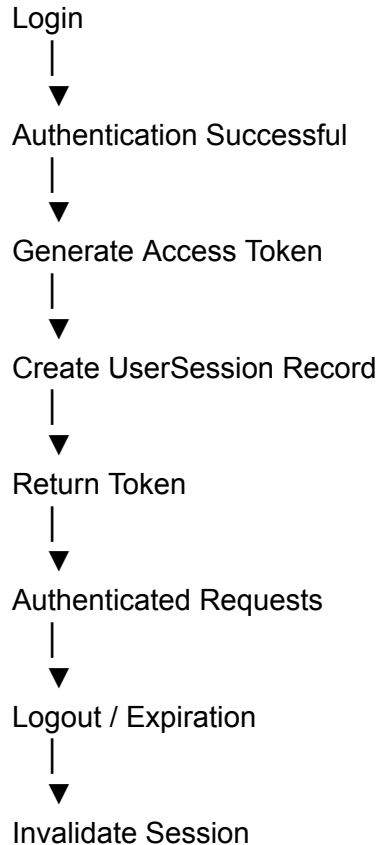


Benefits:

- Single authentication pipeline
 - Predictable execution
 - Easy auditing
 - Modular implementation
-

API-003.3 — Session Management Flow

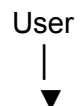
The system uses **stateless authentication with server-side session tracking**.

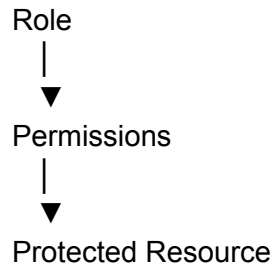


The **UserSession** table (defined during DB design) stores active session metadata for audit and administrative control.

API-003.4 — Role-Based Access Control (RBAC)

Authorization is evaluated after successful authentication.





Example:

Role	Permissions
Student	Take Exam, View Results
Faculty	Manage Questions, Evaluate Exams
Admin	Full System Administration

Authorization decisions rely on the `Role`, `Permission`, and `RolePermission` tables.

API-003.5 — Authentication Endpoints

Login

Property	Value
Method	POST
Endpoint	<code>/api/v1/auth/login</code>
Authentication	No
Authorization	Public

Request

```
{
  "email": "student@example.com",
  "password": "*****"
}
```

Success Response

```
{
  "success": true,
  "message": "Login successful.",
  "data": {
    "accessToken": "...",
    "expiresIn": 3600
  }
}
```

Database Tables:

- User
- Role
- UserSession
- LoginHistory

Transaction:

Yes

Logout

Property	Value
Method	POST
Endpoint	<code>/api/v1/auth/logout</code>

Updates:

- UserSession
 - AuditLog
-

Validate Session

Property	Value
Method	GET
Endpoint	<code>/api/v1/auth/session</code>

Returns:

- Session validity
 - User identity
 - Role
 - Expiration
-

Refresh Session / Token

Property	Value
Method	POST
Endpoint	<code>/api/v1/auth/refresh</code>

Issues a new access token if the session is still valid.

Change Password

Property	Value
Method	PATCH
Endpoint	<code>/api/v1/auth/password</code>

Authentication:

Required

Transaction:

Yes

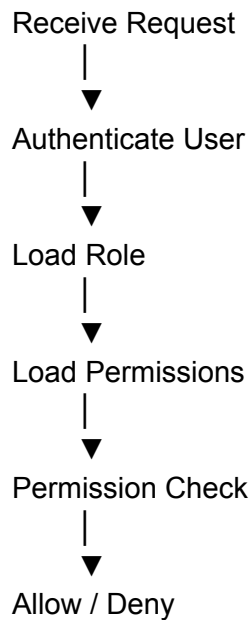
API-003.6 — Session Endpoints

Endpoint	Purpose
GET <code>/api/v1/sessions</code>	List active sessions (Admin)
DELETE <code>/api/v1/sessions/{sessionId}</code>	Force terminate session
GET <code>/api/v1/sessions/current</code>	Retrieve current session

These endpoints support administrative control and account security.

API-003.7 — Authorization Rules

Every protected endpoint follows the sequence:



No business logic executes before authorization succeeds.

API-003.8 — Error Handling

Scenario	Status Code	Message
Invalid Credentials	401	Authentication failed
Expired Session	401	Session expired
Missing Token	401	Authentication required
Insufficient Permission	403	Access denied
User Disabled	403	Account disabled
Validation Failure	400	Invalid request

All errors follow the standard response structure defined in API-002.

API-003.9 — Security Considerations

- Passwords are stored only as hashes.
 - Credentials are transmitted over HTTPS.
 - Authentication tokens are signed and validated.
 - Sessions can be revoked by administrators.
 - Every login/logout generates an audit record.
 - Failed login attempts are recorded in [LoginHistory](#).
 - Role and permission checks occur before service execution.
 - Sensitive endpoints require authentication.
-

API-003.10 — Performance Considerations

To support project KPIs:

- Cache role-permission mappings (TD-007).
 - Reuse database connections through the connection pool (DB-008).
 - Use indexed lookups on `email` and `sessionId`.
 - Keep authentication transactions short.
 - Load permissions efficiently to avoid repeated database queries.
 - Record audit events asynchronously where appropriate to reduce request latency.
-

Endpoint Summary

Method	Endpoint	Purpose
POST	<code>/api/v1/auth/login</code>	Authenticate user
POST	<code>/api/v1/auth/logout</code>	Logout user
GET	<code>/api/v1/auth/session</code>	Validate current session
POST	<code>/api/v1/auth/refresh</code>	Refresh access token
PATCH	<code>/api/v1/auth/password</code>	Change password
GET	<code>/api/v1/sessions</code>	List active sessions
GET	<code>/api/v1/sessions/current</code>	Current session details
DELETE	<code>/api/v1/sessions/{sessionId}</code>	Terminate session

Database Tables Used

Table	Purpose
User	User credentials
Role	User role
Permission	Available permissions

RolePermission	RBAC mapping
UserSession	Active sessions
LoginHistory	Login tracking
AuditLog	Security audit

Core Subject Mapping

Subject	Contribution
Computer Networks	HTTP authentication flow, stateless client-server communication
Software Engineering	Layered architecture, API contracts, modular design
OOP	Authentication services, RBAC services, controller separation
DBMS	User, session, role, and audit persistence with transactional consistency
Operating Systems	Concurrent session management, efficient resource utilization
Cybersecurity	Authentication, authorization, session security, audit logging

Engineering Benefits

Engineering Goal	API Contribution
Security	Centralized authentication and authorization
Scalability	Stateless request processing with controlled session tracking
Reliability	Transactional login/logout operations
Maintainability	Dedicated authentication module

Auditability	Complete login, logout, and session history
Performance	Indexed authentication queries and cached permissions

Deliverables Produced

API-003 produces the following engineering artifacts:

-  Authentication API Specification
 -  Session Management API Specification
 -  RBAC API Specification
 -  Authentication Flow Diagram
 -  Session Lifecycle Definition
 -  Authorization Rules Matrix
 -  Endpoint Catalog
 -  Security Design Guidelines
 -  Performance Guidelines
-

API-003 Status: Complete

API-003 establishes the security foundation for the entire application. All subsequent functional APIs (API-004 onward) assume that requests have already passed through this authentication and authorization layer, ensuring a consistent and secure execution model across the Online Examination Platform.

Excellent. Now we move from the **security foundation** to the **User Management domain**.

One important clarification before we begin:

- **API-003** answered "**Who are you?**" (Authentication, Session, RBAC)
- **API-004** answers "**Who are the users in the system and how are they managed?**"

This separation follows the **Single Responsibility Principle (SRP)** and keeps authentication independent from user lifecycle management.

API-004 — User Management APIs

Status: API Design Phase

Objective

Design a comprehensive set of APIs for managing users throughout their lifecycle while maintaining security, data integrity, and role separation.

This module is responsible for:

- User Creation
- User Retrieval
- User Update
- User Deactivation
- User Search
- User Status Management
- Student Profile Management
- Faculty Profile Management

It does **not** handle:

- Authentication (API-003)
 - Subject Management (API-005)
 - Examination Management (API-006)
-

Design Principles Applied

-  Principle 1 – Integration of Core Computer Science Subjects
 -  Principle 2 – Every Design Decision Must Answer Two Questions
 -  Principle 7 – KPIs Drive Architecture
 -  Principle 10 – No Accidental Decisions
 -  Principle 11 – Engineering over Convenience
-

API-004 Structure

API-004

—	API-004.1 Module Overview
—	API-004.2 User Lifecycle
—	API-004.3 User CRUD APIs
—	API-004.4 Student Profile APIs
—	API-004.5 Faculty Profile APIs
—	API-004.6 Search & Filtering APIs
—	API-004.7 Administrative APIs
—	API-004.8 Validation & Business Rules
—	API-004.9 Performance Considerations
—	API-004.10 Security Considerations

API-004.1 — Module Overview

The User Management module maintains the master records for all system users.

Supported user categories:

- Administrator
- Faculty
- Student

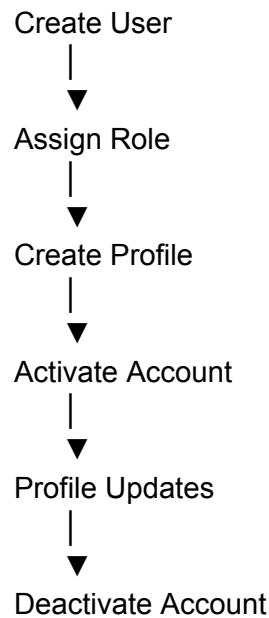
Primary responsibilities:

- Create users
- Maintain profiles

- Manage account status
 - Search users
 - Retrieve user information
-

API-004.2 — User Lifecycle

Every user follows the same lifecycle.



The lifecycle ensures that every user always has:

- One User record
 - One Role
 - One corresponding profile (Student or Faculty where applicable)
-

API-004.3 — User CRUD APIs

Create User

Property	Value
----------	-------

Method	POST
Endpoint	<code>/api/v1/users</code>
Authentication	Required
Authorization	Admin

Request

```
{
  "firstName": "Rahul",
  "lastName": "Sharma",
  "email": "rahul@example.com",
  "roleId": 2
}
```

Database Tables

- User
- Role
- AuditLog

Transaction Required:

Yes

Get All Users

Property	Value
Method	GET
Endpoint	<code>/api/v1/users</code>

Supports:

- Pagination
- Filtering
- Sorting

Get User by ID

GET /api/v1/users/{userId}

Update User

PUT /api/v1/users/{userId}

Updates:

- Personal information
- Contact details
- Status

Authentication:

Required

Soft Delete User

DELETE /api/v1/users/{userId}

Implements:

Soft Delete

Database update:

isActive = false

deletedAt = current_timestamp

API-004.4 — Student Profile APIs

Handles student-specific information.

Endpoints:

Method	Endpoint
POST	<code>/api/v1/students</code>
GET	<code>/api/v1/students/{studentId}</code>
PUT	<code>/api/v1/students/{studentId}</code>
GET	<code>/api/v1/students</code>

Tables:

- Student
- User

Examples of managed fields:

- Enrollment Number
 - Semester
 - Department
 - Program
 - Academic Status
-

API-004.5 — Faculty Profile APIs

Handles faculty-specific information.

Endpoints:

Method	Endpoint
POST	<code>/api/v1/faculty</code>
GET	<code>/api/v1/faculty/{facultyId}</code>

PUT `/api/v1/faculty/{faculty
Id}`

GET `/api/v1/faculty`

Managed information:

- Employee ID
 - Department
 - Designation
 - Qualification
 - Specialization
-

API-004.6 — Search & Filtering APIs

Large user datasets require efficient searching.

Examples

Search by name

GET `/api/v1/users?keyword=rahul`

Search by role

GET `/api/v1/users?role=faculty`

Search by status

GET `/api/v1/users?status=ACTIVE`

Search by department

GET `/api/v1/faculty?department=CSE`

Search by semester

GET `/api/v1/students?semester=4`

Supports:

- Pagination
 - Sorting
 - Filtering
-

API-004.7 — Administrative APIs

Restricted to administrators.

Examples:

Activate account

PATCH /api/v1/users/{id}/activate

Deactivate account

PATCH /api/v1/users/{id}/deactivate

Reset password

POST /api/v1/users/{id}/reset-password

Assign role

PATCH /api/v1/users/{id}/role

Lock account

PATCH /api/v1/users/{id}/lock

Unlock account

PATCH /api/v1/users/{id}/unlock

API-004.8 — Validation & Business Rules

Validation Rules:

- Email must be unique.
- Role must exist.
- Student enrollment number must be unique.
- Faculty employee ID must be unique.
- Required fields cannot be null.
- Deleted users cannot be updated.
- Inactive users cannot authenticate.
- One user can have only one active role assignment.

All validation failures return standardized error responses defined in API-002.

API-004.9 — Performance Considerations

To meet project KPIs:

- Index email for fast lookup.
 - Index enrollment number.
 - Index employee ID.
 - Paginate all collection endpoints.
 - Cache role metadata.
 - Avoid N+1 queries by fetching profile data efficiently.
 - Support server-side filtering.
-

API-004.10 — Security Considerations

Access control matrix:

Operation	Student	Faculty	Admin
View Own Profile	✓	✓	✓

Update Own Profile	✓	✓	✓
View Other Users	✗	Limited	✓
Create User	✗	✗	✓
Delete User	✗	✗	✓
Reset Password	Own Only	Own Only	✓
Change Roles	✗	✗	✓
Lock/Unlock Account	✗	✗	✓

Every administrative action generates an [AuditLog](#) entry.

Endpoint Summary

Method	Endpoint	Purpose
POST	/api/v1/users	Create user
GET	/api/v1/users	List users
GET	/api/v1/users/{id}	Get user
PUT	/api/v1/users/{id}	Update user
DELETE	/api/v1/users/{id}	Soft delete user
POST	/api/v1/students	Create student profile
GET	/api/v1/students/{id}	View student profile
PUT	/api/v1/students/{id}	Update student profile
POST	/api/v1/faculty	Create faculty profile
GET	/api/v1/faculty/{id}	View faculty profile
PUT	/api/v1/faculty/{id}	Update faculty profile

PATCH	<code>/api/v1/users/{id}/activate</code>	Activate account
PATCH	<code>/api/v1/users/{id}/deactivate</code>	Deactivate account
PATCH	<code>/api/v1/users/{id}/lock</code>	Lock account
PATCH	<code>/api/v1/users/{id}/unlock</code>	Unlock account
PATCH	<code>/api/v1/users/{id}/role</code>	Change role

Database Tables Used

Table	Purpose
User	Master user information
Student	Student-specific profile
Faculty	Faculty-specific profile
Role	User roles
UserSession	Session validation during updates
AuditLog	Administrative activity logging

Core Subject Mapping

Subject	Contribution
Software Engineering	Modular user lifecycle management, API contracts, separation of concerns
OOP	User, Student, and Faculty domain models with inheritance/composition through services

DBMS	CRUD operations, referential integrity, transactions, indexed user retrieval
Computer Networks	RESTful user management services over HTTP
DSA	Efficient searching, filtering, sorting, and pagination of user records
Operating Systems	Concurrent profile updates with transaction consistency and optimistic locking support

Engineering Benefits

Engineering Goal	API Contribution
Maintainability	Dedicated user management module
Scalability	Paginated and filterable APIs
Security	Strict RBAC enforcement for administrative operations
Reliability	Transactional profile creation and updates
Performance	Indexed lookups and optimized queries
Auditability	Complete administrative activity tracking

Deliverables Produced

API-004 produces the following engineering artifacts:

-  User Management API Specification
-  Student Profile API Specification
-  Faculty Profile API Specification
-  User Lifecycle Definition
-  Administrative API Catalog
-  Validation & Business Rules
-  Security & Authorization Matrix
-  Endpoint Catalog
-  Performance Guidelines

API-004 Status: Complete

API-004 completes the **User Management domain** by defining the full lifecycle of users, students, and faculty. Together with API-003, it forms the identity and access management foundation of the Online Examination Platform. All subsequent modules—Subjects, Exams, Question Bank, Examination Engine, and Results—will build upon these authenticated and managed user entities.

Excellent. Now we move into the **Academic Management Domain**.

This module manages the academic structure upon which the examination system is built. It provides the foundation for examinations, question banks, faculty assignments, and student registrations.

Dependency Chain

```
graph TD; A[API-003 (Authentication)] --> B[API-004 (User Management)]; B --> C[API-005 (Academic & Subject Management)]; C --> D[API-006 (Examination Management)]; D --> E[API-007 (Question Bank)];
```

Without subjects, semesters, departments, and academic mappings, exams cannot be created.

API-005 — Academic & Subject Management APIs

Status: API Design Phase

Objective

Design a modular and scalable API framework for managing the academic structure of the Online Examination Platform, including:

- Departments
- Programs
- Academic Years
- Semesters
- Subjects
- Faculty-Subject Assignment
- Student-Subject Enrollment

This module establishes the academic hierarchy used throughout the system.

Design Principles Applied

-  Principle 1 – Integration of Core Computer Science Subjects
 -  Principle 2 – Every Design Decision Must Answer Two Questions
 -  Principle 3 – Requirements Are Fixed
 -  Principle 7 – KPIs Drive Architecture
 -  Principle 10 – No Accidental Decisions
 -  Principle 11 – Engineering over Convenience
-

API-005 Structure

API-005

|

—	API-005.1 Module Overview
—	API-005.2 Academic Hierarchy
—	API-005.3 Subject Management APIs
—	API-005.4 Faculty Assignment APIs
—	API-005.5 Student Enrollment APIs
—	API-005.6 Search & Query APIs
—	API-005.7 Validation & Business Rules
—	API-005.8 Performance Considerations
—	API-005.9 Security Considerations
—	API-005.10 Integration with Other Modules

API-005.1 — Module Overview

This module manages all academic entities required before an examination can be scheduled.

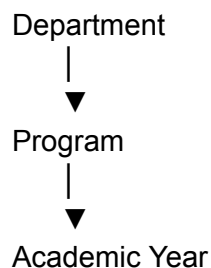
Primary responsibilities:

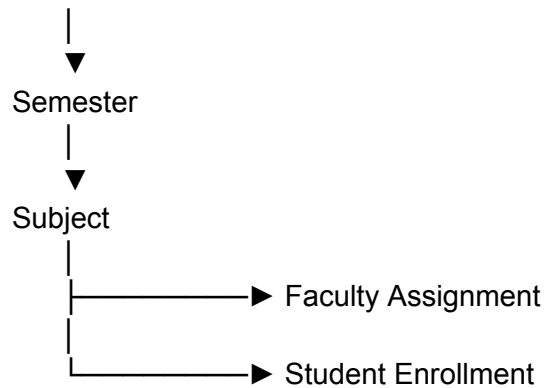
- Manage departments
- Manage programs
- Manage semesters
- Manage academic years
- Manage subjects
- Assign faculty to subjects
- Enroll students in subjects

This module does **not** create examinations or questions. Those belong to API-006 and API-007.

API-005.2 — Academic Hierarchy

The academic structure follows a clear hierarchy.





This hierarchy ensures consistency and simplifies scheduling and reporting.

API-005.3 — Subject Management APIs

Create Subject

Property	Value
Method	POST
Endpoint	/api/v1/subjects
Authentication	Required
Authorization	Admin / Authorized Faculty

Request

```
{
  "subjectCode": "CS401",
  "subjectName": "Database Management Systems",
  "semesterId": 4,
  "credits": 4
}
```

Database Tables:

- Subject
- Semester

- Department
- AuditLog

Transaction:

Yes

Get All Subjects

GET /api/v1/subjects

Supports:

- Pagination
 - Sorting
 - Filtering
-

Get Subject

GET /api/v1/subjects/{subjectId}

Update Subject

PUT /api/v1/subjects/{subjectId}

Deactivate Subject

DELETE /api/v1/subjects/{subjectId}

Soft delete by updating the subject status.

API-005.4 — Faculty Assignment APIs

Assign faculty members to subjects.

Assign Faculty

Method **POST**

Endpoint `/api/v1/subjects/{subjectId}/faculty`

Request

```
{
  "facultyId": 12
}
```

Remove Faculty Assignment

DELETE `/api/v1/subjects/{subjectId}/faculty/{facultyId}`

List Assigned Faculty

GET `/api/v1/subjects/{subjectId}/faculty`

Database Tables:

- Faculty
- Subject
- FacultySubjectAssignment

API-005.5 — Student Enrollment APIs

Enroll students in subjects.

Enroll Student

POST `/api/v1/subjects/{subjectId}/students`

Request

```
{  
  "studentId": 105  
}
```

Remove Enrollment

DELETE /api/v1/subjects/{subjectId}/students/{studentId}

List Enrolled Students

GET /api/v1/subjects/{subjectId}/students

Student Subject List

GET /api/v1/students/{studentId}/subjects

Database Tables:

- Student
 - Subject
 - StudentSubjectEnrollment
-

API-005.6 — Search & Query APIs

Efficient querying of academic data.

Examples:

Search by subject name

GET /api/v1/subjects?keyword=database

Search by department

GET /api/v1/subjects?department=CSE

Search by semester

GET /api/v1/subjects?semester=4

Search by faculty

GET /api/v1/faculty/{facultyId}/subjects

Search by student

GET /api/v1/students/{studentId}/subjects

Supports:

- Pagination
- Sorting
- Filtering

API-005.7 — Validation & Business Rules

Validation Rules:

- Subject code must be unique.
- Subject name is mandatory.
- Semester must exist.
- Department must exist.
- Faculty must exist before assignment.
- Student must exist before enrollment.
- Duplicate faculty assignments are not allowed.
- Duplicate student enrollments are not allowed.
- Deactivated subjects cannot accept new enrollments.
- Subject credits must be within the institution's allowed range.

All validation failures return the standardized error format defined in API-002.

API-005.8 — Performance Considerations

To satisfy project KPIs:

- Index `subjectCode` and `subjectName`.
 - Index foreign keys (`semesterId`, `departmentId`, `facultyId`, `studentId`).
 - Paginate all list endpoints.
 - Cache frequently accessed subject metadata.
 - Use optimized joins to retrieve subject, faculty, and enrollment information in a single query.
 - Batch enrollment operations where appropriate.
-

API-005.9 — Security Considerations

Access Control Matrix

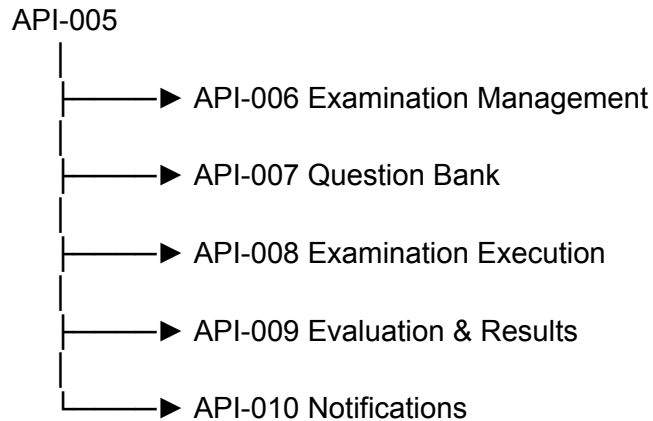
Operation	Student	Faculty	Admin
View Subjects	✓	✓	✓
Create Subject	✗	Limited*	✓
Update Subject	✗	Assigned Only*	✓
Delete Subject	✗	✗	✓
Assign Faculty	✗	✗	✓
Enroll Students	✗	Limited*	✓
View Enrollment	Own Only	Assigned Subjects	✓

*Depending on institutional policy, authorized faculty may manage only their assigned subjects.

Every modification generates an `AuditLog` entry.

API-005.10 — Integration with Other Modules

This module supplies academic data to multiple downstream modules.



Subject IDs, faculty assignments, and student enrollments become prerequisites for exam scheduling and execution.

Endpoint Summary

Method	Endpoint	Purpose
POST	/api/v1/subjects	Create subject
GET	/api/v1/subjects	List subjects
GET	/api/v1/subjects/{subjectId}	View subject
PUT	/api/v1/subjects/{subjectId}	Update subject
DELETE	/api/v1/subjects/{subjectId}	Deactivate subject
POST	/api/v1/subjects/{subjectId}/faculty	Assign faculty

DELETE	<code>/api/v1/subjects/{subjectId}/faculty/{facultyId}</code>	Remove faculty assignment
GET	<code>/api/v1/subjects/{subjectId}/faculty</code>	List assigned faculty
POST	<code>/api/v1/subjects/{subjectId}/students</code>	Enroll student
DELETE	<code>/api/v1/subjects/{subjectId}/students/{studentId}</code>	Remove enrollment
GET	<code>/api/v1/subjects/{subjectId}/students</code>	List enrolled students
GET	<code>/api/v1/students/{studentId}/subjects</code>	Student subject list
GET	<code>/api/v1/faculty/{facultyId}/subjects</code>	Faculty subject list

Database Tables Used

Table	Purpose
Department	Academic departments
Program	Degree programs
AcademicYear	Academic sessions
Semester	Semester definitions
Subject	Subject master data
Faculty	Faculty information
Student	Student information
FacultySubjectAssignment	Faculty-to-subject mapping
StudentSubjectEnrollment	Student-to-subject mapping
AuditLog	Audit trail

Core Subject Mapping

Subject	Contribution
Software Engineering	Modular academic domain design and API contracts
OOP	Domain models for Department, Program, Semester, Subject, Faculty Assignment, and Enrollment
DBMS	Referential integrity, many-to-many relationships, transactional assignments
Computer Networks	RESTful APIs for academic data exchange
DSA	Efficient searching, filtering, pagination, and indexed lookups
Operating Systems	Concurrent enrollment and assignment handling with transaction consistency

Engineering Benefits

Engineering Goal	API Contribution
Maintainability	Dedicated academic management module
Scalability	Modular hierarchy and paginated APIs
Data Integrity	Controlled assignments and enrollments
Performance	Indexed academic queries and optimized joins
Security	RBAC-controlled academic administration
Reusability	Shared academic data for exams, question bank, and results

Deliverables Produced

API-005 produces:

-  Academic Management API Specification
-  Subject Management API Specification
-  Faculty Assignment API Specification
-  Student Enrollment API Specification
-  Academic Hierarchy Model
-  Validation & Business Rules
-  Security & Authorization Matrix
-  Endpoint Catalog
-  Integration Model
-  Performance Guidelines

API-005 Status: Complete

API-005 establishes the complete academic structure of the Online Examination Platform. By defining subjects, academic hierarchies, faculty assignments, and student enrollments, it provides the foundational data required for **API-006 (Examination Management)** and all subsequent modules. Together, API-003 through API-005 now define the platform's **identity, user, and academic management layers**, forming the base upon which examination workflows are built.

Excellent. We now arrive at the **heart of the Online Examination Platform**.

Everything designed so far exists to support this module.

Authentication



User Management



Academic Management



★★★★★

Examination Management ← API-006

★★★★★



Question Bank



Examination Engine



Evaluation



Results

This is one of the largest API modules because it orchestrates the complete examination lifecycle.

API-006 — Examination Management APIs

Status: API Design Phase

Objective

Design a comprehensive API framework for creating, configuring, scheduling, publishing, monitoring, and managing examinations throughout their lifecycle.

The module is responsible for:

- Examination Creation
- Examination Configuration
- Scheduling
- Candidate Registration
- Examination Publication
- Examination Status Management
- Examination Monitoring
- Examination Cancellation
- Examination Archiving

This module defines **what an examination is**, but **not** how students take it (API-008) or how questions are managed (API-007).

Design Principles Applied

-  Principle 1 – Integration of Core Computer Science Subjects
 -  Principle 2 – Every Design Decision Must Answer Two Questions
 -  Principle 3 – Requirements Are Fixed
 -  Principle 7 – KPIs Drive Architecture
 -  Principle 10 – No Accidental Decisions
 -  Principle 11 – Engineering over Convenience
-

API-006 Structure

API-006

—	API-006.1 Module Overview
—	API-006.2 Examination Lifecycle
—	API-006.3 Examination CRUD APIs
—	API-006.4 Scheduling APIs
—	API-006.5 Candidate Registration APIs
—	API-006.6 Publication & Status APIs
—	API-006.7 Monitoring APIs
—	API-006.8 Validation & Business Rules
—	API-006.9 Performance Considerations
—	API-006.10 Security Considerations
—	API-006.11 Integration with Other Modules

API-006.1 — Module Overview

This module manages every examination from creation until archival.

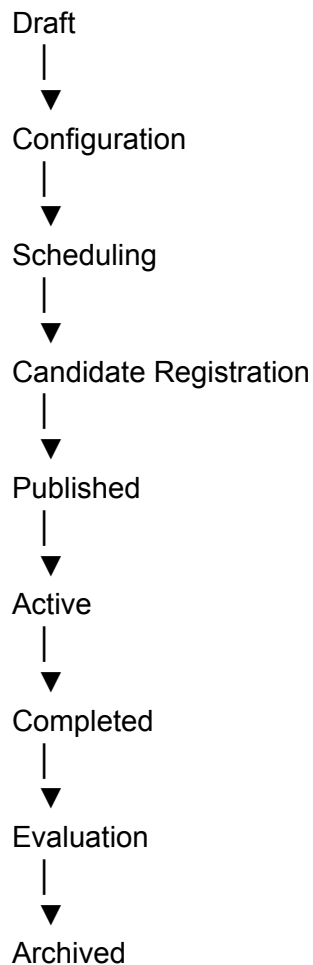
Responsibilities include:

- Create examinations
- Configure examination settings
- Assign subject

- Assign faculty
 - Define duration
 - Configure marking scheme
 - Configure availability
 - Register candidates
 - Publish examination
 - Cancel examination
 - Archive completed examinations
-

API-006.2 — Examination Lifecycle

Every examination follows the same lifecycle.



Cancelled examinations exit the lifecycle after publication or scheduling.

API-006.3 — Examination CRUD APIs

Create Examination

Property	Value
Method	POST
Endpoint	<code>/api/v1/exams</code>
Authentication	Required
Authorization	Admin / Faculty

Request

```
{  
  "title": "DBMS Mid Semester",  
  "subjectId": 5,  
  "duration": 90,  
  "totalMarks": 50,  
  "passingMarks": 20  
}
```

Database Tables

- Exam
- Subject
- Faculty
- AuditLog

Transaction

Yes

Get All Examinations

GET `/api/v1/exams`

Supports:

- Pagination
 - Filtering
 - Sorting
-

Get Examination

GET /api/v1/exams/{examId}

Update Examination

PUT /api/v1/exams/{examId}

Archive Examination

DELETE /api/v1/exams/{examId}

Implements soft deletion by changing the examination status to **ARCHIVED**.

API-006.4 — Scheduling APIs

Schedule when an examination will be conducted.

Schedule Examination

POST /api/v1/exams/{examId}/schedule

Example Request

```
{
```

```
"startTime":"2026-09-12T10:00:00Z",  
"endTime":"2026-09-12T11:30:00Z"  
}
```

Update Schedule

PUT /api/v1/exams/{examId}/schedule

Get Schedule

GET /api/v1/exams/{examId}/schedule

Cancel Schedule

DELETE /api/v1/exams/{examId}/schedule

Database Tables

- Exam
 - ExamSchedule
-

API-006.5 — Candidate Registration APIs

Register eligible students for an examination.

Register Candidate

POST /api/v1/exams/{examId}/candidates

Request

```
{  
  "studentId":105
```


}

Remove Candidate

DELETE /api/v1/exams/{examId}/candidates/{studentId}

List Registered Candidates

GET /api/v1/exams/{examId}/candidates

Student Examination List

GET /api/v1/students/{studentId}/exams

Database Tables

- CandidateRegistration
 - Student
 - Exam
-

API-006.6 — Publication & Status APIs

Examination states

Draft
Scheduled
Published
Active
Completed
Cancelled
Archived

Endpoints

Publish Exam

PATCH /api/v1/exams/{examId}/publish

Start Exam

PATCH /api/v1/exams/{examId}/start

Complete Exam

PATCH /api/v1/exams/{examId}/complete

Cancel Exam

PATCH /api/v1/exams/{examId}/cancel

Archive Exam

PATCH /api/v1/exams/{examId}/archive

API-006.7 — Monitoring APIs

Administrative monitoring endpoints.

Current Active Exams

GET /api/v1/exams/active

Upcoming Exams

GET /api/v1/exams/upcoming

Completed Exams

GET /api/v1/exams/completed

Examination Statistics

GET /api/v1/exams/{examId}/statistics

Statistics may include:

- Total registered candidates
 - Present candidates
 - Absent candidates
 - Submission count
 - Completion percentage
 - Average duration
-

API-006.8 — Validation & Business Rules

Business Rules

- Subject must exist.
 - Faculty must be assigned to the subject.
 - Start time must precede end time.
 - Examination title must be unique within a subject and academic term.
 - Passing marks cannot exceed total marks.
 - Duration must be greater than zero.
 - Candidate registration closes before the scheduled start time.
 - A cancelled examination cannot be published.
 - An archived examination cannot be modified.
 - Schedule conflicts for faculty or examination venues (if applicable) are not permitted.
-

API-006.9 — Performance Considerations

To satisfy project KPIs:

- Index `examId`, `subjectId`, and `scheduleId`.
- Cache examination metadata after publication.
- Paginate examination listings.
- Optimize candidate registration with bulk operations.
- Use indexed schedule queries for upcoming and active examinations.

- Maintain short transactions during registration and scheduling.

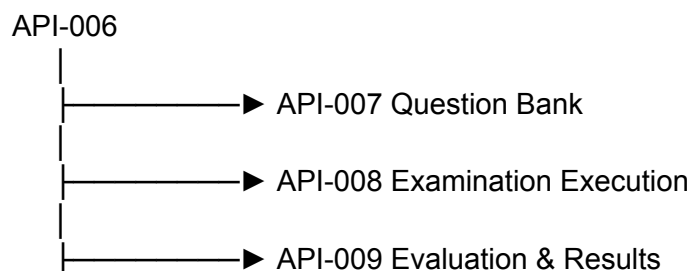
API-006.10 — Security Considerations

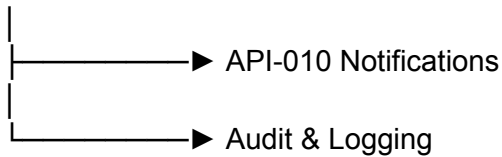
Operation	Student	Faculty	Admin
View Published Exams	✓	✓	✓
Create Exam	✗	Assigned Faculty	✓
Update Draft Exam	✗	Owner Faculty	✓
Schedule Exam	✗	Assigned Faculty	✓
Register Candidate	✗	Limited*	✓
Publish Exam	✗	Assigned Faculty	✓
Cancel Exam	✗	✗	✓
Archive Exam	✗	✗	✓
View Statistics	✗	Assigned Faculty	✓

*Faculty permissions depend on institutional policy.

Every modification generates an [AuditLog](#) entry.

API-006.11 — Integration with Other Modules





The examination definition created here becomes the primary entity used by the remaining functional modules.

Endpoint Summary

Method	Endpoint	Purpose
POST	<code>/api/v1/exams</code>	Create examination
GET	<code>/api/v1/exams</code>	List examinations
GET	<code>/api/v1/exams/{examId}</code>	View examination
PUT	<code>/api/v1/exams/{examId}</code>	Update examination
DELETE	<code>/api/v1/exams/{examId}</code>	Archive examination
POST	<code>/api/v1/exams/{examId}/schedule</code>	Schedule examination
PUT	<code>/api/v1/exams/{examId}/schedule</code>	Update schedule
GET	<code>/api/v1/exams/{examId}/schedule</code>	View schedule
DELETE	<code>/api/v1/exams/{examId}/schedule</code>	Cancel schedule
POST	<code>/api/v1/exams/{examId}/candidates</code>	Register candidate
DELETE	<code>/api/v1/exams/{examId}/candidates/{studentId}</code>	Remove candidate
GET	<code>/api/v1/exams/{examId}/candidates</code>	List registered candidates
GET	<code>/api/v1/students/{studentId}/exams</code>	Student examination list
PATCH	<code>/api/v1/exams/{examId}/publish</code>	Publish examination

PATCH	<code>/api/v1/exams/{examId}/start</code>	Start examination
PATCH	<code>/api/v1/exams/{examId}/complete</code>	Complete examination
PATCH	<code>/api/v1/exams/{examId}/cancel</code>	Cancel examination
PATCH	<code>/api/v1/exams/{examId}/archive</code>	Archive examination
GET	<code>/api/v1/exams/active</code>	Active examinations
GET	<code>/api/v1/exams/upcoming</code>	Upcoming examinations
GET	<code>/api/v1/exams/completed</code>	Completed examinations
GET	<code>/api/v1/exams/{examId}/statistics</code>	Examination statistics

Database Tables Used

Table	Purpose
Exam	Examination master record
ExamSchedule	Scheduling information
Subject	Associated subject
Faculty	Examination owner
CandidateRegistration	Registered candidates
Student	Candidate information
AuditLog	Administrative audit trail

Core Subject Mapping

Subject	Contribution
---------	--------------

Software Engineering	Examination lifecycle management, modular APIs, separation of concerns
OOP	Examination, Schedule, and Registration domain services
DBMS	Transactions, referential integrity, scheduling consistency, registration management
Computer Networks	RESTful APIs for examination administration
DSA	Efficient filtering, searching, scheduling, and statistics retrieval
Operating Systems	Concurrent candidate registration and examination state transitions with transactional consistency

Engineering Benefits

Engineering Goal	API Contribution
Maintainability	Centralized examination management module
Scalability	Stateless APIs with paginated listings
Reliability	Controlled examination lifecycle and transactional scheduling
Data Integrity	Validated scheduling, registration, and status transitions
Performance	Indexed examination queries and optimized registration workflows
Security	RBAC-protected examination administration

Deliverables Produced

API-006 produces:

-  Examination Management API Specification
-  Examination Lifecycle Model
-  Scheduling API Specification
-  Candidate Registration API Specification

-  Examination Status Management APIs
 -  Monitoring & Statistics API Specification
 -  Validation & Business Rules
 -  Security & Authorization Matrix
 -  Endpoint Catalog
 -  Integration Model
-

API-006 Status: Complete

API-006 completes the **Examination Management domain**, defining the full lifecycle of an examination from creation through scheduling, publication, execution readiness, and archival. It provides the orchestration layer that connects the academic structure (API-005) with the Question Bank (API-007), Examination Execution (API-008), Evaluation & Results (API-009), and Notification services (API-010), making it the central coordination module of the Online Examination Platform.

Excellent. We now move to the **Question Bank Management Module**, which is one of the core modules of the Online Examination Platform.

This module manages the complete lifecycle of questions—from creation and categorization to validation, approval, versioning, and association with examinations. Unlike API-006, which manages the exam itself, API-007 manages the content that will appear in those exams.

API-007 — Question Bank Management APIs

Status: API Design Phase

Objective

Design a scalable and secure API framework for managing the centralized Question Bank, including:

- Question Creation
- Question Update
- Question Categorization
- Difficulty Level Management
- Question Approval Workflow
- Question Versioning
- Question Search & Filtering
- Exam Question Mapping

This module ensures that questions are reusable, maintainable, and suitable for automated examination generation.

Design Principles Applied

-  Principle 1 – Integration of Core Computer Science Subjects
 -  Principle 2 – Every Design Decision Must Answer Two Questions
 -  Principle 3 – Requirements Are Fixed
 -  Principle 7 – KPIs Drive Architecture
 -  Principle 10 – No Accidental Decisions
 -  Principle 11 – Engineering over Convenience
-

API-007 Structure

API-007

—	API-007.1 Module Overview
—	API-007.2 Question Lifecycle
—	API-007.3 Question CRUD APIs
—	API-007.4 Question Category & Difficulty APIs
—	API-007.5 Question Approval & Versioning APIs
—	API-007.6 Exam Question Mapping APIs
—	API-007.7 Search & Query APIs

- API-007.8 Validation & Business Rules
 - API-007.9 Performance Considerations
 - API-007.10 Security Considerations
 - API-007.11 Integration with Other Modules
-

API-007.1 — Module Overview

This module manages the centralized repository of examination questions.

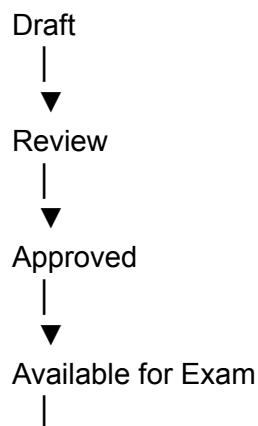
Primary responsibilities:

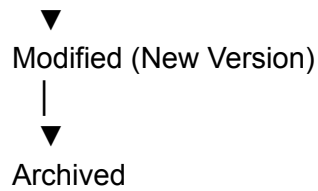
- Create questions
- Update questions
- Delete (soft delete) questions
- Categorize questions
- Assign difficulty levels
- Maintain question versions
- Approve questions before use
- Map questions to examinations
- Search and filter questions

This module does **not** conduct examinations or evaluate answers.

API-007.2 — Question Lifecycle

Every question follows a controlled lifecycle.





Only **Approved** questions can be associated with examinations.

API-007.3 — Question CRUD APIs

Create Question

Property	Value
Method	POST
Endpoint	<code>/api/v1/questions</code>
Authentication	Required
Authorization	Faculty / Admin

Request

```
{  
  "subjectId": 5,  
  "categoryId": 2,  
  "difficultyLevel": "MEDIUM",  
  "questionType": "MCQ",  
  "questionText": "Which normal form removes partial dependency?",  
  "marks": 2  
}
```

Database Tables:

- Question
- Subject
- QuestionCategory
- AuditLog

Transaction:

Yes

Get All Questions

GET /api/v1/questions

Supports:

- Pagination
 - Filtering
 - Sorting
-

Get Question

GET /api/v1/questions/{questionId}

Update Question

PUT /api/v1/questions/{questionId}

Updating an approved question creates a **new version** instead of modifying the original.

Archive Question

DELETE /api/v1/questions/{questionId}

Implements soft deletion by changing the status to **ARCHIVED**.

API-007.4 — Question Category & Difficulty APIs

Create Question Category

POST /api/v1/question-categories

Update Question Category

PUT /api/v1/question-categories/{categoryId}

List Categories

GET /api/v1/question-categories

Difficulty Levels

Supported values:

EASY
MEDIUM
HARD

Retrieve supported levels:

GET /api/v1/question-difficulties

Categories help organize questions by topics such as:

- Database
- Networking
- Programming
- Operating Systems
- Cyber Security

API-007.5 — Question Approval & Versioning APIs

Submit for Review

PATCH /api/v1/questions/{questionId}/submit

Approve Question

PATCH /api/v1/questions/{questionId}/approve

Reject Question

PATCH /api/v1/questions/{questionId}/reject

View Version History

GET /api/v1/questions/{questionId}/versions

Restore Previous Version

PATCH /api/v1/questions/{questionId}/restore/{versionId}

Database Tables:

- Question
 - QuestionVersion
 - AuditLog
-

API-007.6 — Exam Question Mapping APIs

Associate approved questions with examinations.

Add Question to Exam

POST /api/v1/exams/{examId}/questions

Request

```
{  
  "questionId": 102  
}
```

Remove Question from Exam

DELETE /api/v1/exams/{examId}/questions/{questionId}

List Exam Questions

GET /api/v1/exams/{examId}/questions

Bulk Question Assignment

POST /api/v1/exams/{examId}/questions/bulk

Useful for importing multiple approved questions into an examination.

API-007.7 — Search & Query APIs

Examples:

Search by keyword

GET /api/v1/questions?keyword=normalization

Search by subject

GET /api/v1/questions?subjectId=5

Search by category

GET /api/v1/questions?categoryId=3

Search by difficulty

GET /api/v1/questions?difficulty=HARD

Search by approval status

GET /api/v1/questions?status=APPROVED

Supports:

- Pagination
- Filtering
- Sorting

API-007.8 — Validation & Business Rules

Business Rules:

- Subject must exist.
- Category must exist.
- Question text cannot be empty.
- Marks must be greater than zero.
- Difficulty level must be valid.
- Approved questions cannot be edited directly.
- Only approved questions can be assigned to exams.
- Duplicate questions within the same subject should be prevented.

- Archived questions cannot be reused.
 - Each question must have at least one correct answer defined before approval.
-

API-007.9 — Performance Considerations

To satisfy project KPIs:

- Index `questionId`, `subjectId`, `categoryId`, and `difficultyLevel`.
 - Use full-text indexing for question searches.
 - Paginate large question collections.
 - Cache frequently accessed categories and difficulty metadata.
 - Optimize bulk question assignment operations.
 - Use efficient joins for exam-question retrieval.
-

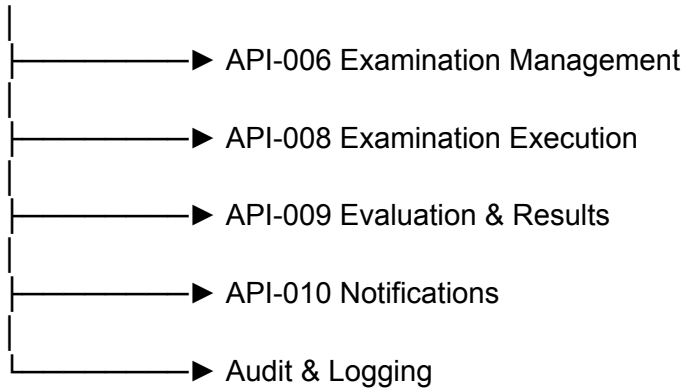
API-007.10 — Security Considerations

Operation	Student	Faculty	Admin
View Approved Questions	✗	Assigned Subjects	✓
Create Question	✗	✓	✓
Update Draft Question	✗	Owner Faculty	✓
Submit for Review	✗	Owner Faculty	✓
Approve / Reject Question	✗	✗	✓
Archive Question	✗	✗	✓
Map Questions to Exam	✗	Assigned Faculty	✓
View Version History	✗	Owner Faculty	✓

Every modification generates an `AuditLog` entry.

API-007.11 — Integration with Other Modules

API-007



Approved questions become the source of truth for examination execution and evaluation.

Endpoint Summary

Method	Endpoint	Purpose
POST	/api/v1/questions	Create question
GET	/api/v1/questions	List questions
GET	/api/v1/questions/{questionId}	View question
PUT	/api/v1/questions/{questionId}	Update question
DELETE	/api/v1/questions/{questionId}	Archive question
POST	/api/v1/question-categories	Create category
PUT	/api/v1/question-categories/{categoryId}	Update category
GET	/api/v1/question-categories	List categories
GET	/api/v1/question-difficulties	List difficulty levels

PATCH	<code>/api/v1/questions/{questionId}/submit</code>	Submit for review
PATCH	<code>/api/v1/questions/{questionId}/approve</code>	Approve question
PATCH	<code>/api/v1/questions/{questionId}/reject</code>	Reject question
GET	<code>/api/v1/questions/{questionId}/versions</code>	View version history
PATCH	<code>/api/v1/questions/{questionId}/restore/{versionId}</code>	Restore previous version
POST	<code>/api/v1/exams/{examId}/questions</code>	Add question to exam
DELETE	<code>/api/v1/exams/{examId}/questions/{questionId}</code>	Remove question from exam
GET	<code>/api/v1/exams/{examId}/questions</code>	List exam questions
POST	<code>/api/v1/exams/{examId}/questions/bulk</code>	Bulk assign questions

Database Tables Used

Table	Purpose
Question	Question master record
QuestionCategory	Category definitions
QuestionVersion	Version history
ExamQuestion	Exam-question mapping
Subject	Subject reference
Faculty	Question owner
AuditLog	Audit trail

Core Subject Mapping

Subject	Contribution
Software Engineering	Modular question management, approval workflow, version control
OOP	Question, Category, Version, and Mapping domain services
DBMS	Referential integrity, version persistence, many-to-many exam-question relationships
Computer Networks	RESTful APIs for question repository access
DSA	Full-text search, indexed filtering, optimized bulk operations
Operating Systems	Concurrent editing protection, transaction management, efficient resource utilization

Engineering Benefits

Engineering Goal	API Contribution
Maintainability	Centralized reusable question repository
Scalability	Efficient search and bulk assignment capabilities
Data Integrity	Approval workflow and version-controlled modifications
Performance	Indexed searches and optimized mappings
Security	RBAC-controlled authoring and approval process
Reusability	Questions can be reused across multiple examinations

Deliverables Produced

API-007 produces:

-  Question Bank API Specification
 -  Question Lifecycle Model
 -  Category & Difficulty Management APIs
 -  Question Approval Workflow
 -  Version Control API Specification
 -  Exam Question Mapping APIs
 -  Search & Filtering API Specification
 -  Validation & Business Rules
 -  Security & Authorization Matrix
 -  Endpoint Catalog
-

API-007 Status: Complete

API-007 completes the **Question Bank Management domain**, providing a centralized, version-controlled repository of examination questions. Together with API-006, it enables complete examination preparation by defining both the examination structure and its content. The next module, **API-008**, will consume these definitions to manage the **live examination execution**, including question delivery, answer submission, timing, auto-save, and examination session control.

Excellent. We now enter the **runtime phase** of the Online Examination Platform.

Up to API-007, everything focused on **preparing** an examination:

- API-003 → Authentication
- API-004 → User Management
- API-005 → Academic Management
- API-006 → Examination Management
- API-007 → Question Bank

Now, **API-008** is responsible for **conducting the examination itself**. This is arguably the most critical API module because it handles the live interaction between students and the examination system.

API-008 — Examination Execution APIs

Status: API Design Phase

Objective

Design a secure, scalable, fault-tolerant API framework for conducting online examinations, including:

- Examination Session Management
- Candidate Verification
- Examination Launch
- Question Delivery
- Answer Submission
- Auto Save
- Time Management
- Examination Submission
- Session Recovery
- Examination Monitoring

This module manages the complete examination attempt lifecycle from start to finish.

Design Principles Applied

-  Principle 1 – Integration of Core Computer Science Subjects
 -  Principle 2 – Every Design Decision Must Answer Two Questions
 -  Principle 3 – Requirements Are Fixed
 -  Principle 7 – KPIs Drive Architecture
 -  Principle 10 – No Accidental Decisions
 -  Principle 11 – Engineering over Convenience
 -  Principle 12 – Reliability Before Optimization
-

API-008 Structure

API-008

- |
 - | — API-008.1 Module Overview
 - | — API-008.2 Examination Attempt Lifecycle
 - | — API-008.3 Examination Session APIs
 - | — API-008.4 Question Delivery APIs
 - | — API-008.5 Answer Submission APIs
 - | — API-008.6 Auto Save & Recovery APIs
 - | — API-008.7 Timer & Submission APIs
 - | — API-008.8 Validation & Business Rules
 - | — API-008.9 Performance Considerations
 - | — API-008.10 Security Considerations
 - | — API-008.11 Integration with Other Modules
-

API-008.1 — Module Overview

This module controls the execution of a live examination.

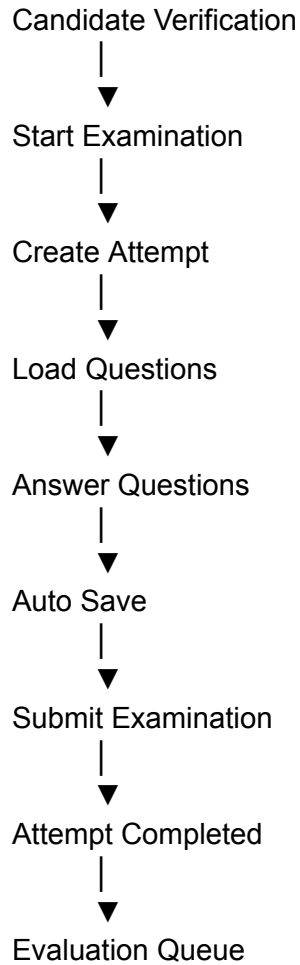
Primary responsibilities:

- Verify candidate eligibility
- Start examination
- Create examination attempt
- Deliver questions
- Receive answers
- Auto-save responses
- Resume interrupted sessions
- Submit examination
- Auto-submit on timeout
- Record examination activity

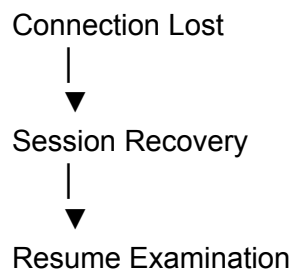
This module does **not** evaluate answers. Evaluation belongs to API-009.

API-008.2 — Examination Attempt Lifecycle

Every examination attempt follows a controlled lifecycle.



If a network interruption occurs:



API-008.3 — Examination Session APIs

Start Examination

Property	Value
Method	POST
Endpoint	<code>/api/v1/exams/{examId}/start</code>
Authentication	Required
Authorization	Registered Student

Request

```
{  
  "studentId": 105  
}
```

Response

```
{  
  "success": true,  
  "attemptId": 501,  
  "remainingTime": 5400,  
  "status": "ACTIVE"  
}
```

Database Tables:

- CandidateRegistration
- ExamAttempt
- UserSession
- AuditLog

Transaction:

Yes

Get Current Attempt

GET `/api/v1/exams/{examId}/attempt`

Returns:

- Attempt Status
 - Remaining Time
 - Progress
 - Last Saved Timestamp
-

Resume Attempt

POST /api/v1/exams/{examId}/resume

Used after network interruption or browser refresh.

End Attempt

PATCH /api/v1/exams/{examId}/end

Marks the examination as completed.

API-008.4 — Question Delivery APIs

Get Next Question

GET /api/v1/exams/{examId}/questions/next

Get Question by Number

GET /api/v1/exams/{examId}/questions/{questionNumber}

List All Questions

GET /api/v1/exams/{examId}/questions

Returns only metadata:

- Question Number
- Answer Status
- Marked for Review

Not the complete question content, unless permitted by the exam configuration.

API-008.5 — Answer Submission APIs

Save Answer

POST /api/v1/exams/{examId}/answers

Request

```
{
  "questionId": 52,
  "selectedOption": "B"
}
```

Database Tables

- StudentAnswer
- ExamAttempt

Transaction

Yes

Update Answer

PUT /api/v1/exams/{examId}/answers/{questionId}

Clear Answer

DELETE /api/v1/exams/{examId}/answers/{questionId}

Mark for Review

PATCH /api/v1/exams/{examId}/answers/{questionId}/review

API-008.6 — Auto Save & Recovery APIs

Auto Save

POST /api/v1/exams/{examId}/autosave

Automatically invoked by the client at configurable intervals (e.g., every 30–60 seconds).

Recover Session

GET /api/v1/exams/{examId}/recovery

Returns:

- Last saved answers
 - Remaining time
 - Current question
 - Examination state
-

Synchronize Client State

POST /api/v1/exams/{examId}/sync

Used after reconnection to synchronize local and server state.

API-008.7 — Timer & Submission APIs

Get Remaining Time

GET /api/v1/exams/{examId}/timer

Response

```
{
  "remainingTime": 1850,
  "status": "ACTIVE"
}
```

Submit Examination

POST /api/v1/exams/{examId}/submit

Operations performed:

- Lock attempt
- Store final answers
- Mark attempt completed
- Queue evaluation
- Generate audit entry

Auto Submission

Automatically triggered when:

- Timer expires
- Examination closes
- Administrator force-submits attempt

No separate public endpoint is exposed; the server performs this action internally.

API-008.8 — Validation & Business Rules

Business Rules

- Student must be registered for the examination.
 - Examination must be in **ACTIVE** state.
 - Only one active attempt per student per examination.
 - Answers cannot be modified after submission.
 - Timer is maintained by the server.
 - Auto-save cannot create duplicate answers.
 - Resume is allowed only within the examination window.
 - Examination automatically submits when time expires.
 - Archived or cancelled examinations cannot be started.
-

API-008.9 — Performance Considerations

To satisfy project KPIs:

- Cache examination metadata during execution.
 - Minimize payload size for question delivery.
 - Batch auto-save operations where possible.
 - Use indexed lookups for attempts and answers.
 - Avoid full question reloads after every answer.
 - Keep answer save transactions short.
-

API-008.10 — Security Considerations

Operation	Student	Faculty	Admin
Start Examination	✓ Registered Only	✗	✗
View Questions	During Active Attempt	Limited (Monitoring)	✓
Save Answer	✓	✗	✗

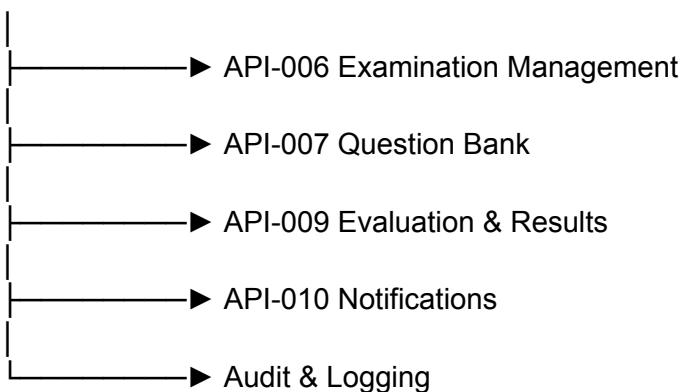
Resume Attempt	✓	✗	✗
Submit Examination	✓	✗	✗
Force Submit	✗	✗	✓
Monitor Active Attempts	✗	Assigned Exams	✓

Security Measures:

- Server-side timer enforcement.
- Session validation before every request.
- Attempt locking after submission.
- Duplicate submission prevention.
- All examination activities recorded in [AuditLog](#).

API-008.11 — Integration with Other Modules

API-008



Once an examination is submitted, the completed attempt is passed to API-009 for evaluation.

Endpoint Summary

Method	Endpoint	Purpose
--------	----------	---------

POST	<code>/api/v1/exams/{examId}/start</code>	Start examination
GET	<code>/api/v1/exams/{examId}/attempt</code>	Current attempt
POST	<code>/api/v1/exams/{examId}/resume</code>	Resume attempt
PATCH	<code>/api/v1/exams/{examId}/end</code>	End attempt
GET	<code>/api/v1/exams/{examId}/questions</code>	List question navigation
GET	<code>/api/v1/exams/{examId}/questions/next</code>	Next question
GET	<code>/api/v1/exams/{examId}/questions/{questionNumber}</code>	Get question
POST	<code>/api/v1/exams/{examId}/answers</code>	Save answer
PUT	<code>/api/v1/exams/{examId}/answers/{questionId}</code>	Update answer
DELETE	<code>/api/v1/exams/{examId}/answers/{questionId}</code>	Clear answer
PATCH	<code>/api/v1/exams/{examId}/answers/{questionId}/review</code>	Mark for review
POST	<code>/api/v1/exams/{examId}/autosave</code>	Auto-save answers
GET	<code>/api/v1/exams/{examId}/recovery</code>	Recover session
POST	<code>/api/v1/exams/{examId}/sync</code>	Synchronize client state
GET	<code>/api/v1/exams/{examId}/timer</code>	Remaining time
POST	<code>/api/v1/exams/{examId}/submit</code>	Submit examination

Database Tables Used

Table	Purpose
Exam	Examination definition

ExamAttempt	Student examination attempts
StudentAnswer	Candidate responses
CandidateRegistration	Candidate eligibility
Question	Question retrieval
UserSession	Session validation
AuditLog	Activity logging

Core Subject Mapping

Subject	Contribution
Software Engineering	Examination workflow orchestration, modular execution services
OOP	Attempt, Session, Answer, and Timer service classes
DBMS	Transactional answer persistence, attempt management, recovery mechanisms
Computer Networks	Stateless REST communication, reliable request handling, session recovery
Operating Systems	Concurrent examination sessions, timer management, synchronization, resource utilization
DSA	Efficient question navigation, progress tracking, and optimized answer retrieval

Engineering Benefits

Engineering Goal	API Contribution
Reliability	Auto-save, session recovery, server-side timer enforcement

Scalability	Stateless APIs with efficient session tracking
Performance	Lightweight question delivery and optimized answer persistence
Security	Attempt locking, duplicate submission prevention, RBAC enforcement
Data Integrity	Transactional answer storage and controlled attempt lifecycle
User Experience	Seamless resume capability and continuous progress saving

Deliverables Produced

API-008 produces:

-  Examination Execution API Specification
 -  Examination Attempt Lifecycle
 -  Session Management APIs
 -  Question Delivery APIs
 -  Answer Submission APIs
 -  Auto Save & Recovery APIs
 -  Timer & Submission APIs
 -  Validation & Business Rules
 -  Security & Authorization Matrix
 -  Endpoint Catalog
-

API-008 Status: Complete

API-008 completes the **live examination execution domain**. It transforms the examination definitions from API-006 and the approved questions from API-007 into an interactive examination experience for students. Through robust session management, automatic answer persistence, recovery mechanisms, and secure submission workflows, it provides a reliable and scalable runtime environment. The next module, **API-009**, will process these submitted attempts to perform evaluation, grading, result generation, and publication.

Excellent. We now reach the **Evaluation & Result Processing Module**, which is responsible for transforming a student's submitted examination into official results.

This module begins immediately after **API-008 (Examination Execution)** completes. It processes submitted answers, calculates marks, applies grading rules, generates results, and publishes them to students and faculty.

API-009 — Evaluation & Result Management APIs

Status: API Design Phase

Objective

Design a secure, scalable, and modular API framework for evaluating examination attempts and generating official results, including:

- Automatic Evaluation
- Manual Evaluation
- Score Calculation
- Grade Calculation
- Result Generation
- Result Publication
- Result Review
- Result Statistics

This module completes the academic examination lifecycle.

Design Principles Applied

-  Principle 1 – Integration of Core Computer Science Subjects
-  Principle 2 – Every Design Decision Must Answer Two Questions

-  Principle 3 – Requirements Are Fixed
 -  Principle 7 – KPIs Drive Architecture
 -  Principle 10 – No Accidental Decisions
 -  Principle 11 – Engineering over Convenience
 -  Principle 12 – Reliability Before Optimization
-

API-009 Structure

API-009

—	API-009.1 Module Overview
—	API-009.2 Evaluation Lifecycle
—	API-009.3 Automatic Evaluation APIs
—	API-009.4 Manual Evaluation APIs
—	API-009.5 Result Generation APIs
—	API-009.6 Result Publication APIs
—	API-009.7 Analytics & Statistics APIs
—	API-009.8 Validation & Business Rules
—	API-009.9 Performance Considerations
—	API-009.10 Security Considerations
—	API-009.11 Integration with Other Modules

API-009.1 — Module Overview

This module is responsible for converting examination attempts into official academic results.

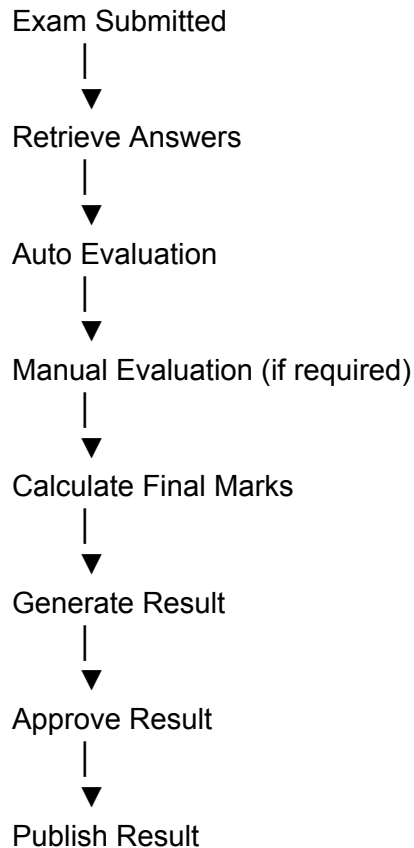
Primary responsibilities:

- Evaluate objective questions
- Support manual evaluation of subjective questions
- Calculate total marks
- Calculate percentage
- Assign grades
- Determine pass/fail status
- Generate result records
- Publish results
- Produce examination analytics

This module **does not** conduct examinations. That responsibility belongs to API-008.

API-009.2 — Evaluation Lifecycle

Every submitted examination follows a controlled evaluation process.



API-009.3 — Automatic Evaluation APIs

Used for objective question types such as:

- MCQ
- True/False
- Multiple Select (if supported)
- Numerical (with exact answer)

Start Automatic Evaluation

Property	Value
Method	POST
Endpoint	<code>/api/v1/evaluation/automatic</code>
Authentication	Required
Authorization	Admin / Authorized Faculty

Request

```
{  
  "examId": 15  
}
```

Evaluate Single Attempt

POST `/api/v1/evaluation/attempt/{attemptId}`

Compares:

- StudentAnswer
- CorrectAnswer

Produces:

- Obtained Marks
- Correct Count
- Wrong Count

Evaluation Status

GET `/api/v1/evaluation/{examId}/status`

Returns:

- Pending
 - Processing
 - Completed
-

API-009.4 — Manual Evaluation APIs

Required for:

- Subjective Questions
 - Essay Questions
 - Long Answer Questions
 - Programming Questions (if manual review is enabled)
-

Get Pending Evaluations

GET /api/v1/evaluation/manual/pending

Evaluate Answer

PATCH /api/v1/evaluation/manual/{answerId}

Request

```
{
  "marksAwarded": 8,
  "remarks": "Good explanation"
}
```

Complete Manual Evaluation

PATCH /api/v1/evaluation/manual/complete/{attemptId}

Marks the attempt as ready for final result generation.

API-009.5 — Result Generation APIs

Generate Result

POST /api/v1/results/generate

Request

```
{  
  "examId": 15  
}
```

Calculations:

- Total Marks
- Obtained Marks
- Percentage
- Grade
- Pass/Fail

Stores the finalized record in the Result table.

Get Student Result

GET /api/v1/results/student/{studentId}

Get Examination Results

GET /api/v1/results/exam/{examId}

Get Result by Attempt

GET /api/v1/results/attempt/{attemptId}

API-009.6 — Result Publication APIs

Results are not immediately visible after generation.

Publication requires explicit authorization.

Publish Results

PATCH /api/v1/results/{examId}/publish

Hide Results

PATCH /api/v1/results/{examId}/hide

Download Result

GET /api/v1/results/{resultId}/download

Supported formats:

- PDF
 - CSV (Administrative export)
-

API-009.7 — Analytics & Statistics APIs

Examination Statistics

GET /api/v1/results/{examId}/statistics

Returns:

- Highest Score
 - Lowest Score
 - Average Score
 - Median
 - Standard Deviation
 - Pass Percentage
 - Fail Percentage
-

Grade Distribution

GET /api/v1/results/{examId}/grades

Example:

A : 18
B : 42
C : 35
D : 10
F : 5

Subject Performance

GET /api/v1/results/subjects/{subjectId}

Provides performance analytics across examinations for the selected subject.

API-009.8 — Validation & Business Rules

Business Rules

- Evaluation starts only after submission.
 - Each attempt can be evaluated only once unless re-evaluation is authorized.
 - Results cannot be published before all evaluations are complete.
 - Marks awarded cannot exceed maximum marks.
 - Grades are derived from the configured grading policy.
 - Hidden results remain inaccessible to students.
 - Archived examinations cannot be re-evaluated without administrative approval.
 - Every evaluation action is logged.
-

API-009.9 — Performance Considerations

To satisfy project KPIs:

- Batch evaluate objective answers.
 - Index `attemptId`, `examId`, `studentId`, and `resultId`.
 - Cache grading policy.
 - Optimize aggregate calculations for statistics.
 - Generate reports asynchronously for large cohorts.
 - Support parallel evaluation of independent attempts.
-

API-009.10 — Security Considerations

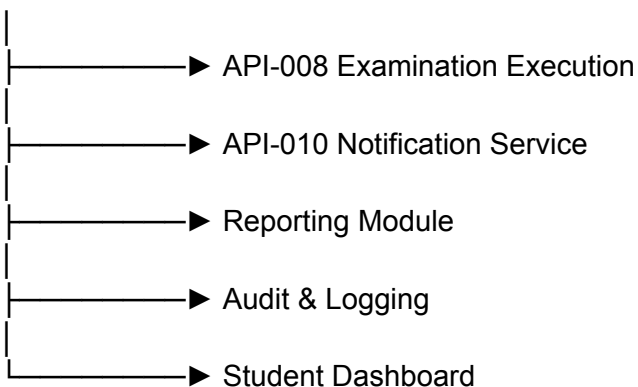
Operation	Student	Faculty	Admin
View Own Result	✓ After Publication	✗	✓
View All Results	✗	Assigned Exams	✓
Start Auto Evaluation	✗	Assigned Faculty	✓
Manual Evaluation	✗	Assigned Faculty	✓
Publish Results	✗	✗	✓
Download Own Result	✓ After Publication	✗	✓
View Statistics	✗	Assigned Exams	✓
Re-evaluate Attempt	✗	With Approval	✓

Security Measures:

- RBAC enforcement.
 - Immutable result records after publication (except authorized re-evaluation).
 - Full audit logging for evaluation and publication.
 - Digital integrity checks before publishing official results.
-

API-009.11 — Integration with Other Modules

API-009



Published results trigger notifications and become available on the student and faculty dashboards.

Endpoint Summary

Method	Endpoint	Purpose
POST	/api/v1/evaluation/automatic	Start automatic evaluation
POST	/api/v1/evaluation/attempt/{attemptId}	Evaluate single attempt
GET	/api/v1/evaluation/{examId}/status	Evaluation status

GET	<code>/api/v1/evaluation/manual/pending</code>	Pending manual evaluations
PATCH	<code>/api/v1/evaluation/manual/{answerId}</code>	Grade subjective answer
PATCH	<code>/api/v1/evaluation/manual/complete/{attemptId}</code>	Complete manual evaluation
POST	<code>/api/v1/results/generate</code>	Generate results
GET	<code>/api/v1/results/student/{studentId}</code>	Student results
GET	<code>/api/v1/results/exam/{examId}</code>	Examination results
GET	<code>/api/v1/results/attempt/{attemptId}</code>	Result by attempt
PATCH	<code>/api/v1/results/{examId}/publish</code>	Publish results
PATCH	<code>/api/v1/results/{examId}/hide</code>	Hide results
GET	<code>/api/v1/results/{resultId}/download</code>	Download result
GET	<code>/api/v1/results/{examId}/statistics</code>	Examination statistics
GET	<code>/api/v1/results/{examId}/grades</code>	Grade distribution
GET	<code>/api/v1/results/subjects/{subjectId}</code>	Subject performance analytics

Database Tables Used

Table	Purpose
ExamAttempt	Completed examination attempts
StudentAnswer	Candidate responses
Question	Correct answers and marking scheme
Result	Final result records
GradePolicy	Grade boundaries and grading rules

Core Subject Mapping

Subject	Contribution
Software Engineering	Modular evaluation pipeline, result publication workflow
OOP	Evaluation, Result, Grade, and Analytics service classes
DBMS	Transactional result generation, grading persistence, aggregate queries
Computer Networks	RESTful evaluation and result services
Operating Systems	Parallel evaluation jobs, concurrent processing, asynchronous report generation
DSA	Efficient score calculation, ranking, statistical aggregation, and analytics

Engineering Benefits

Engineering Goal	API Contribution
Reliability	Controlled evaluation and publication workflow
Scalability	Batch processing and parallel evaluation
Performance	Indexed lookups and optimized statistical queries
Data Integrity	Immutable published results with controlled re-evaluation
Security	RBAC-protected evaluation and publication
Transparency	Complete audit trail and detailed performance analytics

Deliverables Produced

API-009 produces:

-  Evaluation API Specification
-  Automatic Evaluation APIs
-  Manual Evaluation APIs
-  Result Generation APIs
-  Result Publication APIs
-  Analytics & Statistics APIs
-  Evaluation Lifecycle Model
-  Validation & Business Rules
-  Security & Authorization Matrix
-  Endpoint Catalog

API-009 Status: Complete

API-009 completes the **Evaluation & Result Management domain**, transforming submitted examination attempts into officially published academic results. It combines automated and manual evaluation workflows, supports scalable result generation, and provides comprehensive analytics for students, faculty, and administrators. Together with API-008, it completes the full examination execution pipeline—from exam participation to certified result publication. The next module, **API-010**, will provide the platform-wide **Notification & Communication Service**, enabling event-driven notifications for examinations, results, and administrative activities.

Excellent. We now complete the **Core Functional API Layer** of the Online Examination Platform.

API-010 is a **cross-cutting service**. Unlike the previous modules, it is not tied to a single business domain. Instead, it provides communication services that are consumed by every major module in the system.

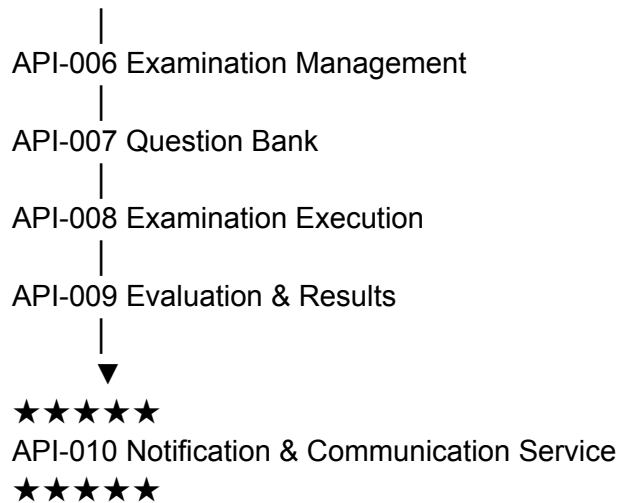
API-003 Authentication

|

API-004 User Management

|

API-005 Academic Management



Every important event generated by the system can trigger notifications through this module.

API-010 — Notification & Communication APIs

Status: API Design Phase

Objective

Design a centralized notification and communication framework responsible for delivering system-generated messages to students, faculty, and administrators.

The module handles:

- In-App Notifications
- Email Notifications
- SMS Notifications (Optional)
- Examination Alerts
- Result Notifications
- Password & Security Notifications
- System Announcements
- Notification Preferences

- Notification History

This module follows an **event-driven architecture**, where other modules publish events and the Notification Service processes and delivers them asynchronously.

Design Principles Applied

- ☒ Principle 1 – Integration of Core Computer Science Subjects
 - ☒ Principle 2 – Every Design Decision Must Answer Two Questions
 - ☒ Principle 7 – KPIs Drive Architecture
 - ☒ Principle 10 – No Accidental Decisions
 - ☒ Principle 11 – Engineering over Convenience
 - ☒ Principle 12 – Reliability Before Optimization
-

API-010 Structure

API-010

—	API-010.1 Module Overview
—	API-010.2 Notification Lifecycle
—	API-010.3 Notification APIs
—	API-010.4 Announcement APIs
—	API-010.5 User Preference APIs
—	API-010.6 Notification History APIs
—	API-010.7 Event Publishing APIs
—	API-010.8 Validation & Business Rules
—	API-010.9 Performance Considerations
—	API-010.10 Security Considerations
—	API-010.11 Integration with Other Modules

API-010.1 — Module Overview

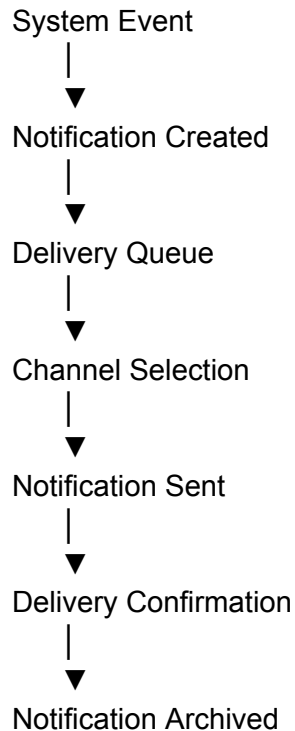
This module provides centralized communication across the platform.

Responsibilities:

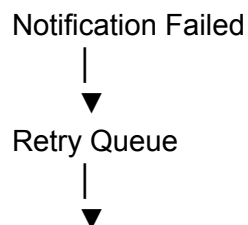
- Send notifications
- Deliver examination reminders
- Publish result announcements
- Notify password changes
- Notify examination schedule changes
- Store notification history
- Manage notification preferences
- Broadcast system announcements

The Notification Service is stateless and processes events generated by other modules.

API-010.2 — Notification Lifecycle



If delivery fails:



Resend

API-010.3 — Notification APIs

Send Notification

Property	Value
Method	POST
Endpoint	<code>/api/v1/notifications</code>
Authentication	Required
Authorization	System / Admin

Request

```
{
  "userId":105,
  "type":"EMAIL",
  "title":"Exam Scheduled",
  "message":"Your DBMS Mid Semester exam has been scheduled."
}
```

Database Tables

- Notification
- User
- AuditLog

Transaction

Yes

Get User Notifications

GET `/api/v1/notifications`

Returns notifications belonging to the authenticated user.

Supports:

- Pagination
 - Sorting
 - Filtering
-

Get Notification

GET /api/v1/notifications/{notificationId}

Mark Notification as Read

PATCH /api/v1/notifications/{notificationId}/read

Delete Notification

DELETE /api/v1/notifications/{notificationId}

Implements soft deletion.

API-010.4 — Announcement APIs

Announcements target multiple users simultaneously.

Create Announcement

POST /api/v1/announcements

Example

```
{  
  "title": "Semester Examination Notice",  
  "audience": "STUDENTS",  
  "message": "Semester examinations begin next Monday."  
}
```

Update Announcement

PUT /api/v1/announcements/{announcementId}

Publish Announcement

PATCH /api/v1/announcements/{announcementId}/publish

View Announcements

GET /api/v1/announcements

API-010.5 — User Preference APIs

Users can configure notification preferences.

Get Preferences

GET /api/v1/notification-preferences

Update Preferences

PUT /api/v1/notification-preferences

Example

```
{  
  "email":true,  
  "inApp":true,  
  "sms":false  
}
```

API-010.6 — Notification History APIs

Retrieve historical notification data.

Notification History

GET /api/v1/notifications/history

Notification Statistics

GET /api/v1/notifications/statistics

Example statistics:

- Total Sent
 - Total Delivered
 - Failed Notifications
 - Read Rate
 - Delivery Rate
-

API-010.7 — Event Publishing APIs

Other modules publish domain events to this service.

Examples:

Exam Published
Exam Cancelled
Exam Starting Soon
Exam Submitted
Evaluation Completed
Result Published
Password Changed
Account Locked
New Announcement

Internal endpoint:

POST /internal/events

This endpoint is consumed only by trusted internal services and is not exposed to end users.

API-010.8 — Validation & Business Rules

Business Rules

- User must exist before notification delivery.
 - Notification type must be supported.
 - Deleted users do not receive notifications.
 - Delivery channels respect user preferences.
 - Failed notifications are retried according to retry policy.
 - Announcements require publication before becoming visible.
 - Notification history cannot be modified by end users.
 - Duplicate notifications for the same event should be prevented within a configurable time window.
-

API-010.9 — Performance Considerations

To satisfy project KPIs:

- Use asynchronous delivery queues.
- Batch notifications for large audiences.
- Index `userId`, `notificationId`, and `createdAt`.
- Cache user notification preferences.
- Paginate notification history.
- Retry failed deliveries using exponential backoff.
- Separate notification processing workers from API servers.

API-010.10 — Security Considerations

Operation	Student	Faculty	Admin
View Own Notifications	✓	✓	✓
Mark as Read	✓ Own	✓ Own	✓
Delete Own Notification	✓	✓	✓
Create Announcement	✗	✗	✓
Publish Announcement	✗	✗	✓
View Statistics	✗	✗	✓
Publish Internal Event	✗	✗	Internal Services Only

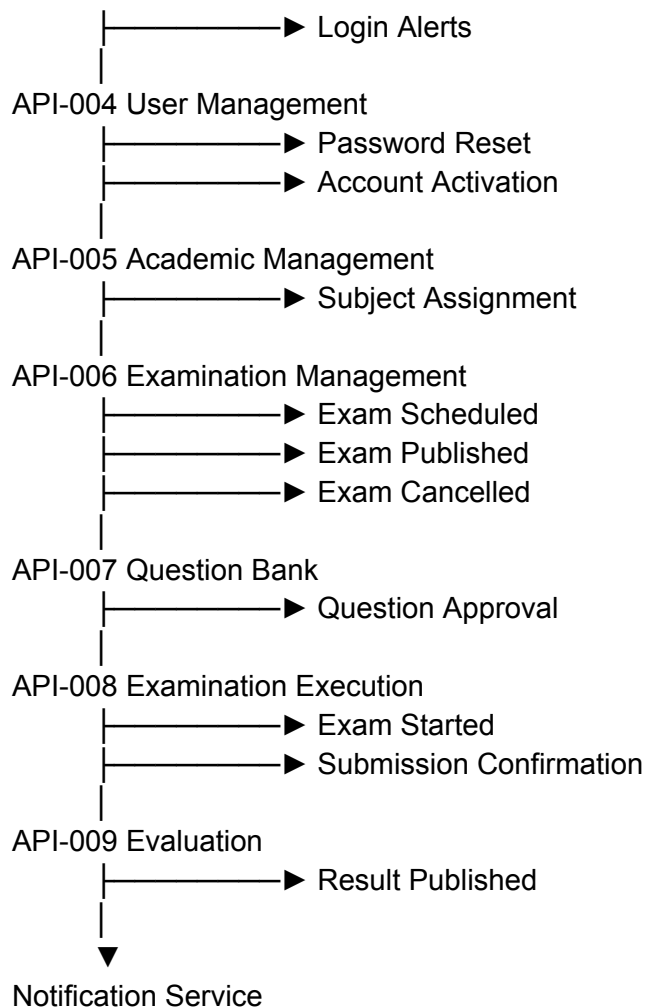
Security Measures:

- RBAC for announcements.
 - User isolation for notification history.
 - Internal event endpoint protected with service authentication.
 - All notification and announcement actions logged in `AuditLog`.
-

API-010.11 — Integration with Other Modules

API-003 Authentication

|



Endpoint Summary

Method	Endpoint	Purpose
POST	/api/v1/notifications	Send notification
GET	/api/v1/notifications	User notifications
GET	/api/v1/notifications/{notificationId}	View notification
PATCH	/api/v1/notifications/{notificationId}/read	Mark as read
DELETE	/api/v1/notifications/{notificationId}	Delete notification

Software Engineering	Event-driven notification architecture, modular communication services
OOP	Notification, Announcement, Preference, and Event service classes
DBMS	Persistent notification storage, delivery tracking, preference management
Computer Networks	REST APIs, asynchronous communication, message delivery
Operating Systems	Background workers, queues, concurrent event processing
DSA	Efficient queue management, filtering, notification retrieval, retry scheduling

Engineering Benefits

Engineering Goal	API Contribution
Reliability	Retry mechanisms and delivery confirmation
Scalability	Asynchronous event-driven processing
Performance	Queue-based delivery and batched notifications
Maintainability	Centralized communication service
Security	RBAC-controlled announcements and user isolation
Extensibility	New notification channels can be added without changing business modules

Deliverables Produced

API-010 produces:

-  Notification Service API Specification
-  Announcement Management APIs
-  Notification Preference APIs
-  Notification History APIs

-  Event Publishing Interface
 -  Notification Lifecycle Model
 -  Validation & Business Rules
 -  Security & Authorization Matrix
 -  Endpoint Catalog
 -  Event-Driven Integration Model
-

API-010 Status: Complete

API-010 completes the **Core Functional API Layer** of the Online Examination Platform. It introduces a centralized, event-driven communication service that integrates with every major module—from authentication and examination scheduling to evaluation and result publication. By decoupling notification delivery from business logic, it improves scalability, maintainability, and reliability while providing a consistent communication framework for the entire platform.

API Design Phase Progress

At this point, the primary functional API design is complete:

-  API-001 — API Architecture & Standards
-  API-002 — Common Response, Error Handling & Validation
-  API-003 — Authentication & Authorization
-  API-004 — User Management
-  API-005 — Academic & Subject Management
-  API-006 — Examination Management
-  API-007 — Question Bank Management
-  API-008 — Examination Execution
-  API-009 — Evaluation & Result Management
-  API-010 — Notification & Communication

The remaining API specifications can now focus on **cross-cutting and advanced capabilities**, such as reporting & analytics, audit & logging, file management, system administration, and the AI-based proctoring module that you intentionally planned to build last.

Excellent. We have now completed the **10 Core Functional API Specifications**.

From this point onward, we move into the **Enterprise Services Layer**. These APIs are not directly involved in conducting examinations but are essential for making the platform production-ready, maintainable, scalable, and enterprise-grade.

The order is:

API-001 API Standards
API-002 Common Response & Error Handling
API-003 Authentication & Authorization
API-004 User Management
API-005 Academic Management
API-006 Examination Management
API-007 Question Bank
API-008 Examination Execution
API-009 Evaluation & Results
API-010 Notification Service

API-011 Reporting & Analytics ← Current
API-012 Audit & Logging
API-013 File & Document Management
API-014 System Administration
API-015 AI Proctoring (Final Module)

API-011 — Reporting & Analytics APIs

Status: API Design Phase

Objective

Design a centralized Reporting & Analytics service responsible for generating operational, academic, administrative, and performance reports from data produced across the Online Examination Platform.

This module provides:

- Examination Reports
- Student Performance Reports
- Faculty Reports
- Subject-wise Reports
- Attendance Reports
- Result Analytics
- Dashboard Statistics
- Data Export
- Report Scheduling

Unlike API-009, which generates **individual results**, API-011 analyzes **large datasets** to produce insights and institutional reports.

Design Principles Applied

-  Principle 1 – Integration of Core Computer Science Subjects
 -  Principle 2 – Every Design Decision Must Answer Two Questions
 -  Principle 7 – KPIs Drive Architecture
 -  Principle 10 – No Accidental Decisions
 -  Principle 11 – Engineering over Convenience
 -  Principle 12 – Reliability Before Optimization
-

API-011 Structure

API-011

	— API-011.1 Module Overview
	— API-011.2 Report Lifecycle
	— API-011.3 Dashboard APIs
	— API-011.4 Examination Report APIs
	— API-011.5 Student Performance APIs
	— API-011.6 Faculty & Subject Analytics APIs
	— API-011.7 Export & Scheduled Reports
	— API-011.8 Validation & Business Rules
	— API-011.9 Performance Considerations
	— API-011.10 Security Considerations
	— API-011.11 Integration with Other Modules

API-011.1 — Module Overview

The Reporting module converts operational data into actionable information.

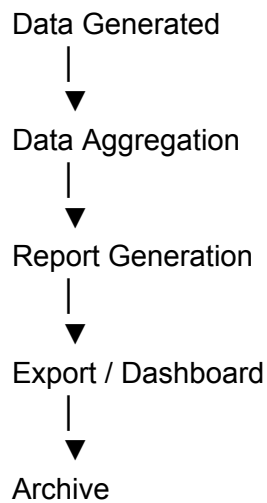
Responsibilities include:

- Generate examination reports

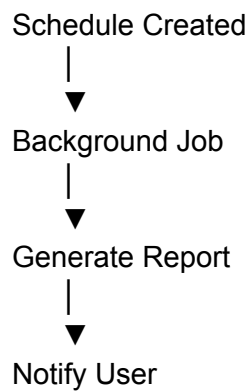
- Generate student performance reports
- Generate faculty workload reports
- Generate subject-wise analytics
- Generate institutional statistics
- Export reports
- Schedule recurring reports
- Feed administrative dashboards

This module is read-heavy and optimized for analytics rather than transactional updates.

API-011.2 — Report Lifecycle



For scheduled reports:



API-011.3 — Dashboard APIs

Dashboard Summary

Property	Value
Method	GET
Endpoint	/api/v1/dashboard
Authentication	Required

Returns summary metrics such as:

```
{
  "totalStudents": 850,
  "totalFaculty": 42,
  "activeExams": 6,
  "completedExams": 120,
  "publishedResults": 118
}
```

Examination Dashboard

GET </api/v1/dashboard/examinations>

Displays:

- Active examinations
- Upcoming examinations
- Scheduled examinations
- Cancelled examinations

Result Dashboard

GET </api/v1/dashboard/results>

Displays:

- Published results
 - Pending evaluations
 - Average score
 - Pass percentage
-

API-011.4 — Examination Report APIs

Examination Report

GET /api/v1/reports/examinations/{examId}

Contains:

- Registered candidates
 - Appeared candidates
 - Absent candidates
 - Submission statistics
 - Completion rate
 - Average score
 - Highest score
 - Lowest score
-

Examination Attendance

GET /api/v1/reports/examinations/{examId}/attendance

Examination Summary

GET /api/v1/reports/examinations

Supports:

- Pagination

- Filtering
 - Academic Year
 - Semester
 - Subject
-

API-011.5 — Student Performance APIs

Student Performance Report

GET /api/v1/reports/students/{studentId}

Includes:

- Examination history
 - Subject-wise marks
 - Percentage
 - Grade history
 - Attendance
 - Overall performance trend
-

Rank List

GET /api/v1/reports/rank-list

Supports filters:

- Semester
 - Subject
 - Examination
-

Performance Trend

GET /api/v1/reports/students/{studentId}/trend

Shows historical improvement across semesters.

API-011.6 — Faculty & Subject Analytics APIs

Faculty Report

GET /api/v1/reports/faculty/{facultyId}

Includes:

- Subjects handled
- Examinations conducted
- Questions created
- Evaluation workload

Subject Analytics

GET /api/v1/reports/subjects/{subjectId}

Displays:

- Pass percentage
- Failure rate
- Average marks
- Grade distribution
- Difficulty analysis

Institution Statistics

GET /api/v1/reports/institution

Institution-wide analytics.

API-011.7 — Export & Scheduled Reports

Export Report

GET /api/v1/reports/export

Supported formats:

- PDF
 - Excel (XLSX)
 - CSV
-

Schedule Report

POST /api/v1/reports/schedule

Example

```
{  
  "reportType":"Institution Summary",  
  "frequency":"MONTHLY"  
}
```

Scheduled Reports

GET /api/v1/reports/schedule

Delete Scheduled Report

DELETE /api/v1/reports/schedule/{scheduleId}

API-011.8 — Validation & Business Rules

Business Rules

- Reports are generated only for authorized users.
 - Archived data remains available for reporting.
 - Large reports are generated asynchronously.
 - Scheduled reports follow configured recurrence rules.
 - Ranking calculations use published results only.
 - Hidden results are excluded from student-facing reports.
 - Exported reports include generation timestamp and requesting user.
-

API-011.9 — Performance Considerations

To satisfy project KPIs:

- Use materialized summary tables for heavy analytics.
 - Cache dashboard metrics.
 - Paginate all report listings.
 - Generate large reports asynchronously.
 - Use indexed aggregate queries.
 - Precompute commonly requested statistics during off-peak hours.
-

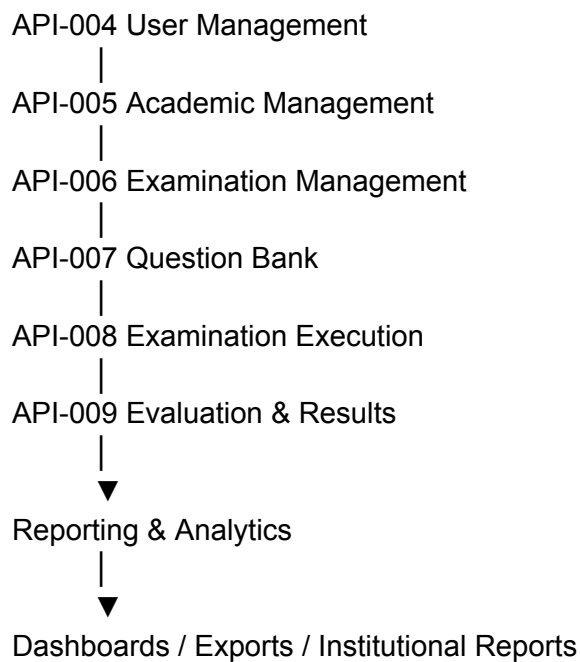
API-011.10 — Security Considerations

Operation	Student	Faculty	Admin
View Own Performance	✓	✗	✓
View Examination Report	✗	Assigned Exams	✓
View Faculty Report	✗	Own Only	✓
View Institution Report	✗	✗	✓
Export Reports	Own Reports	Assigned Reports	✓
Schedule Reports	✗	Limited	✓

Security Measures:

- RBAC enforcement.
 - Data filtering based on user role.
 - Export actions logged in [AuditLog](#).
 - Personally identifiable information included only where authorized.
-

API-011.11 — Integration with Other Modules



Endpoint Summary

Method	Endpoint	Purpose
GET	/api/v1/dashboard	Dashboard summary
GET	/api/v1/dashboard/examinations	Examination dashboard
GET	/api/v1/dashboard/results	Result dashboard

GET	/api/v1/reports/examinations/{examId}	Examination report
GET	/api/v1/reports/examinations/{examId}/attendance	Attendance report
GET	/api/v1/reports/examinations	Examination summary
GET	/api/v1/reports/students/{studentId}	Student performance
GET	/api/v1/reports/students/{studentId}/trend	Performance trend
GET	/api/v1/reports/rank-list	Rank list
GET	/api/v1/reports/faculty/{facultyId}	Faculty report
GET	/api/v1/reports/subjects/{subjectId}	Subject analytics
GET	/api/v1/reports/institution	Institution analytics
GET	/api/v1/reports/export	Export report
POST	/api/v1/reports/schedule	Schedule report
GET	/api/v1/reports/schedule	Scheduled reports
DELETE	/api/v1/reports/schedule/{scheduleId}	Delete scheduled report

Database Tables Used

Table	Purpose
Report	Report metadata and generation history
ReportSchedule	Scheduled report definitions
Result	Academic performance data
Exam	Examination information
Student	Student data

Faculty	Faculty data
Subject	Subject analytics
AuditLog	Export and reporting logs

Core Subject Mapping

Subject	Contribution
Software Engineering	Modular reporting services, scheduled report architecture
OOP	Report, Dashboard, Analytics, and Export service classes
DBMS	Aggregate queries, views, indexing, reporting schema
Computer Networks	RESTful reporting APIs and export services
Operating Systems	Background report generation, scheduling, concurrent analytics jobs
DSA	Ranking algorithms, statistical computations, efficient sorting and aggregation

Engineering Benefits

Engineering Goal	API Contribution
Decision Support	Institution-wide analytics and dashboards
Performance	Cached summaries and asynchronous report generation
Scalability	Scheduled jobs and optimized aggregate queries
Maintainability	Centralized reporting service
Security	Role-based access to reports and exports
Extensibility	Easy addition of new report templates and analytics

Deliverables Produced

API-011 produces:

-  Reporting & Analytics API Specification
-  Dashboard APIs
-  Examination Reporting APIs
-  Student Performance APIs
-  Faculty & Subject Analytics APIs
-  Report Export & Scheduling APIs
-  Report Lifecycle Model
-  Validation & Business Rules
-  Security & Authorization Matrix
-  Endpoint Catalog

API-011 Status: Complete

API-011 completes the **Reporting & Analytics domain** by providing comprehensive dashboards, institutional reports, performance analytics, ranking, and export capabilities. It transforms operational data collected by the previous modules into actionable insights for students, faculty, administrators, and institutional decision-makers.

Next: API-012 – Audit Logging & System Monitoring, which will define centralized auditing, activity tracking, security event logging, system health monitoring, and operational observability for the entire platform.

Excellent. We now move into one of the most important enterprise modules of the Online Examination Platform.

Unlike Reporting (API-011), which is intended for business intelligence, **API-012** focuses on **security, accountability, traceability, compliance, and operational monitoring**.

In any production-grade examination platform, every significant action must be traceable. This module provides the audit trail required for troubleshooting, compliance, forensic analysis, and system health monitoring.

API-012 — Audit Logging & System Monitoring APIs

Status: API Design Phase

Objective

Design a centralized Audit Logging & System Monitoring service responsible for recording user activities, security events, administrative actions, and system health metrics across the Online Examination Platform.

This module provides:

- Audit Logging
- User Activity Tracking
- Security Event Logging
- System Event Logging
- Error Logging
- Login History
- API Request Logs
- System Health Monitoring
- Performance Monitoring
- Operational Dashboards

Unlike API-010 (Notification Service), which communicates events, API-012 permanently records and monitors them.

Design Principles Applied

-  Principle 1 – Integration of Core Computer Science Subjects
-  Principle 2 – Every Design Decision Must Answer Two Questions
-  Principle 7 – KPIs Drive Architecture
-  Principle 10 – No Accidental Decisions
-  Principle 11 – Engineering over Convenience
-  Principle 12 – Reliability Before Optimization

API-012 Structure

API-012

- |
 - API-012.1 Module Overview
 - API-012.2 Audit Lifecycle
 - API-012.3 Audit Log APIs
 - API-012.4 User Activity APIs
 - API-012.5 Security Event APIs
 - API-012.6 System Monitoring APIs
 - API-012.7 Performance Metrics APIs
 - API-012.8 Validation & Business Rules
 - API-012.9 Performance Considerations
 - API-012.10 Security Considerations
 - API-012.11 Integration with Other Modules
-

API-012.1 — Module Overview

This module provides complete operational visibility into the system.

Responsibilities:

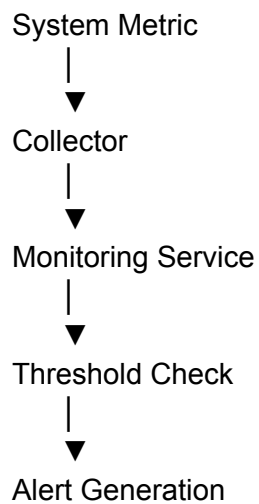
- Record user activities
- Record administrative actions
- Record authentication events
- Track API requests
- Track examination activities
- Monitor server health
- Monitor application performance
- Record system failures
- Provide operational dashboards

This module is append-only for audit records to preserve integrity.

API-012.2 — Audit Lifecycle



System monitoring lifecycle:



API-012.3 — Audit Log APIs

Create Audit Log (Internal)

Property

Value

Method	POST
Endpoint	<code>/internal/audit</code>
Authentication	Internal Services Only

Request

```
{
  "userId":105,
  "action":"LOGIN",
  "resource":"AUTH",
  "status":"SUCCESS",
  "ipAddress":"192.168.1.10"
}
```

Database Tables:

- AuditLog
- User

Transaction:

Yes

Search Audit Logs

GET `/api/v1/audit`

Supports:

- User ID
 - Action
 - Date Range
 - Resource
 - Status
 - Pagination
-

Get Audit Log

GET /api/v1/audit/{auditId}

Export Audit Logs

GET /api/v1/audit/export

Supported Formats:

- CSV
 - PDF
-

API-012.4 — User Activity APIs

Track user behavior across the platform.

Login History

GET /api/v1/activity/login-history

Student Activity

GET /api/v1/activity/students/{studentId}

Examples:

- Login
 - Logout
 - Examination Started
 - Examination Submitted
 - Result Viewed
-

Faculty Activity

GET /api/v1/activity/faculty/{facultyId}

Examples:

- Question Created
 - Question Approved
 - Examination Scheduled
 - Result Published
-

Administrator Activity

GET /api/v1/activity/admin

Tracks:

- User Management
 - Configuration Changes
 - Role Changes
 - System Administration
-

API-012.5 — Security Event APIs

Security-related events are tracked separately.

Security Events

GET /api/v1/security/events

Examples:

- Failed Login
- Multiple Failed Attempts
- Password Changed
- Account Locked
- Unauthorized Access
- Token Expired
- Suspicious API Usage

Failed Login Report

GET /api/v1/security/failed-logins

Active Sessions

GET /api/v1/security/sessions

Displays:

- User
 - Login Time
 - IP Address
 - Device
 - Status
-

API-012.6 — System Monitoring APIs

System Health

GET /api/v1/system/health

Returns:

```
{
  "database":"UP",
  "application":"UP",
  "cache":"UP",
  "storage":"UP",
  "status":"HEALTHY"
}
```

Server Metrics

GET /api/v1/system/metrics

Returns:

- CPU Usage
 - Memory Usage
 - Disk Usage
 - Network Usage
-

Application Metrics

GET /api/v1/system/application

Displays:

- Active Users
 - Active Examinations
 - Requests Per Minute
 - Error Rate
 - Average Response Time
-

API-012.7 — Performance Metrics APIs

API Performance

GET /api/v1/system/api-performance

Displays:

- Slowest APIs
 - Average Response Time
 - Request Count
 - Failure Rate
-

Database Metrics

GET /api/v1/system/database

Displays:

- Active Connections
 - Slow Queries
 - Transaction Rate
 - Deadlocks
 - Connection Pool Usage
-

Background Jobs

GET /api/v1/system/jobs

Shows:

- Running Jobs
 - Failed Jobs
 - Scheduled Jobs
 - Queue Size
-

API-012.8 — Validation & Business Rules

Business Rules

- Audit records are immutable.
- Audit records cannot be deleted through application APIs.
- Internal services only can create audit entries.
- Every authentication event is logged.
- Every examination submission is logged.
- Administrative changes require audit entries.
- Health metrics are read-only.
- Performance metrics are refreshed periodically.
- Sensitive data (passwords, tokens, OTPs) must never be stored in audit logs.
- Retention policies must be configurable to meet institutional compliance requirements.

API-012.9 — Performance Considerations

To satisfy project KPIs:

- Use asynchronous audit logging where appropriate.
 - Index `userId`, `action`, `resource`, and `createdAt`.
 - Partition audit tables by date for scalability.
 - Cache health metrics for dashboard use.
 - Aggregate performance statistics periodically.
 - Archive old audit logs according to retention policy.
-

API-012.10 — Security Considerations

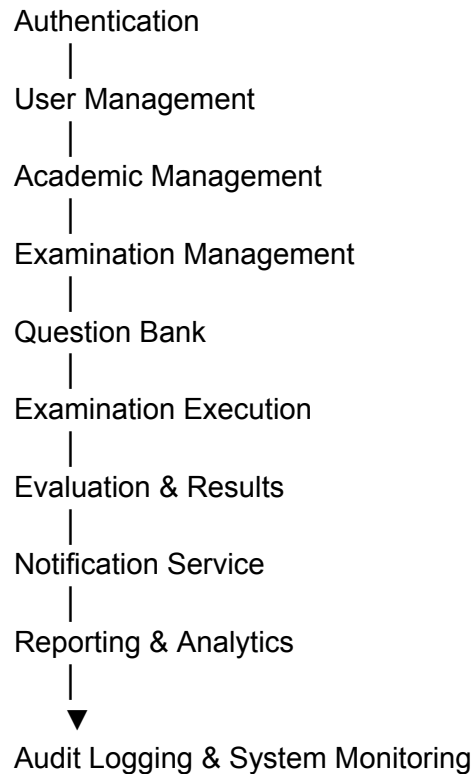
Operation	Student	Faculty	Admin
View Own Login History	✓	✓	✓
View Own Activity	✓	✓	✓
View Audit Logs	✗	✗	✓
Export Audit Logs	✗	✗	✓
View Security Events	✗	✗	✓
View System Health	✗	Limited	✓
View Performance Metrics	✗	✗	✓
Create Audit Record	✗	✗	Internal Services Only

Security Measures:

- Append-only audit storage.
- RBAC-protected audit access.
- Sensitive fields masked before persistence.
- Cryptographic integrity checks (e.g., hash chaining or digital signatures) can be applied to detect tampering.

- All audit access is itself audited.
-

API-012.11 — Integration with Other Modules



Every major module publishes operational and security events to this service for centralized storage and monitoring.

Endpoint Summary

Method	Endpoint	Purpose
POST	/internal/audit	Create audit record (internal)

GET	<code>/api/v1/audit</code>	Search audit logs
GET	<code>/api/v1/audit/{auditId}</code>	View audit record
GET	<code>/api/v1/audit/export</code>	Export audit logs
GET	<code>/api/v1/activity/login-history</code>	Login history
GET	<code>/api/v1/activity/students/{studentId}</code>	Student activity
GET	<code>/api/v1/activity/faculty/{facultyId}</code>	Faculty activity
GET	<code>/api/v1/activity/admin</code>	Administrator activity
GET	<code>/api/v1/security/events</code>	Security events
GET	<code>/api/v1/security/failed-logins</code>	Failed login report
GET	<code>/api/v1/security/sessions</code>	Active sessions
GET	<code>/api/v1/system/health</code>	System health
GET	<code>/api/v1/system/metrics</code>	Server metrics
GET	<code>/api/v1/system/application</code>	Application metrics
GET	<code>/api/v1/system/api-performance</code>	API performance
GET	<code>/api/v1/system/database</code>	Database metrics
GET	<code>/api/v1/system/jobs</code>	Background job metrics

Database Tables Used

Table	Purpose
AuditLog	Immutable audit records
SecurityEvent	Security-related events
LoginHistory	Authentication history

SystemMetric	Periodic infrastructure metrics
ApiRequestLog	API request and response metrics
BackgroundJob	Scheduled and queued job metadata
UserSession	Active user sessions

Core Subject Mapping

Subject	Contribution
Software Engineering	Centralized logging architecture, observability, monitoring services
OOP	Audit, Monitoring, Security Event, and Metrics service classes
DBMS	Immutable audit storage, indexing, partitioning, archival strategies
Computer Networks	API request tracing, network metrics, distributed service communication
Operating Systems	Process monitoring, CPU/memory utilization, concurrent log collection
DSA	Efficient log indexing, filtering, aggregation, and search algorithms

Engineering Benefits

Engineering Goal	API Contribution
Security	Complete traceability of user and system actions
Compliance	Immutable audit records and configurable retention
Reliability	Continuous health and performance monitoring
Performance	Indexed searches and asynchronous logging

Maintainability	Centralized observability across all services
Troubleshooting	Detailed activity history, metrics, and diagnostics

Deliverables Produced

API-012 produces:

-  Audit Logging API Specification
 -  User Activity Tracking APIs
 -  Security Event APIs
 -  System Health Monitoring APIs
 -  Performance Metrics APIs
 -  Audit Lifecycle Model
 -  Validation & Business Rules
 -  Security & Authorization Matrix
 -  Endpoint Catalog
 -  Observability & Monitoring Framework
-

API-012 Status: Complete

API-012 completes the **Audit Logging & System Monitoring domain**, providing a secure, centralized, and immutable record of all significant user, administrative, and system activities. It also introduces operational observability through health checks, performance metrics, security event tracking, and monitoring dashboards, making the Online Examination Platform suitable for enterprise deployment, compliance, and long-term operational maintenance.

Next: API-013 – File & Document Management, which will define APIs for handling profile photos, question imports, answer-sheet attachments (where applicable), report exports, and other documents used throughout the platform.

Excellent. We now move to the **File & Document Management Module**.

This is the last major **Enterprise Infrastructure Module** before System Administration (API-014) and AI Proctoring (API-015).

Unlike API-011 (Reporting) and API-012 (Audit), this module manages **all digital assets** used by the Online Examination Platform. Rather than storing files directly in the database, the

platform stores **metadata in the database** while the actual files are stored in a dedicated object/file storage system (local storage, AWS S3, Azure Blob, MinIO, etc.).

API-013 — File & Document Management APIs

Status: API Design Phase

Objective

Design a secure, scalable File & Document Management service responsible for handling the upload, storage, retrieval, versioning, and deletion of files used across the Online Examination Platform.

This module manages:

- Profile Photos
- Faculty Documents
- Student Documents
- Question Bank Import Files
- Bulk Student Import Files
- Examination Attachments
- Result PDFs
- Report Exports
- System Documents
- File Versioning
- File Metadata

The service acts as a centralized file repository for all platform modules.

Design Principles Applied

-  Principle 1 – Integration of Core Computer Science Subjects

-  Principle 2 – Every Design Decision Must Answer Two Questions
 -  Principle 7 – KPIs Drive Architecture
 -  Principle 10 – No Accidental Decisions
 -  Principle 11 – Engineering over Convenience
 -  Principle 12 – Reliability Before Optimization
-

API-013 Structure

API-013

—	API-013.1 Module Overview
—	API-013.2 File Lifecycle
—	API-013.3 File Upload APIs
—	API-013.4 File Retrieval APIs
—	API-013.5 File Management APIs
—	API-013.6 Bulk Import APIs
—	API-013.7 Export & Download APIs
—	API-013.8 Validation & Business Rules
—	API-013.9 Performance Considerations
—	API-013.10 Security Considerations
—	API-013.11 Integration with Other Modules

API-013.1 — Module Overview

The File Management Service centralizes all document operations.

Responsibilities:

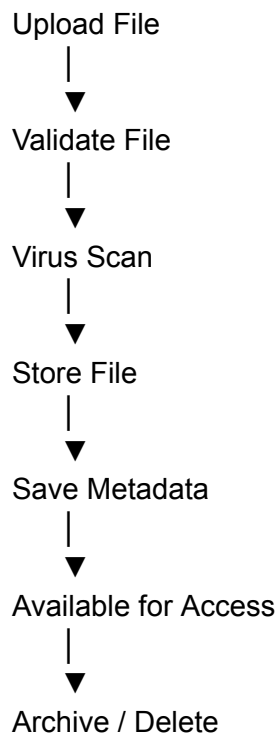
- Upload files
- Download files
- Delete files
- Update metadata
- Version files
- Import bulk data
- Export reports
- Validate uploaded files
- Generate secure download links

Database stores only:

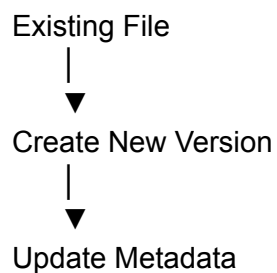
- File metadata
- Owner
- File path
- Storage identifier
- Version information

Actual files remain in external storage.

API-013.2 — File Lifecycle



If an updated version is uploaded:





Retain Previous Version

API-013.3 — File Upload APIs

Upload File

Property	Value
Method	POST
Endpoint	<code>/api/v1/files</code>
Authentication	Required

Request

Multipart/Form-Data

file=<binary>
fileType=PROFILE_PHOTO
ownerId=105

Supported File Types:

- PROFILE_PHOTO
- STUDENT_DOCUMENT
- FACULTY_DOCUMENT
- QUESTION_IMPORT
- RESULT_PDF
- REPORT_EXPORT
- EXAM_ATTACHMENT

Database Tables:

- FileMetadata
- AuditLog

Transaction:

Yes

Upload New Version

POST /api/v1/files/{fileId}/versions

Creates a new version while preserving previous versions.

Bulk Upload

POST /api/v1/files/bulk

Supports:

- ZIP
 - CSV
 - Excel (XLSX)
-

API-013.4 — File Retrieval APIs

Download File

GET /api/v1/files/{fileId}

Returns a secure download response or a time-limited signed URL.

View File Metadata

GET /api/v1/files/{fileId}/metadata

Returns:

- File Name
 - Size
 - Version
 - Upload Date
 - Owner
 - MIME Type
-

List Files

GET /api/v1/files

Supports:

- Pagination
 - Filtering
 - File Type
 - Owner
 - Upload Date
-

File Versions

GET /api/v1/files/{fileId}/versions

API-013.5 — File Management APIs

Update Metadata

PATCH /api/v1/files/{fileId}

Updates metadata only (e.g., display name or tags), not file content.

Archive File

PATCH /api/v1/files/{fileId}/archive

Delete File

DELETE /api/v1/files/{fileId}

Implements soft deletion where institutional policy requires retention.

Restore File

PATCH /api/v1/files/{fileId}/restore

API-013.6 — Bulk Import APIs

Used for importing structured data.

Import Students

POST /api/v1/import/students

Supported:

- CSV
 - XLSX
-

Import Questions

POST /api/v1/import/questions

Supports validation before insertion into the Question Bank.

Import Faculty

POST /api/v1/import/faculty

Returns:

- Imported Count
 - Failed Records
 - Validation Errors
-

API-013.7 — Export & Download APIs

Export Results

GET /api/v1/export/results

Export Reports

GET /api/v1/export/reports

Export Student List

GET /api/v1/export/students

Export Question Bank

GET /api/v1/export/questions

Supported Formats:

- PDF

- CSV
 - XLSX
 - ZIP (for multi-file exports)
-

API-013.8 — Validation & Business Rules

Business Rules

- Only supported file types may be uploaded.
 - Maximum upload size is configurable by administrators.
 - Uploaded files must pass validation and malware scanning before storage.
 - Duplicate file versions increment the version number instead of overwriting.
 - Deleted files follow the configured retention policy.
 - Download permissions are enforced through RBAC.
 - Import operations are transactional; invalid records are reported without corrupting existing data.
 - Every upload, download, delete, restore, and export action is recorded in [AuditLog](#).
-

API-013.9 — Performance Considerations

To satisfy project KPIs:

- Stream large uploads and downloads.
 - Store binary files outside the relational database.
 - Use content hashing to detect duplicate uploads.
 - Compress exports where appropriate.
 - Support resumable uploads for large files.
 - Cache metadata for frequently accessed files.
 - Process imports asynchronously for large datasets.
-

API-013.10 — Security Considerations

Operation

Student

Faculty

Admin

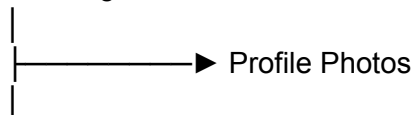
Upload Own Documents	✓	✓	✓
Download Own Files	✓	✓	✓
View File Metadata	Own Files	Own Files	✓
Import Students	✗	✗	✓
Import Faculty	✗	✗	✓
Import Questions	✗	Assigned Faculty	✓
Export Reports	✗	Assigned Reports	✓
Delete Files	Own (Restricted)	Own (Restricted)	✓
Restore Files	✗	✗	✓

Security Measures:

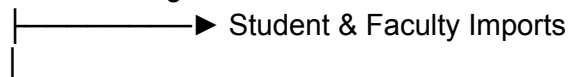
- RBAC-based authorization.
- Secure object storage with private access.
- Time-limited signed download URLs.
- MIME type and extension validation.
- File size limits.
- Malware scanning before persistence.
- File checksum validation to ensure integrity.
- Audit logging for all file operations.

API-013.11 — Integration with Other Modules

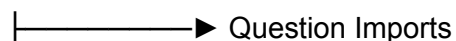
User Management

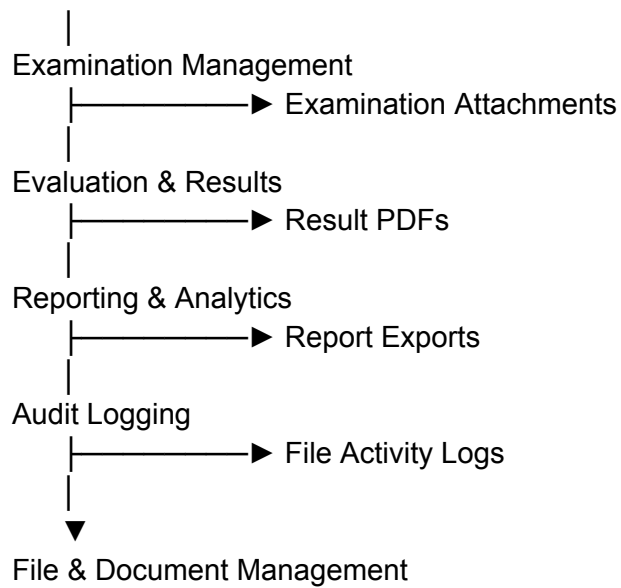


Academic Management



Question Bank





Endpoint Summary

Method	Endpoint	Purpose
POST	<code>/api/v1/files</code>	Upload file
POST	<code>/api/v1/files/{fileId}/versions</code>	Upload new version
POST	<code>/api/v1/files/bulk</code>	Bulk upload
GET	<code>/api/v1/files/{fileId}</code>	Download file
GET	<code>/api/v1/files/{fileId}/metadata</code>	View metadata
GET	<code>/api/v1/files</code>	List files
GET	<code>/api/v1/files/{fileId}/versions</code>	File versions
PATCH	<code>/api/v1/files/{fileId}</code>	Update metadata
PATCH	<code>/api/v1/files/{fileId}/archive</code>	Archive file

PATCH	<code>/api/v1/files/{fileId}/restore</code>	Restore file
DELETE	<code>/api/v1/files/{fileId}</code>	Delete file
POST	<code>/api/v1/import/students</code>	Import students
POST	<code>/api/v1/import/faculty</code>	Import faculty
POST	<code>/api/v1/import/questions</code>	Import questions
GET	<code>/api/v1/export/results</code>	Export results
GET	<code>/api/v1/export/reports</code>	Export reports
GET	<code>/api/v1/export/students</code>	Export students
GET	<code>/api/v1/export/questions</code>	Export question bank

Database Tables Used

Table	Purpose
FileMetadata	File metadata and storage reference
FileVersion	Version history
ImportJob	Bulk import tracking
ExportJob	Export request tracking
User	File ownership
AuditLog	File activity logging

Core Subject Mapping

Subject	Contribution
---------	--------------

Software Engineering	Centralized file service, version control, modular import/export workflows
OOP	File, Version, Import, Export, and Storage service classes
DBMS	Metadata persistence, transactional import jobs, indexing
Computer Networks	Multipart uploads, streaming downloads, secure file transfer
Operating Systems	File I/O management, background processing, concurrent uploads
DSA	File indexing, duplicate detection using hashing, efficient metadata search

Engineering Benefits

Engineering Goal	API Contribution
Scalability	External object storage and streaming transfers
Reliability	Versioning, transactional imports, resumable uploads
Security	Signed URLs, malware scanning, RBAC, integrity checks
Performance	Metadata caching and asynchronous processing
Maintainability	Centralized file management service
Extensibility	Easy integration with cloud storage providers and new file types

Deliverables Produced

API-013 produces:

-  File & Document Management API Specification
-  File Upload & Download APIs
-  File Versioning APIs
-  Bulk Import APIs
-  Export APIs

-  File Lifecycle Model
 -  Validation & Business Rules
 -  Security & Authorization Matrix
 -  Endpoint Catalog
 -  Centralized File Management Architecture
-

API-013 Status: Complete

API-013 completes the **File & Document Management domain**, providing secure and scalable management of all digital assets used throughout the Online Examination Platform. By separating binary file storage from metadata management, supporting versioning, bulk import/export, and enterprise-grade security controls, it establishes a robust document infrastructure for the entire system.

Remaining API Modules

Only **two API specifications** remain:

- **API-014 – System Administration & Platform Configuration**
- **API-015 – AI-Based Proctoring & Exam Integrity** (Final module, as planned)

These will complete the entire API design phase of the Online Examination Platform.

Excellent. We are now at the **second last API module** of the entire Online Examination Platform.

Unlike previous modules that manage examinations, users, or reports, **API-014** is responsible for the **configuration, administration, maintenance, and operational control** of the platform. It gives administrators the ability to manage system-wide settings without modifying application code.

After this, only **API-015 (AI Proctoring & Exam Integrity)** will remain.

API-014 — System Administration & Platform Configuration APIs

Status: API Design Phase

Objective

Design a centralized System Administration & Platform Configuration service responsible for managing platform settings, institutional configurations, security policies, maintenance operations, and operational controls.

This module provides:

- System Configuration
- Institution Configuration
- Role & Permission Administration
- Academic Calendar Configuration
- Examination Policies
- Password & Security Policies
- Feature Toggle Management
- Maintenance Mode
- Backup & Restore Operations
- License & Version Information

This module is accessible only to authorized system administrators and super administrators.

Design Principles Applied

-  Principle 1 – Integration of Core Computer Science Subjects
 -  Principle 2 – Every Design Decision Must Answer Two Questions
 -  Principle 7 – KPIs Drive Architecture
 -  Principle 10 – No Accidental Decisions
 -  Principle 11 – Engineering over Convenience
 -  Principle 12 – Reliability Before Optimization
-

API-014 Structure

API-014

|
|— API-014.1 Module Overview

—	API-014.2 Administration Lifecycle
—	API-014.3 System Configuration APIs
—	API-014.4 Institution Configuration APIs
—	API-014.5 Security Policy APIs
—	API-014.6 Feature Management APIs
—	API-014.7 Backup & Maintenance APIs
—	API-014.8 Validation & Business Rules
—	API-014.9 Performance Considerations
—	API-014.10 Security Considerations
—	API-014.11 Integration with Other Modules

API-014.1 — Module Overview

This module centralizes all administrative controls for the Online Examination Platform.

Responsibilities:

- Configure global system settings
- Configure institution information
- Configure examination policies
- Configure academic calendar
- Configure password policies
- Configure security policies
- Enable/disable platform features
- Manage maintenance mode
- Schedule backups
- Restore system backups
- Manage application version information

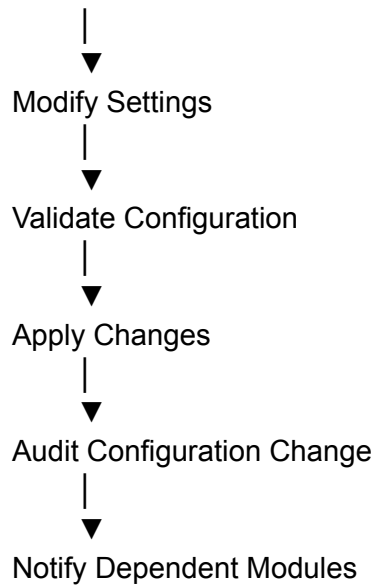
The objective is to ensure that operational changes can be made through configuration rather than application code.

API-014.2 — Administration Lifecycle

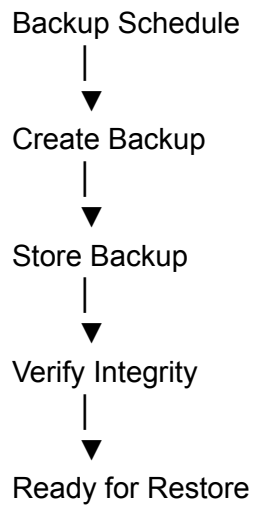
Administrator Login



Load Current Configuration



For backup operations:



API-014.3 — System Configuration APIs

Get System Configuration

Property	Value
Method	GET

Endpoint `/api/v1/admin/system-config`

Authentication Required

Authorization Super Admin

Returns:

```
{
  "systemName": "Online Examination Platform",
  "timezone": "Asia/Kolkata",
  "defaultLanguage": "en",
  "maintenanceMode": false,
  "sessionTimeout": 30
}
```

Update System Configuration

PUT `/api/v1/admin/system-config`

Example

```
{
  "timezone": "Asia/Kolkata",
  "sessionTimeout": 45
}
```

View System Information

GET `/api/v1/admin/system-info`

Returns:

- Application Version
- Build Number
- Database Version
- Deployment Date
- Uptime

API-014.4 — Institution Configuration APIs

Institution Profile

GET /api/v1/admin/institution

Update Institution Profile

PUT /api/v1/admin/institution

Fields include:

- Institution Name
 - Logo
 - Address
 - Contact Information
 - Academic Year
 - Semester Configuration
-

Academic Calendar

GET /api/v1/admin/academic-calendar

Update Academic Calendar

PUT /api/v1/admin/academic-calendar

Supports:

- Semester Dates
- Holidays
- Examination Windows
- Result Publication Windows

API-014.5 — Security Policy APIs

Password Policy

GET /api/v1/admin/security/password-policy

Returns:

- Minimum Length
- Complexity Rules
- Expiration Period
- Password History Limit

Update Password Policy

PUT /api/v1/admin/security/password-policy

Session Policy

GET /api/v1/admin/security/session-policy

Configures:

- Session Timeout
- Concurrent Session Limit
- Idle Timeout
- Token Expiration

Update Session Policy

PUT /api/v1/admin/security/session-policy

API-014.6 — Feature Management APIs

Feature List

GET /api/v1/admin/features

Examples:

- Online Examination
 - Notifications
 - Reports
 - AI Proctoring
 - File Management
 - Bulk Import
-

Enable Feature

PATCH /api/v1/admin/features/{featureId}/enable

Disable Feature

PATCH /api/v1/admin/features/{featureId}/disable

Maintenance Mode

PATCH /api/v1/admin/maintenance

Example

```
{  
  "enabled":true,  
  "message":"System maintenance from 2:00 AM to 3:00 AM."  
}
```

API-014.7 — Backup & Maintenance APIs

Create Backup

POST /api/v1/admin/backup

Creates:

- Database Backup
 - Configuration Backup
 - Metadata Backup
-

Backup History

GET /api/v1/admin/backup

Restore Backup

POST /api/v1/admin/restore/{backupId}

Database Maintenance

POST /api/v1/admin/database/maintenance

Operations:

- Optimize Tables
 - Rebuild Indexes
 - Cleanup Logs
 - Archive Historical Data
-

API-014.8 — Validation & Business Rules

Business Rules

- Only Super Administrators can modify system configuration.
 - Maintenance mode prevents new examination sessions but allows administrators to perform maintenance tasks.
 - Security policy updates affect new authentication sessions immediately.
 - Backup files must pass integrity verification before restoration.
 - Feature toggles must not disable mandatory platform components.
 - Every configuration change is recorded in [AuditLog](#).
 - Restore operations require explicit administrator confirmation.
-

API-014.9 — Performance Considerations

To satisfy project KPIs:

- Cache frequently accessed configuration values.
 - Reload configuration dynamically without restarting the application where possible.
 - Schedule backup jobs during low-traffic periods.
 - Use incremental backups to reduce storage and execution time.
 - Archive historical operational data to maintain database performance.
-

API-014.10 — Security Considerations

Operation	Student	Faculty	Admin	Super Admin
View System Information	✗	✗	Limited	✓
Modify Configuration	✗	✗	✗	✓
Manage Institution Profile	✗	✗	Limited	✓
Update Security Policies	✗	✗	✗	✓
Enable/Disable Features	✗	✗	✗	✓
Backup Database	✗	✗	Limited	✓
Restore Database	✗	✗	✗	✓

Maintenance Mode



Security Measures:

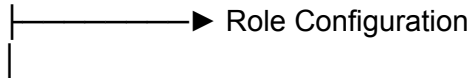
- Multi-level RBAC with Super Administrator privileges.
- Optional Multi-Factor Authentication (MFA) for administrative actions.
- Full audit trail for every configuration and maintenance operation.
- Restore operations require confirmation and can optionally require dual authorization.
- Sensitive configuration values (e.g., API keys, secrets) are encrypted at rest.

API-014.11 — Integration with Other Modules

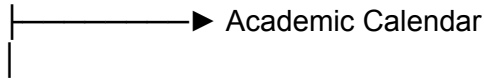
Authentication



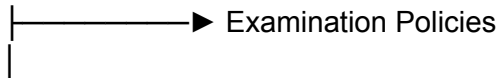
User Management



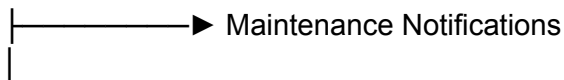
Academic Management



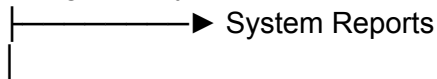
Examination Management



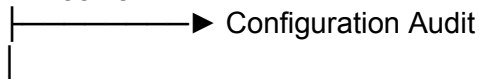
Notification Service



Reporting & Analytics



Audit Logging



System Administration

Endpoint Summary

Method	Endpoint	Purpose
GET	<code>/api/v1/admin/system-config</code>	View system configuration
PUT	<code>/api/v1/admin/system-config</code>	Update system configuration
GET	<code>/api/v1/admin/system-info</code>	View system information
GET	<code>/api/v1/admin/institution</code>	View institution profile
PUT	<code>/api/v1/admin/institution</code>	Update institution profile
GET	<code>/api/v1/admin/academic-calendar</code>	View academic calendar
PUT	<code>/api/v1/admin/academic-calendar</code>	Update academic calendar
GET	<code>/api/v1/admin/security/password-policy</code>	View password policy
PUT	<code>/api/v1/admin/security/password-policy</code>	Update password policy
GET	<code>/api/v1/admin/security/session-policy</code>	View session policy
PUT	<code>/api/v1/admin/security/session-policy</code>	Update session policy
GET	<code>/api/v1/admin/features</code>	List features
PATCH	<code>/api/v1/admin/features/{featureId}/enable</code>	Enable feature
PATCH	<code>/api/v1/admin/features/{featureId}/disable</code>	Disable feature
PATCH	<code>/api/v1/admin/maintenance</code>	Toggle maintenance mode
POST	<code>/api/v1/admin/backup</code>	Create backup
GET	<code>/api/v1/admin/backup</code>	Backup history
POST	<code>/api/v1/admin/restore/{backupId}</code>	Restore backup
POST	<code>/api/v1/admin/database/maintenance</code>	Database maintenance

Database Tables Used

Table	Purpose
SystemConfiguration	Global application settings
InstitutionConfiguration	Institution profile and academic settings
SecurityPolicy	Password and session policies
FeatureToggle	Enable/disable platform features
BackupHistory	Backup metadata and status
MaintenanceLog	Maintenance operations
AuditLog	Administrative activity tracking

Core Subject Mapping

Subject	Contribution
Software Engineering	Configuration management, feature toggles, administrative workflows
OOP	Configuration, Policy, Backup, and Feature service classes
DBMS	Configuration persistence, backup metadata, transaction-safe restore operations
Computer Networks	Administrative REST APIs and secure communication
Operating Systems	Backup scheduling, maintenance jobs, process management
DSA	Efficient configuration caching, scheduling, and maintenance task management

Engineering Benefits

Engineering Goal	API Contribution
Maintainability	Runtime configuration without code changes
Reliability	Backup, restore, and maintenance capabilities
Security	Centralized policy management and administrative controls
Scalability	Dynamic configuration and feature toggles
Operational Excellence	Centralized platform administration
Compliance	Fully audited configuration changes and maintenance operations

Deliverables Produced

API-014 produces:

-  System Administration API Specification
 -  System & Institution Configuration APIs
 -  Security Policy APIs
 -  Feature Toggle Management APIs
 -  Backup & Restore APIs
 -  Maintenance Management APIs
 -  Administration Lifecycle Model
 -  Validation & Business Rules
 -  Security & Authorization Matrix
 -  Endpoint Catalog
-

API-014 Status: Complete

API-014 completes the **System Administration & Platform Configuration domain**, providing comprehensive administrative control over the Online Examination Platform. It enables secure runtime configuration, institutional management, security policy enforcement, feature management, backup and recovery, and maintenance operations while ensuring every administrative action is auditable and governed by strict authorization controls.

API Design Progress

At this stage, **14 out of 15 API modules are complete:**

-  API-001 — API Architecture & Standards
-  API-002 — Common Response & Error Handling
-  API-003 — Authentication & Authorization
-  API-004 — User Management
-  API-005 — Academic Management
-  API-006 — Examination Management
-  API-007 — Question Bank
-  API-008 — Examination Execution
-  API-009 — Evaluation & Result Management
-  API-010 — Notification & Communication
-  API-011 — Reporting & Analytics
-  API-012 — Audit Logging & System Monitoring
-  API-013 — File & Document Management
-  API-014 — System Administration & Platform Configuration

Only **one API module remains:**

API-015 – AI-Based Proctoring & Exam Integrity, which will serve as the capstone of the API design phase and integrate seamlessly with the examination execution workflow while remaining modular enough to be enabled or disabled through the feature management capabilities introduced in API-014.

That's actually a better choice for your project.

Since earlier you decided **not to implement AI/ML-based proctoring** and instead implement a **lightweight exam integrity system**, API-015 should reflect the actual scope of your project rather than introducing features you won't build.

This also aligns better with your college SRS because it focuses on **secure examination execution** rather than advanced AI surveillance.

API-015 — Exam Integrity & Suspicious Activity Monitoring APIs

Status: Final API Design Phase

Objective

Design a lightweight Exam Integrity service responsible for monitoring candidate behavior during an online examination and recording suspicious activities that may indicate malpractice.

Unlike AI-based proctoring systems, this module **does not use machine learning, webcam analysis, facial recognition, or eye tracking**.

Instead, it monitors browser-level and client-side events to help invigilators identify suspicious examination behavior.

This module provides:

- Browser Tab Switching Detection
 - Window Focus Monitoring
 - Full-Screen Exit Detection
 - Keyboard Shortcut Detection
 - Suspicious Activity Logging
 - Live Monitoring Dashboard
 - Integrity Report Generation
 - Warning Management
-

Design Principles Applied

-  Principle 1 – Integration of Core Computer Science Subjects
 -  Principle 2 – Every Design Decision Must Answer Two Questions
 -  Principle 7 – KPIs Drive Architecture
 -  Principle 10 – No Accidental Decisions
 -  Principle 11 – Engineering over Convenience
 -  Principle 12 – Reliability Before Optimization
-

API-015 Structure

API-015

- |
 - API-015.1 Module Overview
 - API-015.2 Monitoring Lifecycle
 - API-015.3 Suspicious Activity APIs
 - API-015.4 Warning Management APIs
 - API-015.5 Live Monitoring APIs
 - API-015.6 Integrity Report APIs
 - API-015.7 Validation & Business Rules
 - API-015.8 Performance Considerations
 - API-015.9 Security Considerations
 - API-015.10 Integration with Other Modules
-

API-015.1 — Module Overview

The Exam Integrity module continuously monitors browser events during an examination.

Responsibilities:

- Detect tab switching
- Detect browser window changes
- Detect full-screen exit
- Detect browser minimization
- Detect prohibited keyboard shortcuts
- Count suspicious activities
- Record timestamps
- Notify invigilators
- Generate integrity reports

No video, audio, or biometric data is collected.

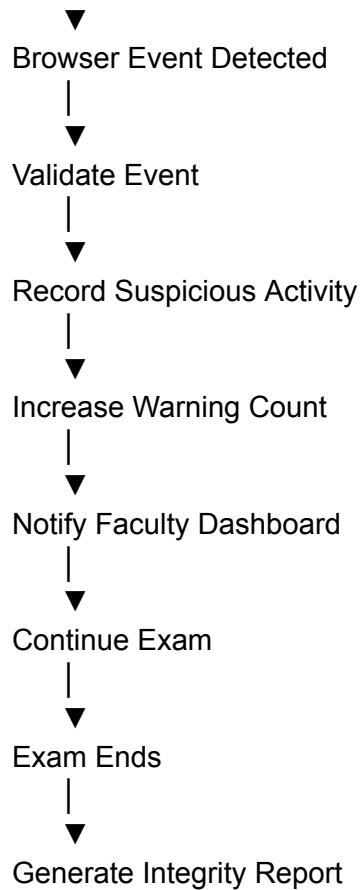
API-015.2 — Monitoring Lifecycle

Exam Starts



Monitoring Enabled





API-015.3 — Suspicious Activity APIs

Report Suspicious Activity

Property	Value
Method	POST
Endpoint	<code>/api/v1/integrity/events</code>
Authentication	Required
Authorization	Authenticated Exam Session

Request

```
{  
  "attemptId": 101,  
  "eventType": "TAB_SWITCH",  
  "timestamp": "2026-08-04T10:15:32Z"  
}
```

Supported Event Types

- TAB_SWITCH
- WINDOW_BLUR
- WINDOW_MINIMIZE
- FULLSCREEN_EXIT
- COPY_ATTEMPT
- PASTE_ATTEMPT
- KEYBOARD_SHORTCUT
- RIGHT_CLICK

Database Tables

- SuspiciousActivity
 - ExamAttempt
 - AuditLog
-

Get Attempt Activity

GET /api/v1/integrity/attempts/{attemptId}

Returns all suspicious events recorded during the examination.

Get Student Activity

GET /api/v1/integrity/students/{studentId}

Shows integrity history across examinations.

API-015.4 — Warning Management APIs

Issue Warning

POST /api/v1/integrity/warnings

Example

```
{
  "attemptId":101,
  "warningNumber":2,
  "message":"Multiple tab switches detected."
}
```

Warning Count

GET /api/v1/integrity/attempts/{attemptId}/warnings

Returns:

- Total Warnings
 - Latest Warning
 - Current Status
-

Reset Warning (Admin Only)

PATCH /api/v1/integrity/warnings/{warningId}/reset

API-015.5 — Live Monitoring APIs

Live Examination Monitoring

GET /api/v1/integrity/live/{examId}

Displays:

- Active Students
 - Current Warnings
 - Latest Events
 - Examination Status
-

Live Alerts

GET /api/v1/integrity/live/alerts

Shows real-time suspicious activities.

API-015.6 — Integrity Report APIs

Attempt Integrity Report

GET /api/v1/integrity/report/{attemptId}

Contains:

- Total Tab Switches
 - Full-screen Exits
 - Copy/Paste Attempts
 - Warning Count
 - Timeline of Events
-

Examination Integrity Report

GET /api/v1/integrity/report/exam/{examId}

Displays:

- Students Flagged
- Total Violations

- Most Common Violation
 - Warning Distribution
-

API-015.7 — Validation & Business Rules

Business Rules

- Monitoring starts only after the examination begins.
 - Events are accepted only for active examination sessions.
 - Every suspicious event is timestamped.
 - Multiple identical events occurring within a short configurable interval may be merged to avoid duplicate logging.
 - Warnings are generated based on configurable thresholds.
 - Monitoring stops automatically when the examination ends.
 - All integrity events are stored in immutable audit records.
-

API-015.8 — Performance Considerations

To satisfy project KPIs:

- Use lightweight browser event listeners.
 - Batch event uploads where appropriate.
 - Index `attemptId`, `studentId`, and `eventType`.
 - Process alerts asynchronously for faculty dashboards.
 - Minimize client-side overhead to avoid affecting the examination experience.
-

API-015.9 — Security Considerations

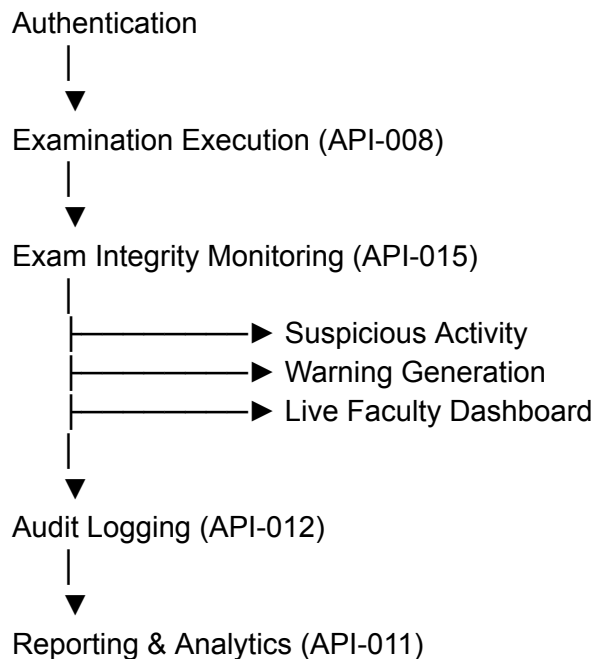
Operation	Student	Faculty	Admin
Report Event	✓ (Client)	✗	✗
View Own Integrity Report	✗	✗	Optional

View Live Monitoring	✗	Assigned Exams	✓
View Integrity Reports	✗	Assigned Exams	✓
Reset Warning	✗	✗	✓

Security Measures:

- Events accepted only from authenticated examination sessions.
- Server-side validation to reduce spoofing of client events.
- TLS encryption for event transmission.
- All integrity events logged in [AuditLog](#).
- Faculty can only view examinations assigned to them.

API-015.10 — Integration with Other Modules



Endpoint Summary

Method	Endpoint	Purpose
POST	/api/v1/integrity/events	Report suspicious event
GET	/api/v1/integrity/attempts/{attemptId}	Attempt activity
GET	/api/v1/integrity/students/{studentId}	Student integrity history
POST	/api/v1/integrity/warnings	Issue warning
GET	/api/v1/integrity/attempts/{attemptId}/warnings	Warning count
PATCH	/api/v1/integrity/warnings/{warningId}/reset	Reset warning
GET	/api/v1/integrity/live/{examId}	Live monitoring
GET	/api/v1/integrity/live/alerts	Live alerts
GET	/api/v1/integrity/report/{attemptId}	Attempt integrity report
GET	/api/v1/integrity/report/exam/{examId}	Examination integrity report

Database Tables Used

Table	Purpose
SuspiciousActivity	Stores browser integrity events
Warning	Stores warning records issued during exams
ExamAttempt	Links integrity events to examination attempts
AuditLog	Permanent record of integrity-related actions

Core Subject Mapping

Subject	Contribution
Software Engineering	Modular integrity monitoring service integrated with exam execution
OOP	Event, Warning, IntegrityReport, and Monitoring service classes
DBMS	Persistent event logging, indexed queries, transactional warning storage
Computer Networks	Secure transmission of browser events via REST APIs
Operating Systems	Event-driven client monitoring, concurrent processing of live events
DSA	Efficient event filtering, threshold counting, timeline generation, and alert processing

Engineering Benefits

Engineering Goal	API Contribution
Academic Integrity	Detects common browser-based malpractice attempts
Performance	Lightweight client-side monitoring with minimal overhead
Security	Authenticated event reporting and immutable audit logs
Scalability	Supports large concurrent examinations through asynchronous event handling
Maintainability	Modular monitoring service independent of examination logic
Transparency	Provides detailed integrity reports for post-exam review

Deliverables Produced

API-015 produces:

-  Exam Integrity Monitoring API Specification
-  Suspicious Activity Detection APIs

-  Warning Management APIs
 -  Live Monitoring APIs
 -  Integrity Report APIs
 -  Monitoring Lifecycle Model
 -  Validation & Business Rules
 -  Security & Authorization Matrix
 -  Endpoint Catalog
-

API-015 Status: Complete

This completes the **API Design Phase** of your Online Examination Platform.

You now have a complete set of **15 API design documents** covering the entire platform—from authentication and examination management to reporting, administration, and lightweight exam integrity monitoring. The final module intentionally avoids AI-based proctoring and instead implements practical browser-event monitoring (tab switching, focus changes, full-screen exits, and related suspicious activities), making it well-suited for your project's scope while remaining modular and extensible for future enhancements.

FrontendDesign

Excellent. We have now reached one of the most important LLD documents.

So far we have defined:

-  **LLD-001**: Where every file will be stored.
-  **LLD-002**: How the backend is organized.

Now we will design **the complete frontend architecture**, which will become the blueprint for every React component we build.

LLD-003 — Frontend Architecture & Component Design

Status: Low-Level Design Phase

Objective

Design a scalable, modular, and maintainable React frontend for the Online Examination Platform that supports multiple user roles, reusable UI components, efficient state management, and seamless integration with the backend APIs.

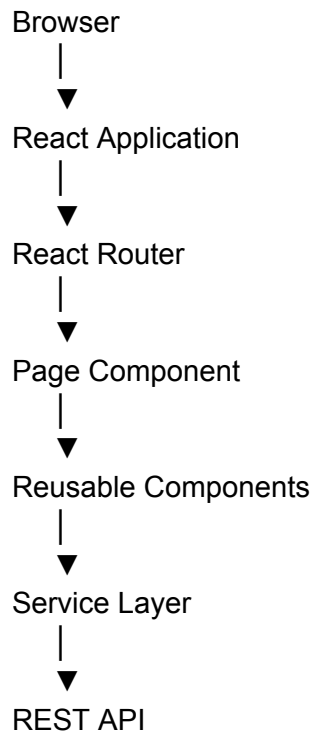
This document defines:

- Frontend architecture
 - Page hierarchy
 - Dashboard layouts
 - Component organization
 - Routing strategy
 - State management
 - API integration
 - Form validation
 - UI interaction flow
 - Role-based navigation
-

Design Principles

- Component-Based Architecture
 - Atomic Design (where appropriate)
 - Separation of Concerns
 - Single Responsibility Principle
 - Reusability
 - Responsive Design
 - Accessibility
 - Lazy Loading
 - Feature-Based Organization
-

Frontend Architecture Overview



Supporting layers:

Context API
Hooks
Validation
Utilities

Constants
Assets

Frontend Folder Organization

```
src/
├── assets/
├── components/
├── pages/
├── layouts/
├── routes/
├── services/
├── hooks/
├── context/
├── utils/
├── constants/
├── validations/
├── styles/
├── App.jsx
└── main.jsx
```

Frontend Architecture Layers

```
UI Layer
  |
  ▼
Pages
  |
  ▼
Reusable Components
  |
  ▼
Hooks / Context
  |
  ▼
Service Layer
  |
```

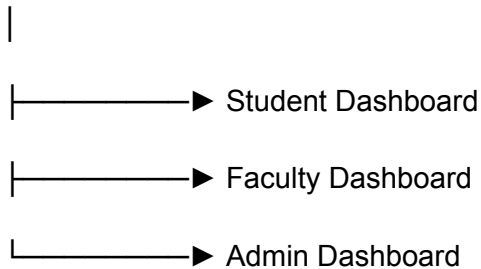
▼
Backend API

Business logic should never be written directly inside UI components.

Layout Design

The system contains **three independent dashboards**.

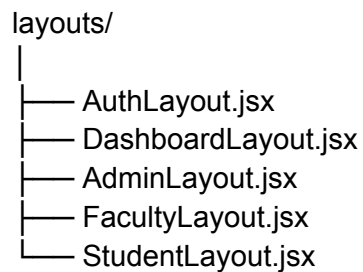
Login



Each dashboard has:

- Sidebar
 - Top Navigation
 - Main Content
 - Footer
-

Shared Layout Components



Responsibilities:

- Navigation
 - Sidebar
 - Header
 - Footer
 - Route Outlet
-

Page Hierarchy

Authentication Pages

pages/auth/

LoginPage

ForgotPasswordPage

ResetPasswordPage

UnauthorizedPage

Student Pages

pages/student/

Dashboard

Profile

Registered Exams

Exam Instructions

Take Exam

Exam History

Results

Notifications

Settings

Faculty Pages

pages/faculty/

Dashboard

Manage Questions

Question Categories

Create Exam

Schedule Exam

Evaluate Answers

Results

Reports

Notifications

Admin Pages

pages/admin/

Dashboard

Users

Roles

Departments

Subjects

[Academic Calendar](#)

[Examinations](#)

[Reports](#)

[System Settings](#)

[Audit Logs](#)

[Integrity Dashboard](#)

Shared Pages

[pages/common/](#)

[Profile](#)

[Change Password](#)

[Notifications](#)

[Help](#)

[About](#)

[404](#)

[500](#)

Component Library

The project uses reusable UI components.

[components/](#)

[common/](#)

forms/

tables/

cards/

charts/

modals/

navigation/

notifications/

integrity/

Common Components

Button

Input

TextArea

Select

Checkbox

RadioButton

DatePicker

TimePicker

SearchBar

Loader

Spinner

Badge

Tooltip

Pagination

Breadcrumb

Reusable across all modules.

Form Components

LoginForm

StudentForm

FacultyForm

ExamForm

QuestionForm

SubjectForm

DepartmentForm

Each form:

- Client validation
 - Error handling
 - API integration
-

Table Components

UserTable

ExamTable

QuestionTable

ResultTable

DepartmentTable

SubjectTable

Supports:

- Pagination
 - Sorting
 - Filtering
 - Search
 - Export
-

Card Components

DashboardCard

StatisticCard

ExamCard

QuestionCard

NotificationCard

Chart Components

BarChart

LineChart

PieChart

AttendanceChart

ResultAnalyticsChart

Used in reports and dashboards.

Modal Components

ConfirmationModal

DeleteModal

EditModal

WarningModal

SuccessModal

ErrorModal

Navigation Components

Sidebar

TopNavbar

MenuItem

ProfileMenu

Breadcrumb

Notification Components

NotificationPanel

NotificationItem

Toast

AlertBanner

Integrity Components

Because we are **not implementing AI proctoring**, only browser-event monitoring components are needed.

WarningCounter

IntegrityStatus

SuspiciousActivityLog

ExamIntegrityBanner

Routing Design

/

↓

Login

↓

Role Verification

↓

Dashboard

↓

Protected Routes

AdminRoute

FacultyRoute

StudentRoute

Unauthorized users are redirected.

State Management

Use **React Context API**.

Contexts:

AuthContext

UserContext

ThemeContext

NotificationContext

ExamContext

Responsibilities:

- Logged-in user
 - JWT token
 - Selected exam
 - Theme
 - Notifications
-

Custom Hooks

useAuth()

useApi()

usePagination()

useNotification()

useExamTimer()

useIntegrityMonitor()

Example:

useExamTimer()

↓

Countdown Timer

↓

Auto Submit

↓

API Call

API Service Layer

All backend communication is centralized.

services/

authService.js

userService.js

examService.js

questionService.js

resultService.js

notificationService.js

integrityService.js

Components never call APIs directly.

Validation Layer

validations/

authValidation.js

studentValidation.js

facultyValidation.js

examValidation.js

questionValidation.js

Responsibilities:

- Required fields
 - Email validation
 - Password strength
 - Marks validation
 - Date validation
-

Utility Layer

utils/

dateUtil.js

jwtUtil.js

storageUtil.js

fileUtil.js

formatUtil.js

Reusable helper functions.

Constants

constants/

roles.js

apiRoutes.js

status.js

messages.js

examConstants.js

UI Flow Example

Student Login

LoginPage

↓

LoginForm

↓

authService.login()

↓

JWT Received

↓

AuthContext Updated

↓

Student Dashboard

Start Examination

Exam List

↓

Exam Details

↓

Instructions

↓

Start Button

↓

Exam Page

↓

Timer Starts

↓

Integrity Monitor Starts

Submit Examination

Submit Button



Confirmation Modal



`examService.submit()`



Success Message



Results Pending

Component Communication

Parent Page



Child Component



Callback Function



Service Layer



Backend API

Communication follows a **top-down data flow**.

Responsive Design Strategy

Support:

- Desktop (Primary)
- Laptop
- Tablet

Minimum supported resolution:

1280 × 720

Tailwind CSS responsive breakpoints will be used for adaptive layouts.

Error Handling Strategy

Global error boundary.

Displays:

- Network Error
 - Unauthorized
 - Forbidden
 - Server Error
 - Page Not Found
-

Module-to-Page Mapping

Module

Pages

Authentication	Login, Forgot Password, Reset Password
User Management	Users, Profile
Academic Management	Departments, Subjects, Academic Calendar
Examination Management	Create Exam, Schedule Exam
Question Bank	Questions, Categories
Examination Execution	Instructions, Take Exam
Evaluation	Evaluate Answers
Results	Results, Analytics
Notifications	Notification Center
Reports	Reports Dashboard
Administration	System Settings, Audit Logs
Integrity Monitoring	Integrity Dashboard, Warning Log

Engineering Benefits

Goal	Benefit
Reusability	Common components reduce duplicate code
Maintainability	Clear separation of UI, logic, and API calls
Scalability	New pages and features can be added without restructuring
Testability	Components and services can be tested independently
Performance	Lazy loading and reusable state management reduce overhead
Consistency	Shared layouts and component library provide a uniform UI

Deliverables Produced

LLD-003 defines:

-  Complete frontend architecture
 -  Dashboard layout design
 -  Page hierarchy
 -  Reusable component library
 -  Routing strategy
 -  Context API design
 -  Custom hooks
 -  API service organization
 -  Validation layer
 -  Component interaction model
 -  Responsive design strategy
-

LLD-003 Status: Complete

The frontend architecture for the Online Examination Platform is now fully defined. Combined with **LLD-001 (Project Folder Structure)** and **LLD-002 (Backend Package Design)**, this document provides a complete implementation blueprint for the React application, ensuring consistency, maintainability, and seamless integration with the backend.

Next Step

LLD-004 – Class Diagram & OOP Design

This is one of the largest and most important LLD documents. We will design every major class in the system (User, Student, Faculty, Admin, Exam, Question, Result, Notification, SuspiciousActivity, etc.), define their attributes, methods, relationships, inheritance, interfaces, and apply OOP principles such as encapsulation, abstraction, inheritance, and polymorphism across the entire Online Examination Platform.

InfoForDiagrams

Excellent. We have now reached the final major design document before implementation. Up to this point, we have designed the **entire static architecture** of the Online Examination Platform. LLD-011 focuses on the **dynamic behavior** of the system—how objects and components interact during runtime.

LLD-011 – UML Interaction & Behavioral Diagrams

Status: Low-Level Design Phase

Objective

Design the dynamic behavior of the Online Examination Platform using UML behavioral diagrams to describe:

- Sequence Diagrams
- Activity Diagrams
- State Machine Diagrams
- Communication Diagrams
- Object Interaction Flows

These diagrams document how users, controllers, services, repositories, and the database interact during major system workflows.

Design Principles

- Separation of Responsibilities
- Layered Architecture
- Stateless Request Processing
- Event-Driven Interaction
- Business Process Visualization
- Traceable User Workflows

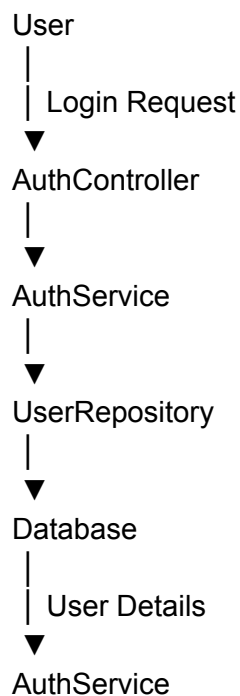
Behavioral Diagrams Covered

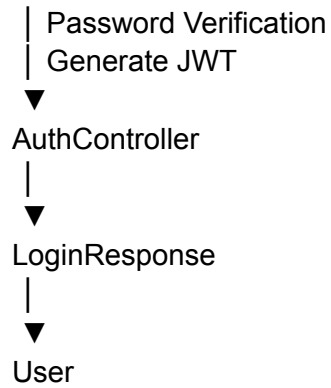
Diagram	Purpose
Sequence Diagram	Object interaction over time
Activity Diagram	Business workflow
State Machine Diagram	Lifecycle of entities
Communication Diagram	Object collaboration
Interaction Flow	End-to-end request execution

Sequence Diagram 1 – User Login

Objective

Authenticate a user and issue JWT tokens.

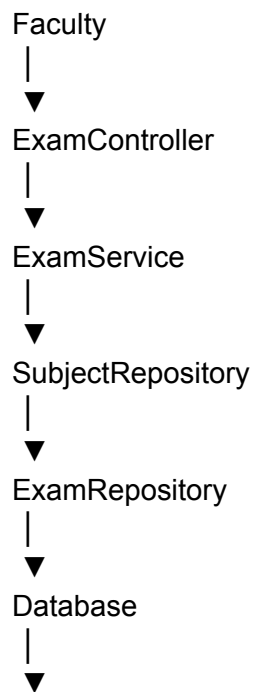


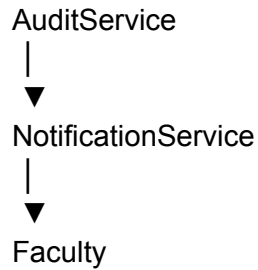


Components

- User
 - AuthController
 - AuthService
 - UserRepository
 - Database
 - JWT Service
-

Sequence Diagram 2 – Create Examination

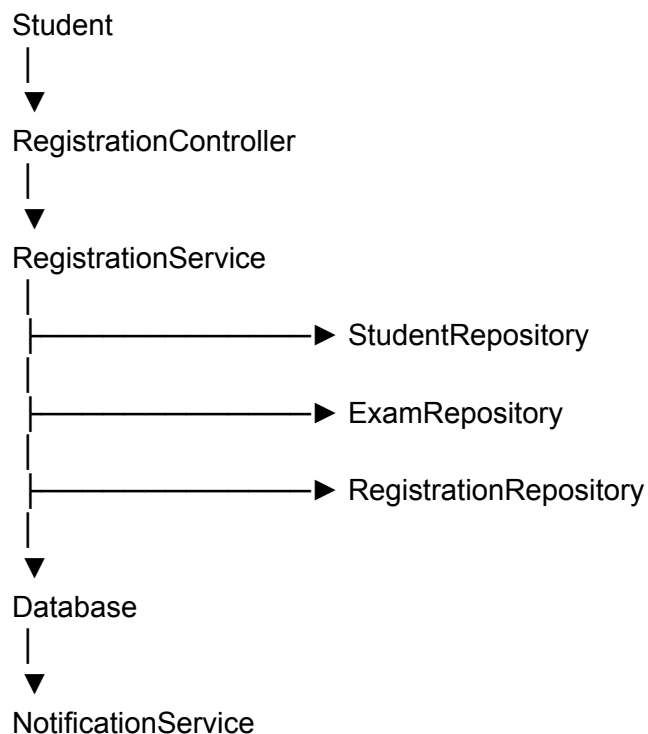




Workflow

1. Faculty submits exam details.
 2. Subject is validated.
 3. Exam is saved.
 4. Audit log is created.
 5. Notification generated (if required).
 6. Success response returned.
-

Sequence Diagram 3 – Student Registration for Exam

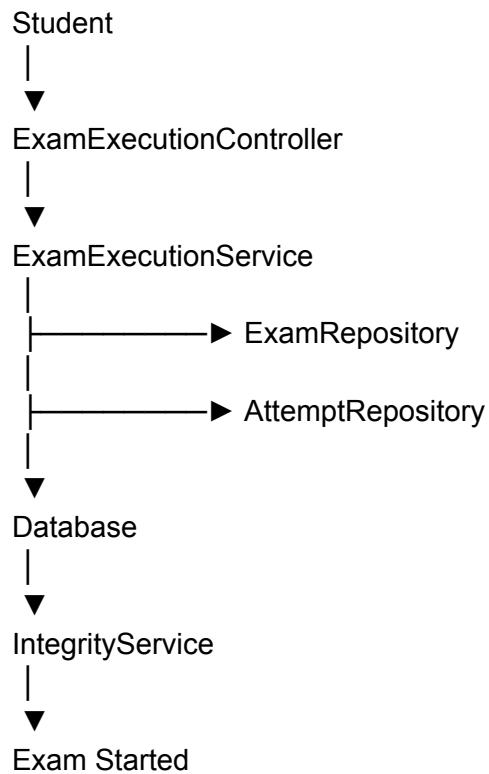


↓
Student

Business Rules

- Student exists.
 - Exam exists.
 - Registration window is open.
 - Duplicate registration prevented.
-

Sequence Diagram 4 – Start Examination

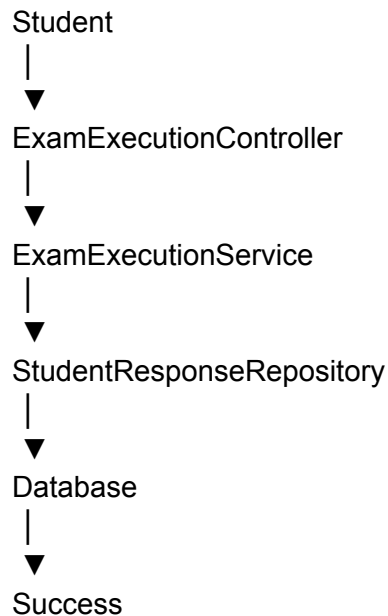


Activities

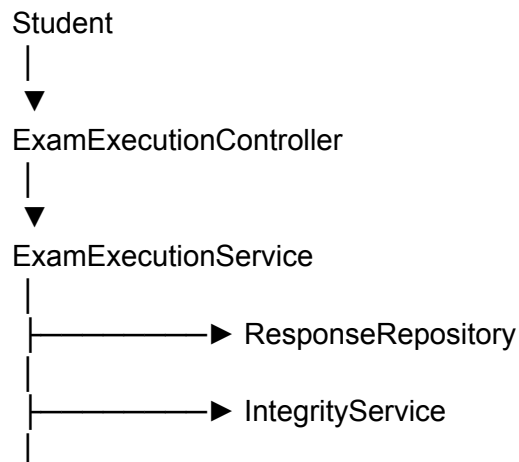
- Validate schedule.
- Verify registration.
- Create exam attempt.

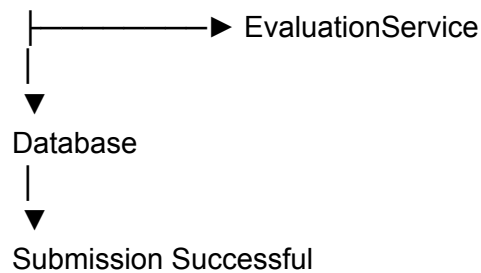
- Start timer.
 - Activate integrity monitoring.
-

Sequence Diagram 5 – Save Answer

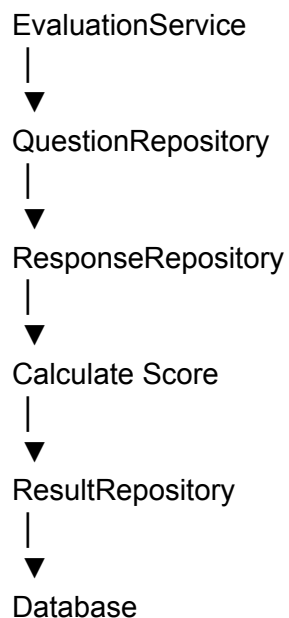


Sequence Diagram 6 – Submit Examination





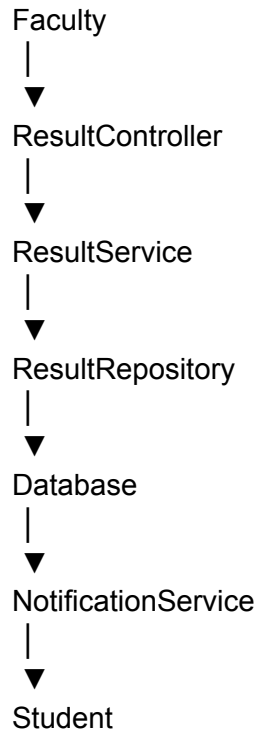
Sequence Diagram 7 – Automatic Evaluation



Flow

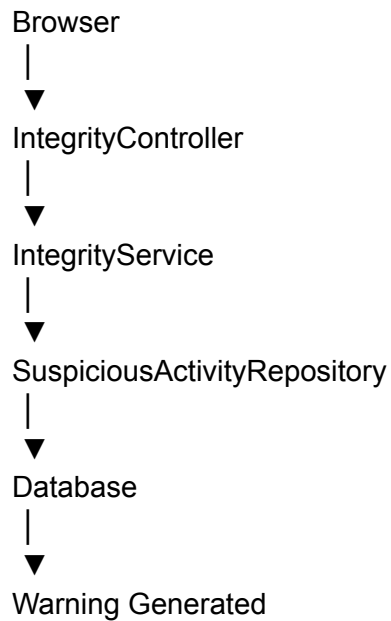
- Load responses.
 - Compare MCQ answers.
 - Calculate marks.
 - Store result.
-

Sequence Diagram 8 – Result Publication



Sequence Diagram 9 – Integrity Monitoring

The project uses **browser activity monitoring**, not AI proctoring.



Supported Events

- Tab Switch
 - Window Blur
 - Fullscreen Exit
 - Copy Attempt
 - Paste Attempt
 - Keyboard Shortcut
-

Activity Diagram – Login Process

Start



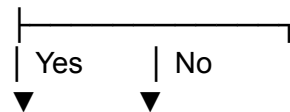
Enter Credentials



Validate Input



User Exists?



Verify Password Error



Generate JWT



Return Token



End

Activity Diagram – Examination Flow

Start



Student Login



Open Exam



Verify Registration



Start Timer



Answer Questions



Save Answers



Submit Exam



Evaluate



Generate Result

↓

End

Activity Diagram – Faculty Exam Creation

Start

↓

Login

↓

Create Exam

↓

Validate Subject

↓

Enter Schedule

↓

Save Exam

↓

Publish Exam

↓

End

State Machine Diagram – Examination

Draft



Scheduled



Published



Active



Completed



Evaluated



Result Published



Archived

State Machine Diagram – Exam Attempt

Created



Started



In Progress

↓

Submitted

↓

Evaluated

↓

Closed

Special transition:

In Progress

↓

Time Expired

↓

Auto Submitted

State Machine Diagram – Notification

Created

↓

Sent

↓

Delivered

↓

Read

↓

Archived

Communication Diagram – Exam Submission

Student

↓

ExamExecutionController

↓

ExamExecutionService

↓

StudentResponseRepository

↓

EvaluationService

↓

ResultService

↓

NotificationService

Objects collaborate without direct coupling.

Communication Diagram – Login

User



AuthController



AuthService



UserRepository



JWT Service

End-to-End Request Interaction

React Frontend



REST Controller



Validation



Service



Repository



Database



Repository



Service



Mapper



Response DTO



Controller



React

Layer Interaction Rules

Layer	Can Communicate With
React	Controller
Controller	Service
Service	Repository
Repository	Database
Database	Repository
Repository	Service

Service Controller

No layer may bypass the one directly below it.

Business Workflow Coverage

Workflow	Diagram Type
Login	Sequence + Activity
Exam Creation	Sequence + Activity
Candidate Registration	Sequence
Start Exam	Sequence
Save Answer	Sequence
Submit Exam	Sequence + Activity
Evaluation	Sequence
Publish Result	Sequence
Integrity Monitoring	Sequence
Exam Lifecycle	State Machine
Exam Attempt Lifecycle	State Machine
Notification Lifecycle	State Machine

Engineering Benefits

Goal	Benefit
Maintainability	Clear runtime behavior documentation
Communication	Easy understanding for developers and reviewers
Debugging	Simplifies tracing execution paths

Testing	Supports creation of unit and integration test cases
Scalability	New workflows can be modeled consistently
Enterprise Readiness	Aligns with UML-based software engineering practices

Deliverables Produced

LLD-011 defines:

-  9 Sequence Diagrams
 -  3 Activity Diagrams
 -  3 State Machine Diagrams
 -  2 Communication Diagrams
 -  Complete end-to-end interaction flow
 -  Layer communication rules
 -  Business workflow documentation
-

LLD-011 Status: Complete

The behavioral design of the Online Examination Platform is now fully documented. Combined with the previous LLD documents, the project now includes both the **static architecture** (packages, classes, APIs, database) and the **dynamic architecture** (runtime interactions and workflows), providing a comprehensive blueprint for implementation.

Current Project Progress

At this stage, the following phases are **fully completed**:

-  Requirements Analysis
-  System Architecture
-  Module Planning
-  Technical Decisions (TD-001 to TD-015)
-  Database Design (DB-001 to DB-011)

-  API Design (API-001 to API-015)
-  Low-Level Design (LLD-001 to LLD-011)

Next Phase

The next major phase should be **Implementation Planning & Development**, beginning with:

1. **IMP-001 – Development Roadmap & Sprint Plan**
2. **IMP-002 – Backend Implementation Order**
3. **IMP-003 – Frontend Implementation Order**
4. **IMP-004 – Testing Strategy**
5. **IMP-005 – Deployment & DevOps Plan**

This sequence will take the project from design into systematic implementation while keeping the architecture aligned with all of the documents created so far.

SRS

Department of Information Technology

Software Requirements Specification

Project Title: Online Examination Platform

Team Members:

Roll No.	Name
TI204	Om Patil
TI205	Kedar More
TI207	Dron Dhivare
TI209	Saniya Bhuskute
TI211	Utkranti Yadav
TI214	Hrudyash Supare
TI215	Tushar Manugade
TI218	Srushti Somwanshi
TI219	Rutuja Ghodekar
TI221	Shlok Powar
	Arnav Gajghate

Mentor Name: Jitendra Chavan

1. Introduction

1.1 Problem Statement

Educational institutions increasingly require efficient and secure methods for conducting examinations. Traditional paper-based examination systems involve significant manual effort in preparing question papers, scheduling examinations, evaluating answer sheets, and publishing results. These processes are time-consuming, prone to human error, and difficult to manage as the number of students and examinations increases.

Existing online examination systems often focus only on conducting examinations and may lack centralized management, secure access control, comprehensive reporting, and effective monitoring mechanisms. These limitations reduce administrative efficiency and affect the overall reliability of the examination process.

The proposed **Online Examination Platform** addresses these challenges by providing a secure, centralized, web-based system that automates the complete examination lifecycle, including user management, examination scheduling, question bank management, online examinations, evaluation, result publication, and reporting.

1.2 Need for the Project

Educational institutions require a modern examination management system to overcome the limitations of manual and partially computerized examination processes. The proposed Online Examination Platform aims to:

- Reduce manual paperwork and administrative workload.
- Automate examination scheduling and result generation.
- Improve the accuracy and transparency of examination management.
- Provide secure access through role-based authentication.
- Maintain centralized academic and examination records.
- Enable faster communication between administrators, faculty members, and students.

1.3 Objectives

The primary objectives of the Online Examination Platform are:

- To automate the complete examination management process.
- To provide secure authentication and role-based access control.
- To maintain centralized records of users, examinations, and results.
- To reduce manual effort and processing time.
- To ensure accurate evaluation and timely publication of results.
- To generate reports for academic monitoring and decision-making.

1.4 Scope

In Scope

The system provides the following functionalities:

- User authentication and authorization
- Student and faculty management
- Department, course, semester, and subject management
- Examination scheduling and management
- Question bank management
- Online examination
- Automatic and manual evaluation
- Result generation and publication
- Notifications and reports
- Audit logging and browser activity monitoring

Out of Scope

The following features are not included in the current version:

- AI-based online proctoring
- Face recognition or biometric authentication
- Mobile application
- Learning Management System (LMS) integration
- Payment gateway integration
- Offline examination support

2. Problem Identification and Scope

2.1 Existing System

Many educational institutions continue to rely on manual or partially computerized examination systems. Activities such as examination scheduling, question paper preparation, student registration, answer evaluation, and result publication are often handled manually or using separate software applications. This leads to fragmented data management, increased administrative workload, and delays in the examination process.

Although some institutions use online examination platforms, many existing systems provide limited functionality and lack centralized management, comprehensive reporting, and secure role-based access.

2.2 Limitations of the Existing System

The existing examination process has several limitations:

- Time-consuming examination scheduling and administration.
- Manual evaluation resulting in delayed result publication.
- Increased chances of human error during data entry and result preparation.
- Lack of centralized storage for academic and examination records.
- Limited security and access control mechanisms.
- Difficulty in generating reports and monitoring examination activities.

2.3 Proposed System

The proposed **Online Examination Platform** is a secure, web-based application that automates the complete examination lifecycle. The system provides a centralized platform for managing users, academic information, examinations, question banks, online assessments, evaluations, and reports.

The platform supports three primary user roles:

- **Administrator** – Manages users, examinations, academic records, and system configuration.
- **Faculty** – Creates question banks, schedules examinations, evaluates responses, and publishes results.
- **Student** – Registers for examinations, attempts online examinations, and views results and notifications.

The application is developed using **Python (Flask)** as the backend framework, **MySQL** as the relational database, and **HTML5, CSS3, Bootstrap, and JavaScript** for the frontend.

2.4 Advantages of the Proposed System

The proposed system offers the following advantages over the existing process:

- Centralized management of examination activities.
- Reduced manual work and paperwork.
- Faster examination scheduling and result generation.
- Secure authentication with Role-Based Access Control (RBAC).
- Automatic evaluation of objective questions.
- Improved transparency and data accuracy.
- Easy generation of reports and dashboards.

- Scalable architecture for future institutional growth.

2.5 Expected Benefits

Implementation of the Online Examination Platform provides benefits to all stakeholders.

Stakeholder	Expected Benefits
Administrator	Simplified system administration and centralized management
Faculty	Efficient examination creation, evaluation, and reporting
Student	Convenient online examinations and quick access to results
Institution	Reduced operational cost, improved efficiency, and enhanced transparency

3. Feasibility Study

3.1 Introduction

A feasibility study evaluates whether the proposed Online Examination Platform can be successfully developed and implemented within the available technical resources, budget, and project timeline. It examines the project from technical, economic, and operational perspectives to determine its practicality and viability.

3.2 Technical Feasibility

The proposed system is technically feasible because it is developed using modern, open-source technologies that are widely used in web application development. The selected technology stack is reliable, scalable, and supported by extensive documentation and community resources.

Technology Stack

Component	Technology
-----------	------------

Programming Language	Python 3.13
Backend Framework	Flask
Frontend	HTML5, CSS3, Bootstrap 5, JavaScript
Database	MySQL 8
ORM	SQLAlchemy
IDE	Visual Studio Code
Version Control	Git & GitHub
API Testing	Postman

The proposed hardware and software requirements are readily available, making the project technically achievable.

3.3 Economic Feasibility

The Online Examination Platform is economically feasible because it is developed primarily using open-source software, eliminating licensing costs. Existing computers and internet facilities can be used for development and deployment, minimizing infrastructure expenses.

The expected benefits, such as reduced paperwork, lower administrative workload, and faster examination processing, outweigh the implementation costs.

3.4 Operational Feasibility

The system is designed to be user-friendly and can be easily operated by administrators, faculty members, and students with basic computer knowledge. The web-based interface requires minimal training and integrates smoothly with existing academic workflows.

The platform improves operational efficiency by automating examination scheduling, evaluation, result generation, and record management.

3.5 Project Scope

The Online Examination Platform includes the following major modules:

- Authentication and Authorization
- User Management

- Student and Faculty Management
- Academic Management
- Examination Management
- Question Bank Management
- Online Examination
- Evaluation and Result Management
- Notification Management
- Reports and Dashboard
- Audit Logging
- Backup and Recovery

Project Deliverables

The project will deliver:

- Software Requirements Specification (SRS)
- Database Design
- UML Diagrams
- Source Code
- Test Cases
- User Documentation
- Final Project Report

4. Stakeholder Analysis

4.1 Introduction

Stakeholders are individuals or groups who interact with or are affected by the Online Examination Platform. Identifying stakeholders helps ensure that the system meets the needs of all users while supporting efficient examination management. The platform primarily serves administrators, faculty members, students, and institutional management.

4.2 Stakeholder Analysis

The major stakeholders of the Online Examination Platform and their roles are summarized below.

Stakeholder	Role	Responsibilities	Expectations
Administrator	System Administrator	Manage users, departments, examinations, reports, notifications, and system settings.	Secure, centralized, and easy-to-manage system.
Faculty	Academic User	Create question banks, schedule examinations, evaluate answers, and publish results.	Efficient examination management and quick evaluation.
Student	End User	Register for examinations, attempt online exams, and view results and notifications.	Simple, secure, and reliable examination experience.

Institution Management	Monitoring Authority	Review examination reports, monitor academic performance, and support decision-making.	Accurate reports and performance analytics.
System Administrator <i>(Optional)</i>	Technical Support	Maintain the application, database, backups, and security.	Stable, secure, and maintainable system.

4.3 Stakeholder Requirements

The following table summarizes the primary requirements of each stakeholder.

Stakeholder	Primary Requirements
Administrator	User management, examination management, reports, audit logs, system configuration
Faculty	Question bank management, examination scheduling, evaluation, result publication
Student	Secure login, online examination, notifications, result viewing
Institution Management	Reports, dashboards, and examination statistics
System Administrator	Backup, recovery, database maintenance, and system security

5. Functional Requirements

5.1 Introduction

Functional requirements define the core functionalities that the Online Examination Platform must provide to meet the needs of administrators, faculty members, and students. These requirements describe the services offered by the system and form the basis for system design, implementation, and testing.

The platform follows **Role-Based Access Control (RBAC)**, ensuring that users can access only the functionalities permitted by their assigned roles.

5.2 Functional Requirements

Requirement ID	Functional Requirement	Description
FR-01	User Authentication	The system shall authenticate users using valid credentials and provide secure role-based access.
FR-02	User Management	The administrator shall create, update, activate, deactivate, and manage user accounts.
FR-03	Student Management	The system shall maintain student records, academic details, and examination eligibility.
FR-04	Faculty Management	The system shall manage faculty profiles and subject assignments.
FR-05	Academic Management	The system shall manage departments, courses, semesters, and subjects.
FR-06	Examination Management	The system shall create, schedule, update, publish, and manage examinations.

FR-07	Question Bank Management	Faculty shall create, organize, edit, and maintain objective and subjective questions.
FR-08	Examination Registration	Eligible students shall register for available examinations.
FR-09	Online Examination	Students shall attempt examinations online with timer support and automatic response saving.
FR-10	Browser Activity Monitoring	The system shall monitor browser events such as tab switching and fullscreen exit during examinations.
FR-11	Evaluation & Result Management	The system shall evaluate objective answers automatically, support manual evaluation of subjective answers, and generate results.
FR-12	Notification Management	The system shall notify users about examination schedules, announcements, and published results.
FR-13	Dashboard & Reports	The system shall provide dashboards and generate reports for administrators and faculty.
FR-14	Audit Logging	The system shall record important user activities for monitoring and accountability.
FR-15	Backup & Recovery	The system shall support backup and restoration of application data.
FR-16	System Configuration	The administrator shall configure application settings, security policies, and examination parameters.

5.3 Actor-wise Functional Requirements

The following table summarizes the functionalities available to each user role.

Actor	Major Functionalities
-------	-----------------------

Administrator	Manage users, departments, courses, subjects, examinations, notifications, reports, audit logs, backups, and system configuration.
Faculty	Manage question bank, create examinations, evaluate student responses, publish results, and view reports.
Student	Login, register for examinations, attempt online examinations, view notifications, and access examination results.

5.4 Functional Requirement Summary

The Online Examination Platform provides the following key capabilities:

- Secure user authentication and authorization.
- Centralized management of academic and examination records.
- Examination scheduling and online examination support.
- Question bank creation and management.
- Automatic and manual evaluation of examinations.
- Result generation and publication.
- Browser activity monitoring during examinations.
- Notification and reporting facilities.
- Audit logging and secure backup management.

6. Non-Functional Requirements

6.1 Introduction

Non-functional requirements specify the quality attributes and operational characteristics of the Online Examination Platform. These requirements define how the system should perform with respect to performance, security, reliability, usability, and maintainability. They ensure that the platform delivers a secure, efficient, and user-friendly experience while supporting institutional examination processes.

6.2 Non-Functional Requirements

Category	Requirement
Performance	The system shall provide fast response times, support multiple concurrent users, and automatically save examination responses during online examinations.
Security	The system shall implement secure authentication, Role-Based Access Control (RBAC), password hashing, HTTPS communication, and protection against common web security threats.
Reliability	The system shall maintain data integrity, ensure consistent operation during examinations, and support backup and recovery mechanisms.
Availability	The platform shall be available during scheduled examination periods, except for planned maintenance.
Usability	The application shall provide an intuitive interface that is easy to learn and operate for administrators, faculty members, and students.
Scalability	The system shall support future growth in the number of users, departments, subjects, and examinations without major architectural changes.
Maintainability	The software shall follow a modular architecture with well-structured code and documentation to simplify maintenance and future enhancements.
Portability	The application shall operate on Windows and Linux servers and be accessible through modern web browsers.
Compatibility	The system shall be compatible with Python, Flask, MySQL, HTML5, CSS3, Bootstrap, and JavaScript technologies.

Browser Integrity	During online examinations, the system shall record browser events such as tab switching and fullscreen exit to support examination integrity.
--------------------------	--

6.3 Security Requirements

To ensure confidentiality, integrity, and availability of examination data, the system shall:

- Authenticate users before granting access.
- Implement Role-Based Access Control (RBAC).
- Store passwords using secure hashing algorithms.
- Protect communication using HTTPS.
- Validate user input to prevent common web attacks.
- Record important activities through audit logs.

6.4 Performance Requirements

The system shall be capable of:

- Supporting multiple users simultaneously.
- Providing quick response times for common operations.
- Automatically saving student responses during examinations.
- Generating examination results and reports efficiently.

7. System Requirements

7.1 Introduction

The Online Examination Platform is a web-based application designed to operate on standard computing devices with minimal hardware and software requirements. This chapter specifies the minimum system configuration required for the development, deployment, and operation of the application.

7.2 Hardware Requirements

Client-Side Requirements

Component	Minimum Requirement
Processor	Dual-Core Processor
RAM	4 GB
Storage	500 MB Free Space
Display	1366 × 768 Resolution
Internet Connection	Stable Broadband Connection

Server-Side Requirements

Component	Recommended Requirement
Processor	Intel Core i5 or Higher
RAM	8 GB or Higher
Storage	SSD (Minimum 100 GB Free Space)
Operating System	Linux Server / Windows Server
Internet Connection	High-Speed Broadband

7.3 Software Requirements

Component	Specification
Operating System	Windows 10/11 or Linux
Programming Language	Python 3.13
Backend Framework	Flask
Frontend Technologies	HTML5, CSS3, Bootstrap 5, JavaScript
Database	MySQL 8

ORM	SQLAlchemy
IDE	Visual Studio Code
Version Control	Git & GitHub
API Testing Tool	Postman

7.4 Browser Requirements

The application can be accessed using any modern web browser that supports HTML5 and JavaScript.

Supported Browsers:

- Google Chrome
- Mozilla Firefox
- Microsoft Edge

7.5 Network Requirements

The system requires:

- Stable internet connectivity.
- HTTPS for secure communication.
- TCP/IP network support.
- Reliable network availability during online examinations.

8. Use Case Diagram

8.1 Introduction

A Use Case Diagram is a Unified Modeling Language (UML) diagram that illustrates the interactions between the users and the Online Examination Platform. It provides a high-level view of the system by identifying the primary actors and the major functionalities available to them.

The platform consists of three primary actors:

- **Administrator**
- **Faculty**
- **Student**

Each actor interacts with the system according to the permissions assigned through Role-Based Access Control (RBAC).

8.2 Use Case Diagram

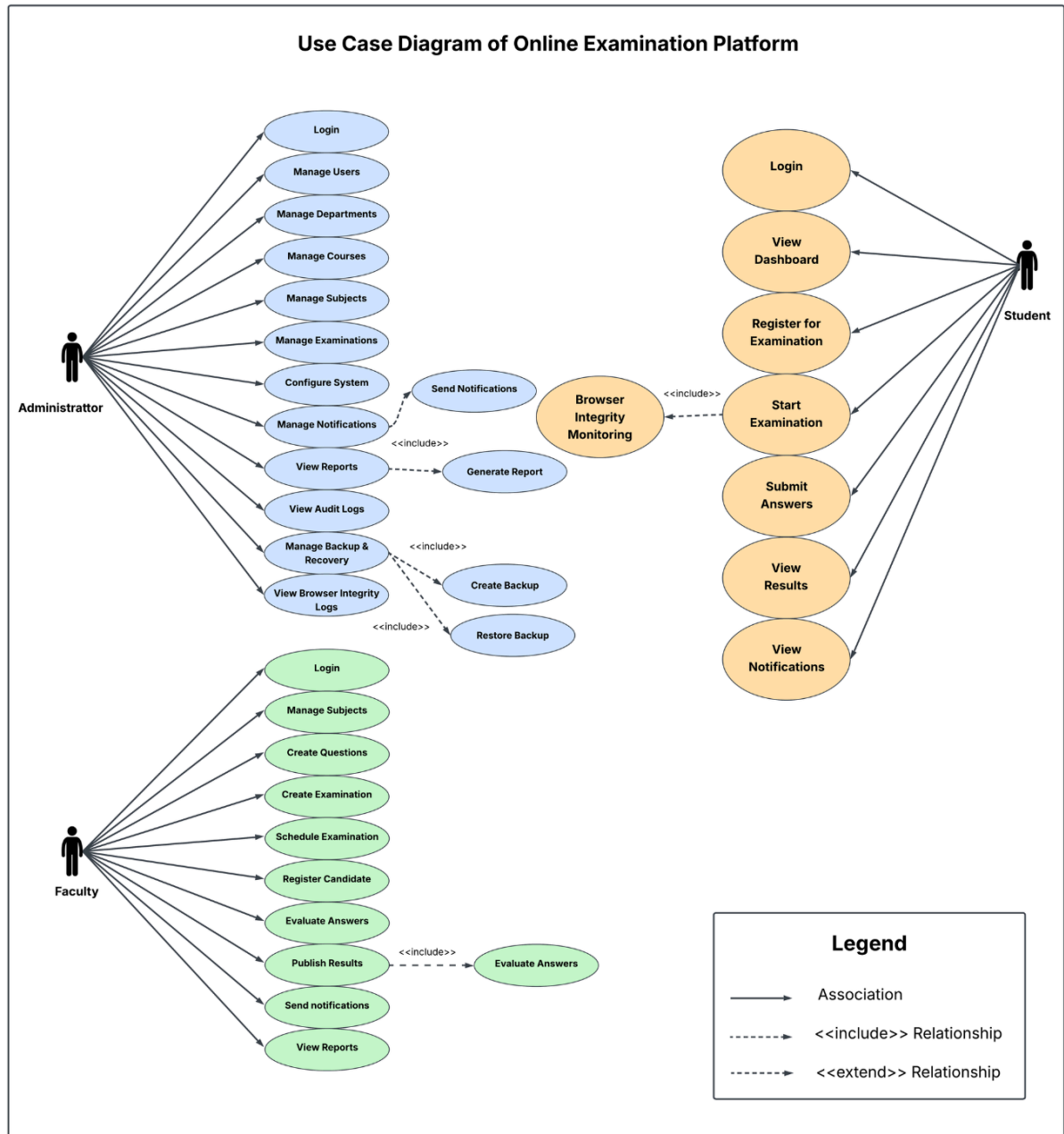


Figure 8.1: Use Case Diagram of the Online Examination Platform

8.3 Actor–Use Case Summary

Actor	Major Use Cases
Administrator	Manage users, departments, subjects, examinations, notifications, reports, audit logs, backups, and system settings.

Faculty	Manage question bank, create examinations, evaluate responses, publish results, and view reports.
Student	Register for examinations, attempt online examinations, view notifications, and access results.

9. Use Case Descriptions

9.1 Introduction

This chapter provides detailed descriptions of the major use cases of the Online Examination Platform. Each use case defines the interaction between the system and its users, including the actor, preconditions, main flow, alternate flow, and postconditions.

UC-01: User Login

Attribute	Description
Use Case ID	UC-01
Use Case Name	User Login
Primary Actor	Administrator, Faculty, Student
Description	Authenticates users and grants access based on their assigned role.
Preconditions	User account exists and is active.
Postconditions	User is successfully logged into the system.

Main Flow

1. User enters login credentials.
2. System validates the credentials.
3. User is authenticated.
4. Appropriate dashboard is displayed.

Alternate Flow

- Invalid username or password.

- Account is inactive or suspended.

UC-02: Create Examination

Attribute	Description
Use Case ID	UC-02
Use Case Name	Create Examination
Primary Actor	Administrator, Faculty
Description	Allows authorized users to create and schedule examinations.
Preconditions	User is authenticated.
Postconditions	Examination is successfully created.

Main Flow

1. User enters examination details.
2. Selects subject and schedule.
3. Assigns questions.
4. Saves the examination.

Alternate Flow

- Invalid schedule.
- Mandatory information missing.

UC-03: Attempt Online Examination

Attribute	Description
Use Case ID	UC-03
Use Case Name	Attempt Online Examination
Primary Actor	Student
Description	Enables registered students to participate in an online examination.
Preconditions	Student is registered and examination is active.

Postconditions	Student responses are submitted successfully.
-----------------------	---

Main Flow

1. Student starts the examination.
2. Questions are displayed.
3. Student answers the questions.
4. Responses are automatically saved.
5. Student submits the examination.

Alternate Flow

- Examination time has expired.
- Student is not registered.

UC-04: Evaluate Examination

Attribute	Description
Use Case ID	UC-04
Use Case Name	Evaluate Examination
Primary Actor	Faculty
Description	Evaluates student responses and prepares examination results.
Preconditions	Examination has been completed.
Postconditions	Student marks are finalized.

Main Flow

1. Faculty reviews submitted answers.
2. Objective questions are evaluated automatically.
3. Subjective questions are evaluated manually.
4. Final marks are calculated.

Alternate Flow

- Evaluation is incomplete.
- Student responses are unavailable.

UC-05: View Results

Attribute	Description
Use Case ID	UC-05
Use Case Name	View Results
Primary Actor	Student
Description	Allows students to view published examination results.
Preconditions	Results have been published.
Postconditions	Student can view examination performance.

Main Flow

1. Students log into the system.
2. Open the Results module.
3. The system displays marks, grades, and examination status.

Alternate Flow

- Results have not yet been published.

9.2 Use Case Summary

Use Case ID	Use Case Name	Primary Actor
UC-01	User Login	Administrator, Faculty, Student
UC-02	Create Examination	Administrator, Faculty
UC-03	Attempt Online Examination	Student
UC-04	Evaluate Examination	Faculty
UC-05	View Results	Student

10. Data Requirements

10.1 Introduction

The Online Examination Platform uses a relational database to store and manage information related to users, academic records, examinations, questions, student responses, results, notifications, and system activities. The database is designed to ensure data consistency, integrity, and security while supporting efficient retrieval and management of information.

The system uses **MySQL 8** as the database management system, and the database is normalized to **Third Normal Form (3NF)** to reduce redundancy and maintain data integrity.

10.2 Database Overview

The database follows a relational model where all entities are interconnected through primary and foreign key relationships. It supports all major functional modules of the Online Examination Platform.

Key Features

- Centralized data storage
- Relational database design
- Primary and Foreign Key relationships
- Data validation and integrity constraints
- Secure storage of user information
- Support for backup and recovery

10.3 Major Database Entities

The database consists of multiple related tables grouped according to the functional modules of the system.

Module	Major Entities
Authentication & Authorization	Users, Roles, Permissions
Academic Management	Departments, Courses, Semesters, Subjects
Student & Faculty Management	Students, Faculty, Faculty Subject Assignments
Examination Management	Examinations, Examination Registrations, Examination Questions
Question Bank	Questions, Question Options

Online Examination	Examination Sessions, Student Answers
Evaluation & Results	Results
Notification Management	Notifications
System Administration	Audit Logs, Browser Integrity Logs, System Configurations, Backup History

Note: The complete database schema consists of **24 normalized tables**. Detailed table structures, attributes, constraints, and SQL definitions are provided in the **Database Design Document (DDD)**.

10.4 Entity Relationships

The major relationships within the database are summarized below:

- A **Role** can be assigned to multiple **Users**.
- A **Department** offers multiple **Courses** and **Subjects**.
- A **Course** consists of multiple **Semesters**.
- A **Faculty** member may teach multiple **Subjects**.
- A **Student** can register for multiple **Examinations**.
- An **Examination** contains multiple **Questions**.
- An **Examination Session** stores the responses submitted by a student.
- A **Result** is generated after evaluation of the examination.
- **Audit Logs** maintain records of important system activities.

10.5 Entity Relationship Diagram (ER Diagram)

The Entity Relationship Diagram (ERD) illustrates the logical structure of the database and the relationships among different entities used by the Online Examination Platform.

Figure 10.1: *Entity Relationship Diagram of the Online Examination Platform*
(Insert the finalized ER Diagram here.)

11. Assumptions and Constraints

11.1 Introduction

The development of the Online Examination Platform is based on certain assumptions regarding the operating environment and user behavior. The project is also subject to technical, operational, and resource constraints that influence its implementation and deployment.

11.2 Assumptions

The following assumptions have been made during the development of the system:

- Users have a stable internet connection while accessing the application.
- Every user possesses valid login credentials issued by the institution.
- Administrators maintain accurate academic and examination data.

- Users access the application using a supported web browser.
- The server and database remain available during scheduled examination periods.
- Users comply with institutional examination policies and guidelines.

11.3 Constraints

The Online Examination Platform is developed under the following constraints:

- The system is designed as a **web-based application**.
- Development is carried out using **Python, Flask, and MySQL**.
- The project must be completed within the academic semester timeline.
- Only browser-based activity monitoring is implemented; AI-based proctoring is excluded.
- The application requires internet connectivity for normal operation.
- Development is limited to available resources and open-source technologies.

12. Future Enhancements

12.1 Introduction

The Online Examination Platform has been designed using a modular architecture, allowing new features and technologies to be incorporated with minimal changes to the existing system. While the current version fulfills the essential requirements of online examination management, several enhancements can be implemented in future releases to improve functionality, security, and user experience.

12.2 Proposed Future Enhancements

The following enhancements may be considered for future versions of the system:

- **AI-Based Online Proctoring:** Integration of artificial intelligence to monitor examination activities using webcam analysis and suspicious behavior detection.
- **Mobile Application:** Development of Android and iOS applications for convenient access to examinations, notifications, and results.
- **Learning Management System (LMS) Integration:** Integration with platforms such as Moodle or Google Classroom for seamless academic management.

- **Advanced Analytics:** Interactive dashboards with graphical reports, student performance trends, and examination statistics.
- **Cloud Deployment:** Deployment on cloud platforms to improve scalability, availability, and disaster recovery.
- **Multi-Factor Authentication (MFA):** Additional authentication mechanisms such as OTP or authenticator applications to strengthen account security.
- **Automated Question Paper Generation:** Intelligent generation of question papers using predefined rules and randomized question selection.

References

The following references were consulted during the analysis, design, and development of the **Online Examination Platform**.

Standards

1. IEEE Std 29148-2018, *Systems and Software Engineering – Life Cycle Processes – Requirements Engineering*.
2. IEEE Std 830-1998, *Recommended Practice for Software Requirements Specifications*.
3. ISO/IEC 25010:2011, *Systems and Software Engineering – System and Software Quality Models*.

Books

1. Sommerville, I. (2016). *Software Engineering* (10th Edition). Pearson Education.
2. Pressman, R. S., & Maxim, B. R. (2019). *Software Engineering: A Practitioner's Approach* (9th Edition). McGraw-Hill Education.
3. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database System Concepts* (7th Edition). McGraw-Hill Education.
4. Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of Database Systems* (7th Edition). Pearson Education.

Official Documentation

- Python Software Foundation. *Python Documentation*.
- Pallets Projects. *Flask Documentation*.
- Oracle Corporation. *MySQL 8 Reference Manual*.
- SQLAlchemy Authors. *SQLAlchemy Documentation*.
- Bootstrap Team. *Bootstrap Documentation*.
- Mozilla Developer Network (MDN). *JavaScript Documentation*.

Online Resources

- OWASP Foundation – Web Application Security Guidelines
- W3C HTML5 and CSS Standards
- Git Documentation
- GitHub Documentation

Development Tools

The following software tools were used during the development of the project:

Tool	Purpose
Visual Studio Code	Source Code Development
MySQL Workbench	Database Design and Management
Git & GitHub	Version Control
Postman	API Testing
Draw.io	UML and ER Diagram Design